A Lean 4 Formalization of Scott's *Continuous Lattices* (1972)

Lars Warren Ericson[1]

[1]Catskills Research Company
[1]https://github.com/catskillsresearch/scott1972
[1]lars.ericson@catskillsresearch.com

June 29, 2026

ORCID: 0000-0001-8299-9361
Primary Category: cs.LO (Logic in Computer Science)
Secondary Category: math.LO (Logic)

Abstract

We present a complete machine-checked formalization of Dana Scott's landmark 1972 paper *Continuous Lattices* **[Sco72]**, carried out in Lean 4 against mathlib and including the March 1972 Milner correction in **[Sco72]** (pp. 135–136).

Scott's paper develops a model for λ-calculus from a topological starting point. He defines *injective* T_0-spaces—those with a strong extension property for continuous maps—and shows that they are exactly the *continuous lattices*: complete lattices whose Scott topology is determined by the order via the way-below relation ($\ll$). On this foundation he studies projections, retractions, products, function spaces, and inverse limits. The capstone (Theorem 4.4) constructs an inverse limit D_∞ of function-space approximants and proves $D_\infty \cong [D_\infty \to D_\infty]$, yielding a purely mathematical model for Church's untyped λ-calculus.

Our development formalizes **43 numbered results** from Scott's §1–§4 (Propositions, Corollaries, Lemmas, and Theorems), each as a sorry-free Lean theorem, together with supporting infrastructure (step functions, the $\Uparrow$`a` basis of Scott opens, Milner's coarser-than-Scott hypothesis, the function-space tower, and the i_∞/j_∞ pair). The formalization is **classical** (uses `Classical.choice` transitively) and follows Scott's proof dependency order. Where the Lean proof required choices not visible in the original—or where dead ends were encountered—we record detailed notes in §5. All proofs check with the standard footprint `[propext, Classical.choice, Quot.sound]`.

---

1 Introduction

Scott's 1972 paper *Continuous Lattices* is his first fully detailed, peer-reviewed publication of the D_∞ model for the semantics of Church's untyped λ-calculus—but with one crucial historical nuance: **the model was a complete accident**. While the 1972 paper is the landmark account, Scott had been trying to prove that such a model was mathematically *impossible*.

Three factors frame the breakthrough:

1.1 The goal was types, not the untyped calculus

In the late 1960s Dana Scott worked alongside Christopher Strachey at Oxford. Scott was a skeptic of Alonzo Church's untyped λ-calculus. He believed programming languages should be

strictly typed and argued that the untyped calculus lacked a legitimate mathematical foundation. He began developing a model for λ-calculus specifically to provide a denotational semantics for *typed* languages.

1.2 The discovery of D_∞ (1969)

To model the untyped λ-calculus, a space D must be isomorphic to its own function space ($D \cong [D \to D]$). In standard set theory, Cantor's theorem makes this size-wise impossible: the power of a function space strictly exceeds the size of the base set.

In November 1969, while attempting to formalize why this restriction made untyped models impossible, Scott realized that if one restricts to Scott-continuous functions (those preserving directed suprema) rather than all arbitrary functions, the space does not explode in size. By constructing an inverse limit of algebraic lattices ($D_0 \to D_1 \to D_2 \to \cdots$), he built D_∞, the first non-degenerate, purely mathematical model of the untyped λ-calculus.

1.3 Chronology of the papers

Although the definitive mathematical breakdown appeared in 1972, Scott's "first attempts" span a few tightly knit manuscripts:

- **1969 (unpublished manuscript): [Sco69]** *Lattice-theoretic models for the λ-calculus*—the literal first write-up distributed among colleagues right after the November discovery.
- **1970 (conference paper): [Sco70]** *Outline of a mathematical theory of computation*—a brief, high-level introductory account.
- **1972 (published paper): [Sco72]** *Continuous Lattices*—prepared as a technical report in 1971; recognized as the landmark paper that gave the mathematical and computer science communities the D_∞ model.

Scott's paper opens by arguing that T_0-spaces, long treated as a mere exercise in separation axioms, are natural once one cares about function spaces and extension properties rather than geometry. That shift—from typed skepticism to the accidental D_∞ model—is the backdrop for the formalization in §3–§5.

The development documented below follows **[Sco72]** through the March 1972 Milner correction. §2 summarizes Scott's original paper; §3–§4 describe the Lean development and catalog the formalized theorems; §5 records proof notes where the mechanization adds detail beyond the published text.

---

2 Scott's *Continuous Lattices*

[Sco72] develops a model for λ-calculus from injective T_0-spaces. Scott's own abstract states the arc of the paper: starting topologically, he introduces spaces with a strong extension property for continuous maps; shows they are exactly the continuous lattices—complete lattices whose topology is the Scott topology determined by the order; studies projections, subspaces, embeddings, products, and function spaces; and proves the main result that one can embed every space in a continuous lattice D_∞ that is homeomorphic (and order-isomorphic) to its own function space $[D_\infty \to D_\infty]$, yielding models for the Church–Curry λ-calculus.

Scott organizes the paper in four technical sections (following an introductory §0):

Scott §	Title	Main content
§1	**Injective spaces**	Definition of injectivity; $\mathbb{O}$ and its powers; retract characterization (Cor. 1.6–1.7)
§2	**Continuous lattices**	Way-below ($\ll$), Scott topology, Scott-continuous maps; products and retractions; injectivity $\Longleftrightarrow$ continuous lattice (Thm. 2.12)
§3	**Function spaces**	$[D \to D']$ as a continuous lattice (Thm. 3.3); λ-abstraction, evaluation, projections, fixed points
§4	**Inverse limits**	D_∞ as inverse/direct limit; capstone $D_\infty \cong [D_\infty \to D_\infty]$ (Thm. 4.4)

Our working source text is `sources/ScottContinLatt1972.md`: a plain-text OCR transcription of **[Sco72]** through the Milner correction (pp. 135–136). Image-based PDFs are poor inputs for mechanized proof development; this file gives the formalization a reliable, searchable text to quote against. (An earlier filename suffix `_vision` reflected the OCR toolchain only and has been dropped to avoid confusion with the published paper.)

---

3 The Lean formalization

3.1 Scope and methodology

The Lean development lives under `Scott1972/ContinuousLattice/` (root import `Scott1972.lean`). We track Scott's numbered statements: each row in §4 corresponds to a named theorem in the repository, proved sorry-free. Results not separately numbered by Scott but required for later steps—such as `wayBelow_interpolate`, `exists_wayBelow_subset`, and the Milner infrastructure—appear as supporting lemmas in the module map below.

Milner infrastructure (March 1972 correction): `CoarserThanScottTopology`, `scottOpen_of_coarserThanScott`, `scottLowerSubbasisSet`, `scottPrincipalUpSet` in `MilnerCorrection.lean`.

Notation: ⊔`S′` denotes the ambient join in D' (`ambientSSup`); ⊔`S` the subspace join; `j(⊔S′) = ⊔S` is `retr_ambientSSup_eq_sSup`.

3.2 Constructivity

The formalization is **classical**. It uses mathlib topology, `Classical.choice` (transitively), embedding into Sierpiński powers, and order-theoretic arguments that have not been audited for constructivity. Every completed proof reports `#print axioms` as `[propext, Classical.choice, Quot.sound]`.

3.3 Module map

Scott §	Title	Lean modules
§1	**Injective spaces**	`Injective.lean`
§2	**Continuous lattices**	`WayBelow.lean`, `Specialization.lean`, `ScottMaps.lean`, `Constructions.lean`, `MilnerCorrection.lean`
§3	**Function spaces**	`FunctionSpaces.lean`
§4	**Inverse limits**	`InverseLimits.lean`, `FunctionSpaceTower.lean`

---

4 Catalog of formalized results

We formalize **43** numbered results from Scott §1–§4. Each entry is a full statement matching Scott's numbering, proved without `sorry`. Theorem 4.4 is split into four subgoals **(a)**–**(d)** in both Scott and Lean.

Supporting keystones (not separately numbered by Scott): `directedOn_wayBelow`, `wayBelow_interpolate` (interpolation for $\ll$, **axiom-free**), and `exists_wayBelow_subset` (the $\uparrow a$ basis of the Scott topology) in `WayBelow.lean`; these underpin Proposition 2.11.

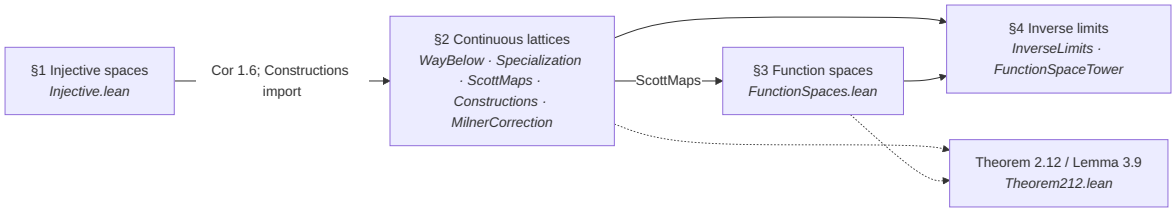
§	Scott	Lean identifier(s)	Module
1	Prop 1.2	`proposition_1_2`	`Injective.lean`
1	Prop 1.3	`proposition_1_3`	`Injective.lean`
1	Prop 1.4	`proposition_1_4`	`Injective.lean`
1	Prop 1.5	`proposition_1_5`	`Injective.lean`
1	Cor 1.6	`corollary_1_6`	`Injective.lean`
1	Cor 1.7	`corollary_1_7`	`Injective.lean`
2	Prop 2.1	`proposition_2_1`	`Specialization.lean`
2	Prop 2.2	`bot_wayBelow`, `WayBelow.sup`, `WayBelow.trans_le`, `WayBelow.le_trans`, `wayBelow_self_iff_scottOpen_Ici`, `wayBelow_sSup_iff`	`WayBelow.lean`
2	Prop 2.4	`isContinuousLattice_iff_isLUB_sInf_nhds`	`WayBelow.lean`
2	Prop 2.5	`proposition_2_5`	`ScottMaps.lean`
2	Prop 2.6	`proposition_2_6`	`ScottMaps.lean`
2	Prop 2.8	`proposition_2_8`	`Constructions.lean`
2	Prop 2.9(a)	`proposition_2_9_a`	`Constructions.lean`
2	Prop 2.9(b)	`proposition_2_9_b`, `proposition_2_9`	`Constructions.lean`
2	Prop 2.10(a)	`proposition_2_10_a`	`FunctionSpaces.lean`
2	Prop 2.10(b)	`proposition_2_10_b`, `proposition_2_10`	`FunctionSpaces.lean`

§	Scott	Lean identifier(s)	Module
2	Prop 2.11	proposition_2_11	Constructions.lean
2	Thm 2.12	theorem_2_12, theorem_2_12_backward, theorem_2_12_forward	Theorem212.lean
3	Prop 3.2	proposition_3_2	FunctionSpaces.lean
3	Thm 3.3(a)	theorem_3_3_isContinuousLattice, ScottMap.instCompleteLattice, stepMap, stepMap_wayBelow, stepMap_pointwise_sSup	FunctionSpaces.lean
3	Thm 3.3(b)	theorem_3_3_topology, theorem_3_3, wayBelow_le_finset_sup_step, pointwiseSubbasic_scottOpen	FunctionSpaces.lean
3	Cor 3.4	corollary_3_4_jointly_continuous, corollary_3_4_preservesDirectedSup, corollary_3_4	FunctionSpaces.lean
3	Prop 3.5	proposition_3_5, scottLambda, curry_left/right_preservesDirectedSup, lambda_outer_preservesDirectedSup	FunctionSpaces.lean
3	Prop 3.7	proposition_3_7_retraction, proposition_3_7_projection	FunctionSpaces.lean
3	Prop 3.8	proposition_3_8, scottExtend_maximal, continuous_eq_sSup_openInfs	Constructions.lean
3	Lemma 3.9	lemma_3_9, scottExtend_maximal_le	Theorem212.lean
3	Prop 3.10	incl_sSup, incl_injective, incl_wayBelow, proposition_3_10_converse, retr_eq_sSup	FunctionSpaces.lean

§	Scott	Lean identifier(s)	Module
3	Prop 3.12	`proposition_3_12`, `IsProjection`, `isProjection_sSup`, `Projections.instCompleteLattice`	`FunctionSpaces.lean`
3	Prop 3.13	`proposition_3_13`, `Proposition313.projection`	`FunctionSpaces.lean`
3	Prop 3.14	`proposition_3_14`, `Proposition314.fixMap`, `fix_eq`, `fix_le`, `fix_unique`	`FunctionSpaces.lean`
4	Prop 4.1	`proposition_4_1`, `InverseLimit`, `inverseLimitRetraction`	`InverseLimits.lean`
4	Prop 4.2	`proposition_4_2`, `embInf`, `projInf`, `iComp`, `embInf_succ`, `inverseLimit_eq_iSup`	`InverseLimits.lean`
4	Cor 4.3	`corollary_4_3`, `coconeInf`, `coconeInf_comp_embInf`	`InverseLimits.lean`
4	Lemma 4.5	`lemma_4_5`, `idInf_eq_iSup`	`InverseLimits.lean`
4	Thm 4.4(a)	`embInfInf`, `projInfInf`, `iInfTerm`, `jInfTerm`, `*_apply`, `*_preservesDirectedSup`	`FunctionSpaceTower.lean`
4	Thm 4.4(b)	`projInfInf_comp_embInfInf`	`FunctionSpaceTower.lean`
4	Thm 4.4(c)	`embInfInf_comp_projInfInf`	`FunctionSpaceTower.lean`
4	Thm 4.4(d)	`theorem_4_4`, `theorem_4_4_orderIso`	`FunctionSpaceTower.lean`

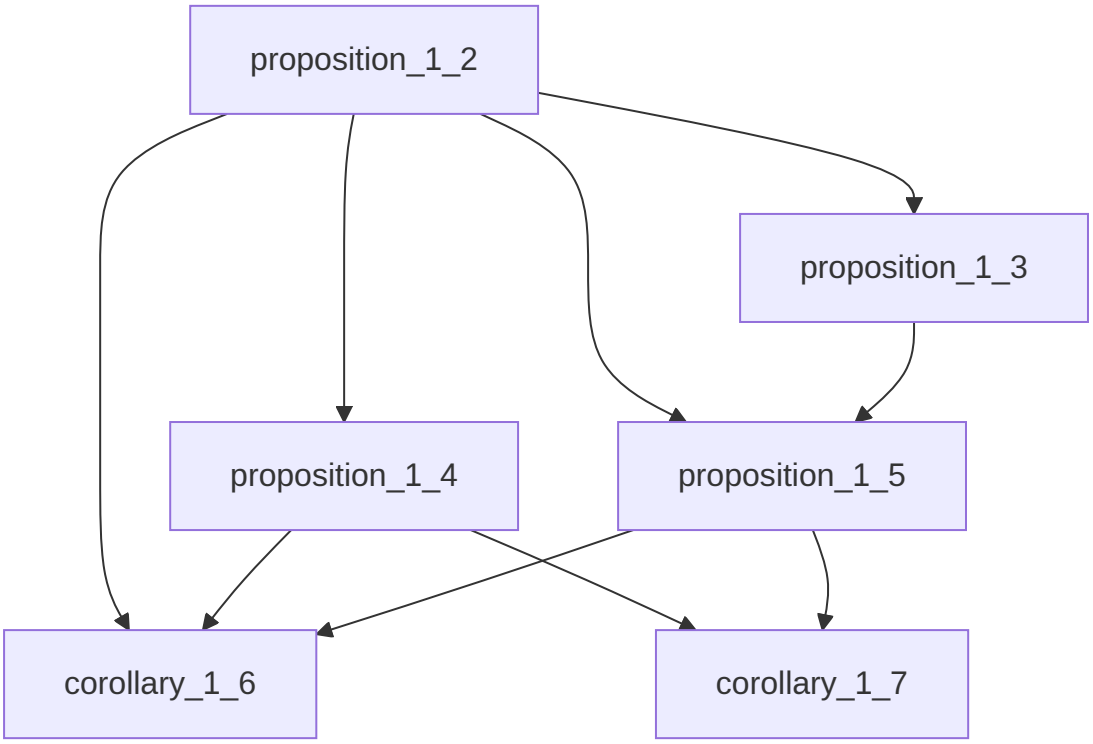
4.1 Proof dependency structure

Scott §1–§4 are not independent modules; the Lean import graph follows Scott's exposition order. Note that Propositions 2.10(a)–(b) live in `FunctionSpaces.lean` (§3 module) even though Scott numbers them in §2; `Theorem212.lean` bridges §2 and §3 for Theorem 2.12 and Lemma 3.9.

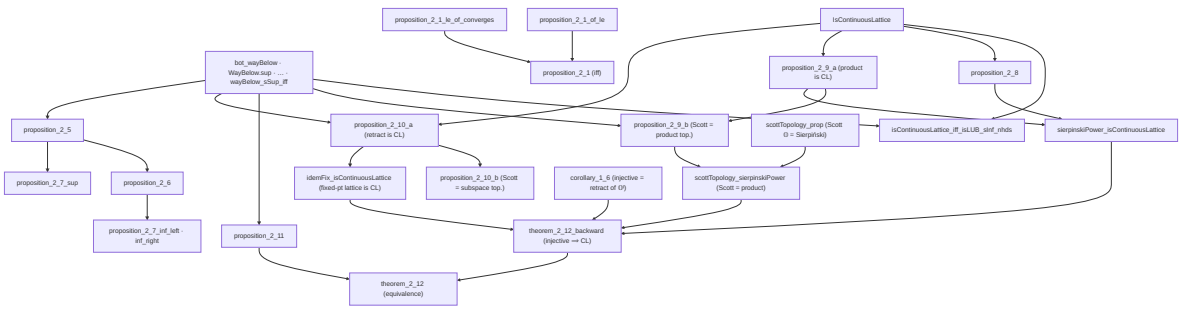


4.2 §1 Injective spaces — result hierarchy

The six results of §1 form a short chain from the Sierpiński space $\mathbb{O}$ to the retract characterization of injectivity.



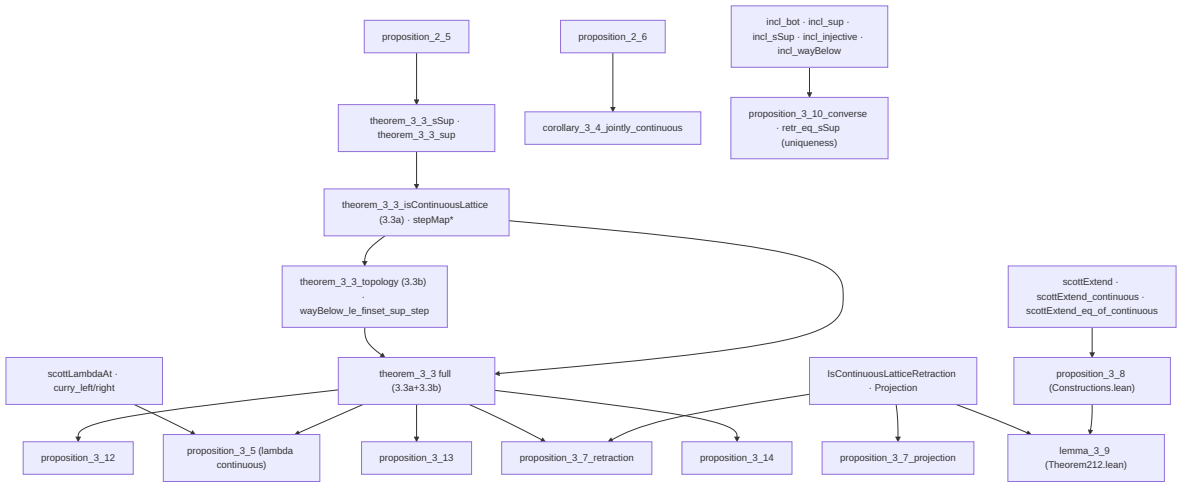
4.3 §2 Continuous lattices — result hierarchy



4.4 §3 Function spaces — result hierarchy

Proposition 2.7 is proved in `ScottMaps.lean` but not tracked separately in §4; it feeds Proposition 2.6's infrastructure. Propositions 2.10(a)–(b) are proved in `FunctionSpaces.lean`. Propo-

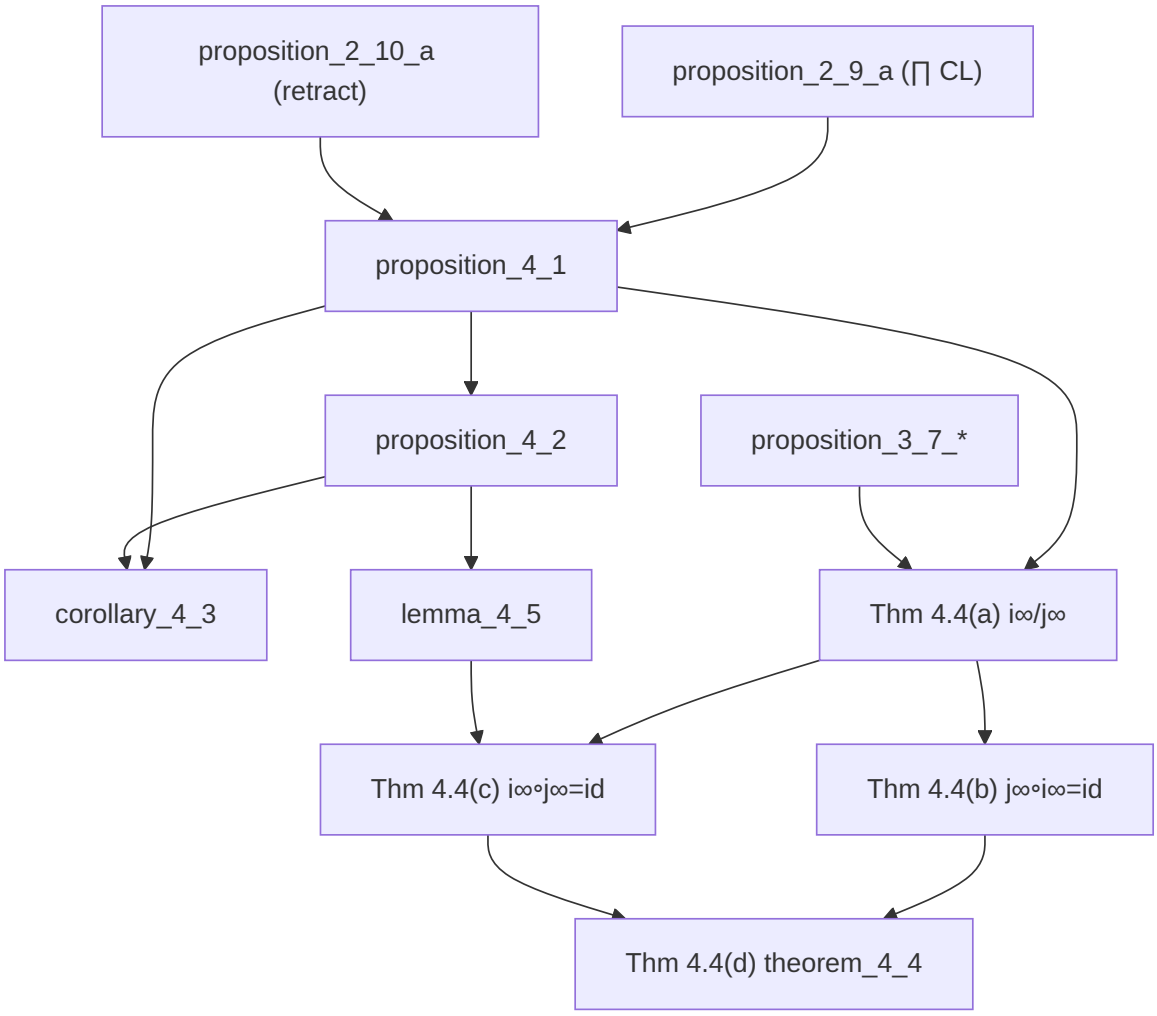
sition 3.2 is formalized but is not on the critical path to later results.



4.5 §4 Inverse limits — result hierarchy

Scott §4 is complete in Lean: Propositions 4.1–4.2, Corollary 4.3, Lemma 4.5, and Theorem 4.4 **(a)**–**(d)**. See §5.3 for proof notes on the capstone.

The Lean proof of Proposition 4.1 uses the order-theoretic adjoint route (Props 2.9a and 2.10a), not Scott's injectivity route through Propositions 3.8 and Lemma 3.9. Lemma 4.5 enters Theorem 4.4(c); Lemma 3.9 is not used in the capstone proof.



---

5 Notes on the formalization

This section records proof strategy, Lean engineering choices, and lessons learned where the mechanization goes beyond Scott's published text. Results in §1 follow Scott's short arguments directly and are not discussed here. Proposition 2.1 is split in Lean as `proposition_2_1_of_le` and `proposition_2_1_le_of_converges`, then bundled as `proposition_2_1`.

5.1 Continuous lattices (Scott §2)

Section §2 is where most of the order-topology alignment work lives: products (2.9), retractions (2.10), injectivity (2.11), and the equivalence theorem (2.12) all required careful handling of Scott topologies without registering competing instances.

5.1.1 Proposition 2.6 (joint ↔ separate continuity) — `proposition_2_6`

Scott states that a function of several variables between complete lattices is continuous jointly if and only if it is continuous in each variable separately. We formalize the two-variable case `f : D × D' → D''`, with continuity phrased as `PreservesDirectedSup` (justified by Prop 2.5), and the product `D × D'` carrying the componentwise complete-lattice structure (whose induced topology is the product topology). The proof follows Scott's directed-net argument:

- **Joint $\Longrightarrow$ separate.** Precompose `f` with the slice map `x ↦ (x, y)`. The image of a directed `S ⊆ D` under this map is directed in `D × D'` with least upper bound `(⊔S, y)` (computed componentwise via `Prod.fst_sSup` / `Prod.snd_sSup`, using `S` nonempty for the constant second coordinate). Joint preservation of that supremum therefore yields preservation in the first variable; the second variable is symmetric.
- **Separate $\Longrightarrow$ joint** (the substance). For directed `S* ⊆ D × D'`, project to the directed sets `S = fst '' S*` and `S' = snd '' S*` (directedness via `DirectedOn.fst` / `DirectedOn.snd`), so that `⊔S* = (⊔S, ⊔S')`. Then:
  - `⊔(f '' S*) ≤ f(⊔S*)` is immediate from monotonicity of `f` (assembled from the separate monotonicities `hmono1`, `hmono2`).
  - `f(⊔S*) ≤ ⊔(f '' S*)`: unfolding separate continuity twice gives `f(⊔S*) = ⊔_{x∈S} ⊔_{y∈S'} f(x, y)`; for each pair `x ∈ S`, `y ∈ S'` there exist witnesses `(x, b), (a, y) ∈ S*`, and **directedness of** `S*` supplies `r ∈ S*` above both, so `(x, y) ≤ r` and `f(x, y) ≤ f(r) ≤ ⊔(f '' S*)` by monotonicity. This is exactly Scott's "monotonicity + directedness" step.

Sorry-free; `#print axioms` gives `[propext, Classical.choice, Quot.sound]` (the standard classical footprint throughout this development).

5.1.2 Proposition 2.8 (finite lattices are continuous) — `proposition_2_8`

Scott states this as a one-line example. The Lean proof isolates the genuinely finite step in a reusable lemma `directedOn_finite_sSup_mem`: *a non-empty finite directed set attains its supremum* (`⊔S ∈ S`). A maximal element `m ∈ S` exists by `Set.Finite.exists_maximal`; by directedness any `s ∈ S` and `m` have an upper bound `c ∈ S`, and maximality forces `c ≤ m`, so `s ≤ m`. Hence `m` is the greatest element, `IsLUB S m`, and `⊔S = m ∈ S`. With this, every principal up-set `Set.Ici y` is Scott-open (a directed `S` with `y ≤ ⊔S` has `⊔S ∈ S`), so `y ≪ y` via `wayBelow_self_iff_scottOpen_Ici`, and `y` is trivially the supremum of `{x | x ≪ y}`. `[Finite D]` suffices (subsets are finite via `Set.toFinite`).

5.1.3 Proposition 2.9 (products of continuous lattices) — `proposition_2_9_a`, `proposition_2_9_b`

Scott's Proposition 2.9 is a **conjunction** of an order-theoretic and a topological claim, so we split it: `proposition_2_9_a` (the product is a continuous lattice), `proposition_2_9_b` (the Scott topology of the product equals the product of the Scott topologies), and the bundled `proposition_2_9 := ⟨a, b⟩`.

2.9(a) — order content (`proposition_2_9_a`). A product `∀ i, E i` of continuous lattices is a continuous lattice. The construction is the cylinder element: for `a ≪ y i` in factor `E i`, let `[a]^i := Function.update ⊥ i a`. Then `[a]^i ≪ y` in the product, witnessed by the preimage `{z | z i ∈ U}` of a Scott-open `U ⊆ E i` with `y i ∈ U ⊆ Ici a`: this set is an upper set, and inaccessible because suprema are coordinatewise (`sSup_apply_eq_sSup_image`), so a directed `S` with `(⊔S) i ∈ U` already has some `f ∈ S` with `f i ∈ U`. Given any upper bound `b` of `{x | x ≪ y}`, each `[a]^i ≤ b` gives `a = ([a]^i) i ≤ b i`; ranging over `a ≪ y i` and using continuity of `E i` (`(hE i).sSup_wayBelow`) yields `y i ≤ b i` for all `i`, i.e. `y ≤ b`.

2.9(b) — topology agreement (`proposition_2_9_b`). We prove the *full equality* of topologies `scottTopologicalSpace = Pi.topologicalSpace (fun _ => scottTopologicalSpace)` by `le_antisymm`; no Milner-style coarseness hypothesis is needed. Working with explicit topology terms (`E i` carries no `TopologicalSpace` instance) keeps us clear of the `specializationPreorder` diamond, and the mathlib order `t₁ ≤ t₂` unfolds *definitionally* to `∀ U, IsOpen[t₂] U → IsOpen[t₁]`

U. - **Product ⊆ Scott** (scott ≤ ⋂ᵢ induced (eval i)): each projection preserves directed suprema (sSup_apply_eq_sSup_image), hence is Scott-continuous (continuous_of_preservesDirectedSup); le_iInf + continuous_iff_le_induced finish. - **Scott ⊆ Product**: for a Scott-open U ∋ z the ⇑a basis (exists_wayBelow_Ici_subset, the Ici-strengthening of exists_wayBelow_subset) gives a ≪ z with ↑a ⊆ U. Three new structural lemmas about way-below in a product do the rest: wayBelow_proj (a ≪ z ⟹ aᵢ ≪ zᵢ, via the preimage under v ↦ Function.update z i v, Scott-open by update_preservesDirectedSup) and wayBelow_finite_support (a ≪ z has finite support: the truncations Z F = (if · ∈ F then z· else ⊥) are directed with sup z, so a ≤ Z F for some finite F). The finite box ⋂_{i∈F} eval i ⁻¹' Vᵢ (with Vᵢ ∋ zᵢ Scott-open inside Ici aᵢ) is product-open (isOpen_biInter_finset of induced-opens, each ≥ the product topology by iInf_le) and lies in ↑a ⊆ U (off F, aⱼ = ⊥ ≤ wⱼ; on F, aᵢ ≤ wᵢ).

classical supplies the DecidableEq for Function.update; footprint [propext, Classical.choice, Quot.sound] for all of 2.9(a)/(b).

Engineering notes / lessons from 2.9(b). This was the hardest single proof in the development; recording the dead-ends so the next session does not re-pay the cost):

- *Avoid letI for the factor/product topologies.* The tempting move is letI : ∀ i, TopologicalSpace (Eᵢ) := fun _ => scottTopologicalSpace so that mathlib's Pi.topologicalSpace, continuous_apply, isOpen_biInter_finset, … resolve by instance. But our imports make specializationPreorder an active instance, so a TopologicalSpace (Eᵢ) in scope introduces a **second** Preorder (Eᵢ) that fights the CompleteLattice one — the same diamond that broke scottExtend_eq_of_continuous earlier. Keeping every topology an **explicit term** (@Pi.topologicalSpace …, @IsOpen _ scottTopologicalSpace …) and never registering an instance is what makes the proof go through. The order reasoning (way-below, sSup, finite support) lives in *instance-free* lemmas (wayBelow_proj, wayBelow_finite_support) precisely so they never see a competing topology.

- *Use the definitional unfolding of the topology order.* TopologicalSpace.le_def shows t₁ ≤ t₂ **is** ∀ U, IsOpen[t₂] U → IsOpen[t₁] U (the partial order's le field), so intro U hU works directly on a P ≤ S goal and iInf_le _ i _ hopen turns an induced-open into a product-open with no le_def rewrite or IsOpen.mono lemma. This is the single most useful fact for product/Scott topology bridges.

- *Prefer Set.Ici a ⊆ U over ⇑a ⊆ U.* exists_wayBelow_subset actually proves the stronger Set.Ici a ⊆ U (the witness a lies in the upper-set U), so the new exists_wayBelow_Ici_subset lets the box-containment step ask only for a ≤ w instead of a ≪ w. This **eliminates the way-below ⟸ characterization** (componentwise-≪ + finite-support ⟹ product-≪) entirely — a large, fiddly Finset.sup-of-cylinders argument we would otherwise have needed.

- *Finite support falls out of the truncations, not a separate axiom.* a ≪ z plus the directed family Z F = (if · ∈ F then z· else ⊥) (sup z) gives a ≤ Z F for some finite F via wayBelow_sSup_iff; then aⱼ ≤ (Z F)ⱼ = ⊥ off F. No independent "way-below ⟹ finite support" theorem is required.

- *@-argument order is worth checking empirically.* isOpen_biInter_finset autobinds as @isOpen_biInter_finset X α [inst] s f h (space first, index second); isOpen_induced_iff needs the codomain topology, supplied painlessly by the named argument (t := scottTopologicalSpace) rather than a positional @. When in doubt, feed one wrong argument and read the "expected type" in the error to recover the true order.

- *Beta-reduce before rw.* PreservesDirectedSup f unfolds to f (sSup T) = … with f a literal lambda, so the goal is (fun v => update z i v) (sSup T) j; a Function.update_

`self` rewrite only matches after a `show` (or `dsimp only`) forces the beta reduction to `Function.update z i (sSup T)`.

5.1.4 Proposition 2.10 (a retract of a CL is a CL) — `proposition_2_10_a`, `proposition_2_10_b`

Like 2.9, Scott's 2.10 bundles an order claim and a topology claim; we split it as `proposition_2_10_a` / `proposition_2_10_b` with the bundled `proposition_2_10`. A *retract* is the existing `IsContinuousLatticeRetraction D D'`: Scott maps `i : D → D'`, `j : D' → D` with `j ∘ i = id`. We take `D'` continuous and conclude both halves for `D`.

The single engine is `retr_wayBelow_of_wayBelow_incl`: `x' ≪ i(d) in D' ⟹ j(x') ≪ d in D`. Witness the `D`-way-below by `i⁻¹V'` for an ambient Scott-open witness `V'` of `x' ≪ i(d)` (`i⁻¹V'` is Scott-open since `i` preserves directed sups, `scottOpen_preimage`); for `z ∈ i⁻¹V'`, `x' ⊑ i(z)` gives `j(x') ⊑ j(i(z)) = z`. With `sSup_image_retr_wayBelow` (`d = ⊔_D {j(x') : x' ≪ i(d)}`, from `j(⊔'S') = ⊔S` + continuity of `D'`): - **2.10(a).** Any upper bound `b` of `{x | x ≪ d}` dominates every `j(x')`, hence the supremum `d`. (`IsLUB` is immediate.) - **2.10(b).** `scott = induced i scott'`. The easy `scott ≤ induced` is `scottOpen_preimage` again. The hard `induced ≤ scott` (Milner) shows the family `{i⁻¹(⇑x') : x' ∈ D'}` is a **basis** of `D`'s Scott topology: given Scott-open `U ∋ d`, the directed family `{j(x') : x' ≪ i(d)}` (sup `d`) meets `U` at some `j(x')`, and `i⁻¹(⇑x') ⊆ U` because `x' ≪ i(z) ⟹ j(x') ⊑ z` and `U` is upper. Each `i⁻¹(⇑x')` is induced-open by construction, so every Scott-open is a union of induced-opens, i.e. induced-open.

Engineering notes / lessons from 2.10:

- *No projection, no Milner hypothesis needed.* Scott proves 2.10 for general retractions and only needs *projections* later (for the function-space 3.7/3.9). The whole proof goes through with the bare `IsContinuousLatticeRetraction` (Scott maps + `j ∘ i = id`); `incl_retr_le` is never used. And, as with 2.9(b), the topology agreement is a genuine equality — `CoarserThanScottTopology` does not appear. The Milner subtlety ("lubs in the subspace are *larger*, so a relativised open need not be lattice-open") is dissolved by the retraction: `j(⊔S') = ⊔S` realigns the inequality.

- *Reuse the abstract structure instead of building a `CompleteLattice` on a subtype.* The tempting faithful reading — fixed points `{x // j x = x}` of an idempotent Scott map, with transported joins `sSup_K S = j(⊔' i''S)` — forces a hand-built `CompleteLattice` instance (every axiom, then continuity, then topology) and is several hundred lines. Phrasing the retract as *its own* lattice `D` with Scott maps to/from `D'` captures exactly the same content (`i` preserving directed sups **is** the statement that `D`-joins are `j` of ambient joins) at a fraction of the cost.

- *`isOpen_induced_iff` needs the codomain topology pinned.* $\mathtt{E_i}$/`D'` carry no `TopologicalSpace` instance, so `rw [isOpen_induced_iff]` fails instance synthesis; supply `(t := scottTopologicalSpace)` (same trick as 2.9(b)).

- *`scottOpen_preimage` is the workhorse.* "Preimage of a Scott-open under a Scott map is Scott-open" appears three times here (the way-below witness, and both topology inclusions). Packaging `incl_preservesDirectedSup : PreservesDirectedSup ⇑i` once keeps the call sites clean.

This unblocks the **backward half of Theorem 2.12** (injective ⟹ CL) at the *retract* level; the embedding of an injective space into a power of $\mathbb{O}$ (1.6) supplies the rest, and 2.12 is now **complete** (see the Theorem 2.12 note below).

5.1.5 Keystones for 2.11: interpolation and the ⇑a basis — `WayBelow.lean`

Two standard facts about $\ll$ that mathlib does not provide and that the capstone needs:

- **Interpolation** (`wayBelow_interpolate`): in a continuous lattice `a ≪ c ⟹ ∃ b, a ≪ b ≪ c`. The set `M = {m | ∃ x, m ≪ x ∧ x ≪ c}` is directed (apply directedness of `{· ≪ x}` twice) with `⊔M = c` (continuity twice); then `a ≪ c = ⊔M` forces `a ≪ m ≤ x ≪ c` for some `m ≪ x ≪ c`, so `b := x`. Notably this is **axiom-free** (`#print axioms` reports none).
- **⇑a basis** (`exists_wayBelow_subset`): every Scott-open `U ∋ z` contains a basic neighbourhood `⇑a = {w | a ≪ w}` with `a ≪ z`. Since `z = ⊔{a | a ≪ z}` is a directed sup in the open `U`, inaccessibility yields `a ≪ z` with `a ∈ U`, and `⇑a ⊆ ↑a ⊆ U`.

5.1.6 Proposition 2.11 (continuous lattices are injective) — `proposition_2_11`

The substantial half of Theorem 2.12. The witness is an explicit operator `scottExtend e f y = ⊔ { ⊓ f''(e⁻¹V) : V an open nbhd of y }` (a standalone `def`, purely order-theoretic). Two lemmas about it:

- **Extends f** (`scottExtend_eq_of_continuous`). The $\leq$ bound is immediate (`f x₀` is one of the values met). For $\geq$, continuity of the lattice is essential: for each `a ≪ f x₀`, the Scott-open `⇑a` pulls back along the continuous `f`, and the **embedding** turns that into an open `V ⊆ Y` with `e⁻¹V = f⁻¹(⇑a)`; on `e⁻¹V`, `f ≥ a`, so `a ≤ ⊓ f''(e⁻¹V) ≤ g(e x₀)`. Summing over `a ≪ f x₀` (continuity) gives `f x₀ ≤ g(e x₀)`.
- **Continuous** (`scottExtend_continuous`). Uses the `⇑a` basis: for Scott-open `U` and `g y₀ ∈ U` pick `a ≪ g y₀` with `⇑a ⊆ U`; as `g y₀` is a directed sup, `a ≪ ⊓ f''(e⁻¹V)` for some open `V ∋ y₀`, and that value is `≤ g y'` for all `y' ∈ V`, so `V ⊆ g⁻¹U`.

A Lean-specific wrinkle: `E` carries no global `TopologicalSpace` instance (its topology is `scottTopologicalSpace`), so lemmas like `IsOpen.preimage` that *synthesize* `[TopologicalSpace E]` fail. The order-heavy `scottExtend_eq_of_continuous` uses `continuous_def` (whose topology arguments are ordinary implicits, unified from the hypothesis) to avoid both the synthesis failure and the specialization-order diamond a `letI` would introduce; the purely topological `scottExtend_continuous` and `proposition_2_11` use `letI : TopologicalSpace E := scottTopologicalSpace`. Footprint `[propext, Classical.choice, Quot.sound]`.

5.1.7 Theorem 2.12 (injective ⟺ continuous lattice) — `theorem_2_12`, `theorem_2_12_backward` (`Theorem212.lean`)

Both directions are now closed; `theorem_2_12` is the full biconditional:

> A T_0-space is injective **iff** it is homeomorphic to a continuous lattice under its Scott topology.

- **Forward** (CL $\Longrightarrow$ injective) is `theorem_2_12_forward` (= 2.11).
- **Backward** (injective $\Longrightarrow$ CL) is `theorem_2_12_backward`. The argument:
  1. By Corollary 1.6, an injective T_0-space `D` is a *retract* of a Sierpiński power `L = ι → 𝕆` (`𝕆 = Prop`): there are continuous `s : D → L`, `r : L → D` with `r ∘ s = id`.
  2. `L` is a continuous lattice (`sierpinskiPower_isContinuousLattice`, from 2.8 + 2.9a) whose Scott topology *is* its product topology (`scottTopology_sierpinskiPower`, from 2.9b plus `scottTopology_prop`: the Scott topology on $\mathbb{O}$ is the Sierpiński topology).

3. `e := s ∘ r` is therefore a **Scott-continuous idempotent** on `L`. Its fixed-point set `IdemFix e` carries the ambient-supremum-corrected complete-lattice structure (`IdemFix.completeLattice`), is a continuous lattice by Proposition 2.10 (`idemFix_isContinuousLattice`), and `d ↦ s d` is a homeomorphism `D ≃ₜ IdemFix e`.

Engineering notes / lessons from 2.12:

- *Fixed points of a monotone idempotent are a complete lattice* for free via `completeLatticeOfSup`: take `sSup_K S = e (sSup_L (val '' S))` and `sInf` derived. No closure/kernel (`e ≤ id` or `e ≥ id`) hypothesis is needed — only monotone + idempotent — and Scott-continuity of `e` is what makes the inclusion/corestriction `ScottMap`s, so the retract machinery of 2.10 applies verbatim.
- *The subtype-topology trap.* `IdemFix e = {x : L // e x = x}` is reducibly a subtype of `L`, so it **auto-inherits the subspace** `TopologicalSpace`, which competes with the Scott topology coming from its (non-instance) `CompleteLattice`. This breaks `Continuous.comp`/`subtype_mk` (they synthesize the *subspace* instance, not Scott). The fix: build the homeomorphism against the canonical subspace topology (where those lemmas work), then transport across the propositional equality `scott = subspace` — itself `idemFix_scottTopology` (= `induced val scott_L`) composed with `scottTopology_sierpinskiPower` (`scott_L = product`), closing by `rfl`.
- *Statement shape.* Endowing an abstract injective space with a lattice is impossible literally, so the faithful statement is "homeomorphic to a continuous lattice under its Scott topology"; the reverse arrow transfers injectivity across the homeomorphism via `IsInjectiveSpace.of_retract`.
- Footprint `[propext, Classical.choice, Quot.sound]`.

5.2 Function spaces (Scott §3)

Section §3 builds the function-space lattice, proves agreement with pointwise convergence (Theorem 3.3), and develops the projection and fixed-point infrastructure needed for §4.

5.2.1 Theorem 3.3(a) (`[D → D']` is a continuous lattice) — `theorem_3_3_isContinuousLattice` (`FunctionSpaces.lean`)

Scott's "pointwise" argument, in three movements.

1. **Complete lattice on `[D → D']`.** `ScottMap D D'` is a genuine `def` (a subtype of `D → D'`), so — unlike the `IdemFix` subtype trap of 2.12 — it carries *no* auto-synthesized order/-topology to fight. We register `instPartialOrder` (pointwise ≤), `instSupSet` (`sSupMaps F x = ⊔{g x | g ∈ F}`, which is itself a `ScottMap` because pointwise suprema of Scott maps preserve directed sups), prove `isLUB_sSup`, and close with `completeLatticeOfSup`. Crucially `sSup` here is *pointwise* (`sSup_apply` is `rfl`), matching Scott's observation that **arbitrary** (not just directed) joins are computed pointwise — while infima are *not* (derived as ⊔ of lower bounds by `completeLatticeOfSup`).
2. **Step functions.** `ē[e,e'](x) = e'` if `e ≪ x` else ⊥, encoded as `⋃ _ : e ≪ x, e'` (`stepFun`) to dodge any `Decidable (e ≪ x)`. Scott-continuity of `stepFun` is exactly the Scott-openness of the way-above set `{x | e ≪ x}` (`scottOpen_wayBelow`, true in *any* complete lattice): inaccessibility of that open set supplies the member of a directed `S` realizing the value.

3. **Way-below + reconstruction.** `e' ≪ f e ⟹ ē[e,e']  ≪ f`, witnessed by the Scott-open `{g | e' ≪ g e}` (open because joins are pointwise, so inaccessibility reduces to `wayBelow_sSup_iff` in `D'`); this is the **topological** way-below of `WayBelow.lean`, so we never need an order-theoretic ≪-characterization. And `f x = ⊔{e' | ∃ e ≪ x, e' ≪ f e}` (`stepMap_pointwise_sSup`) follows from `x = ⊔{e ≪ x}` (continuity of `D`), `f` preserving that directed sup, and `f x = ⊔{w ≪ f x}` (continuity of `D'`) + `wayBelow_sSup_iff`. Continuity of `[D → D']` then drops out: any upper bound `g` of `{h ≪ f}` dominates every `ē[e,e'] ≪ f`, hence pointwise `e' ≤ g x`, hence `f x = ⊔{...} ≤ g x`.

Engineering notes / lessons from 3.3(a):

- *Register the lattice as a real instance.* Because `ScottMap` is a plain `def`, a global `CompleteLattice` instance is safe and makes ≪, `ScottOpen`, and `IsContinuousLattice` typecheck on the function space with no `@`/`letI` gymnastics — the opposite experience to the `IdemFix` subtype.
- `⋃ _ : p, a` *is the clean "indicator".* It is `a` when `p` holds (`iSup_pos`) and ⊥ otherwise (`iSup_neg`), needs no `Decidable`, and commutes with the proofs cleanly.
- *Topological ≪ is an asset, not a burden here.* Proving `ē[e,e'] ≪ f` by exhibiting one Scott-open set is shorter than any directed-set argument; the lattice's pointwise `sSup` makes its inaccessibility immediate.
- Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.2 Theorem 3.3(b) (lattice topology = pointwise-convergence topology) — `theorem_3_3_topology` (`FunctionSpaces.lean`)

The function space carries two topologies: the Scott topology of the continuous lattice `[D → D']` (from `ScottMap.instCompleteLattice`) and the product/pointwise-convergence topology `scottMapPointwiseTopology` generated by `{f | f x ∈ U}` (`U` Scott-open in `D'`). They are equal.

- **pointwise ⊆ Scott** (`le_generateFrom_iff_subset_isOpen`): each subbasic `{f | f x ∈ U}` is Scott-open in the lattice (`pointwiseSubbasic_scottOpen`). Inaccessibility is immediate because the lattice's `sSup` is *pointwise* (`ScottMap.sSup_apply`), reducing to inaccessibility of `U` in `D'`. (This is the half Scott notes is automatic on p. 136: lubs are pointwise, so **no Milner hypothesis is needed** — unlike 2.9–2.10.)
- **Scott ⊆ pointwise** is the substance, via the ⇑φ-basis of a continuous lattice (`exists_wayBelow_subset`, using 3.3(a)): given a Scott-open `U ∋ g`, pick $\varphi \ll$ `g` with ⇑$\varphi \subseteq$ `U`. The key lemma `wayBelow_le_finset_sup_step` then shows $\varphi \ll$ `g` forces $\varphi \le \sqcup_i$ `ē[eᵢ,eᵢ']` for **finitely many** pairs with `eᵢ' ≪ g eᵢ`: the finite joins of step functions below `g` form a *directed* family (`Finset.sup` over $F_1 \cup F_2$) with supremum `g` (pointwise reconstruction again), so `wayBelow_sSup_iff` lands φ below one of them. The finite intersection `W = ⋂ᵢ {h | eᵢ' ≪ h eᵢ}` is then a pointwise-open (`isOpen_biInter_finset`) neighbourhood of `g` with `W ⊆` ⇑$\varphi \subseteq$ `U`: any `h ∈ W` has every `ē[eᵢ,eᵢ'] ≪ h` (`stepMap_wayBelow`), hence `⊔ᵢ ē[eᵢ,eᵢ'] ≪ h` (`wayBelow_finset_sup`) and $\varphi \ll$ `h`.

Engineering notes / lessons from 3.3(b):

- *Finiteness enters exactly once.* The only place finiteness of the step-function decomposition is needed is to keep `W` a *finite* intersection (hence open). It is delivered by realizing `g` as a directed sup of `Finset.sup`s of step functions and invoking `wayBelow_sSup_iff` — the standard "compact basis" move, here done concretely with `Finset (D × D')`.

- *No ambient instance on `ScottMap`.* Since the two topologies are competing non-instances, the proof passes them explicitly (`@isOpen_iff_forall_mem_open`, `@isOpen_biInter_finset`); this is painless precisely because `ScottMap` carries no auto-synthesized `TopologicalSpace`.
- *Beware ascription into `sSup`.* `(sSup Sg : D → D')` makes Lean elaborate `sSup` at type `D → D'` (wrong `SupSet`); write `((sSup Sg : ScottMap D D') : D → D')` to keep the lattice join.
- This closes **3.3 in full** (`theorem_3_3`), with no Milner hypothesis, contrary to the earlier expectation recorded for 2.9–2.10.
- Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.3 Corollary 3.4 (joint continuity of evaluation) — `corollary_3_4_jointly_continuous` (`FunctionSpaces.lean`)

`eval : [D → D'] × D → D', (f, x) ↦ f x`, is jointly Scott-continuous. The proof is a clean application of **Proposition 2.6** (joint ↔ separate Scott-continuity on a product lattice): reduce `PreservesDirectedSup eval` to the two separate slots. In `x` (fixed `f`) it is exactly `f`'s own Scott-continuity (`proposition_2_5` + `ScottMap.continuous`); in `f` (fixed `x`) it is the pointwise formula for suprema in `[D → D']` (`ScottMap.sSup_apply`: `(⊔F) x = ⊔ {g x | g ∈ F}`). Then `continuous_of_preservesDirectedSup` upgrades to topological continuity. Via Theorem 3.3(b) (and 2.9(b)) the Scott topology of the product lattice is the product of the pointwise topology on `[D → D']` and the Scott topology on `D`, so this is joint continuity for Scott's product topology. Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.4 Proposition 3.5 (functional abstraction) — `proposition_3_5` (`FunctionSpaces.lean`)

`lambda : [[D × D'] → D''] → [D → [D' → D'']]`, `lambda f x y = f (x, y)`, is Scott-continuous — note this *uses 3.3* twice, since the codomain `[D → [D' → D'']]` must itself be a continuous lattice (hence a legitimate target). Two layers:

- *`lambda f` is a Scott map* (`lambda_outer_preservesDirectedSup`): equality in `[D' → D'']` is pointwise, so it reduces to **left**-currying `x ↦ f (x, y)` being Scott-continuous (`curry_left_preservesDirectedSup`, mirror of the existing right-currying), itself a one-line consequence of `f`'s joint continuity and `sSup {(x, y) | x ∈ S} = (⊔S, y)`.
- *`lambda` is a Scott map* (`proposition_3_5_preservesDirectedSup`): evaluating both sides pointwise at `(x, y)` and unfolding the (three nested!) pointwise `ScottMap.sSup_apply`, both collapse to `⊔ {f (x, y) | f ∈ ℱ}`; `@[simp] scottLambda_apply` (definitional) closes the resulting image-set equality with a bare `congr 1`.

The pleasant outcome: once `[D → D']` is a genuine `CompleteLattice` instance with *pointwise* `sSup` (`ScottMap.sSup_apply` is `rfl`), all of §3's continuity facts (3.4, 3.5) are short pointwise computations. Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.5 Proposition 3.8 (maximal extension along a subspace embedding) — `proposition_3_8` (`Constructions.lean`)

For `E` a continuous lattice and `e : X → Y` a subspace embedding, Scott's explicit formula `scottExtend e f y = ⊔ { ⊓ f''(e⁻¹V) : V an open nbhd of y }` is *the maximal extension* of a continuous `f : X → E` to `[Y → E]`. The full statement bundles three clauses:

- **Continuous** and **extends** `f`: reused verbatim from the 2.11 injectivity machinery (`scottExtend_continuous`, `scottExtend_eq_of_continuous`) — the *same* operator `scottExtend` serves both 2.11 and 3.8, so 3.8 is essentially 2.11 plus a maximality clause.
- **Maximal** (`scottExtend_maximal`): for any continuous solution `f'` of `f' ∘ e = f`, expand `f'` itself via `continuous_eq_sSup_openInfs` (the order-theoretic identity `f' y = ⊔ { ⊓ f''(U) : U open nbhd of y }`, proved by interpolating from below with `f' y = ⊔ {a ≪ f' y}` and openness of each `f'⁻¹(⇑a)`). Restricting each meet from the open `U` to the embedded subspace `e(X) ∩ U` only *enlarges* the meet and lands it on a defining term of `scottExtend`, giving `f' y ≤ scottExtend e f y` — exactly Scott's two-line chain on p.116.

Engineering notes / lessons from 3.8: the partial `FunctionSpaces.scottSubspaceExtend` (renamed `scottSubspaceExtend_maximal`) had ranged `U` over the *Scott* topology of `Y` (forcing a spurious `CompleteLattice Y`), which is unfaithful to Scott (where `Y` is an arbitrary T_0 space). The faithful route was to retarget the whole proposition onto the already-continuous `scottExtend` from 2.11, which ranges `U` over `Y`'s *given* topology — turning "Stuck (one-sided bound)" into a clean **Pass** that simply repackages existing lemmas. Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.6 Proposition 3.10 (characterization of projection inclusions) — `proposition_3_10_converse`, `retr_eq_sSup` (`FunctionSpaces.lean`)

A map `i : D → D'` between continuous lattices is the inclusion of a projection **iff** it (i) preserves arbitrary suprema, (ii) is injective, and (iii) preserves $\ll$. The **forward** direction was already in place (`incl_sSup`, `incl_injective`, `incl_wayBelow`); this completes the **converse** and the **uniqueness** of Scott's formula (iv) `j(x') = ⊔ { x | i(x) ⊑ x' }`.

- *Order-reflection from (i)+(ii)* (`le_of_incl_le`): condition (i) on the two-element set gives `i(x ⊔ y) = i x ⊔ i y` (`incl_sup_of_preservesSSup`); then `i x ⊑ i y ⟹ i(x⊔y)=i y ⟹ x⊔y=y` (injectivity) `⟹ x ⊑ y`. This is exactly Scott's "`x ⊑ y ⟺ x ⊔ y = y`" remark, and it makes `i` an order-embedding.
- $j \circ i = id$ (`converseRetr_incl`): order-reflection collapses `{x | i x ⊑ i y}` to `Iic y`, whose join is `y`.
- $i \circ j \sqsubseteq id$ (`incl_converseRetr_le`): immediate from (i), since `i(⊔{x | i x ⊑ x'}) = ⊔{i x | i x ⊑ x'} ⊑ x'`.
- j *continuous* (`converseRetr_preservesDirectedSup`): the one place (iii) is needed. For a directed `S'` and `i x ⊑ ⊔S'`, interpolate `x = ⊔{z ≪ x}` (continuity of `D`); each `z ≪ x` gives `i z ≪ i x ⊑ ⊔S'`, so `i z ⊑ x'` for some `x' ∈ S'` (directed `wayBelow_sSup_iff`), whence `z ⊑ j x' ⊑ ⊔ j''S'`.
- *Uniqueness* (`retr_eq_sSup`): any projection's `j` satisfies `j x' = ⊔{x | i x ⊑ x'}` — $\leq$ since `i(j x') ⊑ x'` makes `j x'` a member; $\geq$ since each member `x` has `x = j(i x) ⊑ j x'`.

Engineering notes / lessons from 3.10: condition (i) is stated for *arbitrary* `S`, so it trivially supplies `PreservesDirectedSup i` (whence `i` is a legitimate `ScottMap`) with a one-line `fun _ _ _ => hi _` — no need to separately assume continuity of `i`. Set-membership in `{x | i x ⊑ x'}` is *definitionally* the predicate, so `le_sSup`/`sSup_le` chains go through with bare `.le` coercions and `show` re-statements rather than `Set.mem_setOf` rewrites. Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.7 Lemma 3.9 (extensions commute with a range projection) — `lemma_3_9` (`Theorem212.lean`)

With `e : X → Y` a subspace embedding and `i, j : D ⇄ D'` a projection on the *range*, if continuous `f : X → D` and `g : X → D'` satisfy `f = j ∘ g`, then their maximal extensions (3.8) satisfy $\bar{f} = j \circ \bar{g}$. This is the key compatibility used to build inverse limits (§4: $\bar{f}_n = j_n \circ \bar{f}_{n+1}$). The proof is a clean two-inequality sandwich, exactly Scott's:

- $j \circ \bar{g} \sqsubseteq \bar{f}$: $j \circ \bar{g}$ is continuous and $(j \circ \bar{g}) \circ e = j \circ g = f$, so the *equality* maximality of $\bar{f}$ (`scottExtend_maximal`) applies.
- $i \circ \bar{f} \sqsubseteq \bar{g}$: $(i \circ \bar{f}) \circ e = i \circ f = i \circ j \circ g \sqsubseteq g$ (using $i \circ j \sqsubseteq \mathrm{id}$), so the *sub-solution* maximality `scottExtend_maximal_le` (the remark after 3.8, added here as the $\leq$-analogue of `scottExtend_maximal` — identical proof, final `=` weakened to $\leq$) applies.
- combine: $\bar{f} = j \circ i \circ \bar{f} \sqsubseteq j \circ \bar{g} \sqsubseteq \bar{f}$ (apply monotone j to the second bound, and $j \circ i = \mathrm{id}$).

Engineering notes / lessons from 3.9: the lemma lives in `Theorem212.lean` because it is the only module importing *both* `scottExtend` (Constructions) and `IsContinuousLatticeProjection` (FunctionSpaces). The one real friction was composition continuity: the Scott topology is a bare `def`, not an `instance`, so `Continuous.comp` cannot synthesize `TopologicalSpace D`. Registering it with `letI` works, but **only if scoped inside the `have` for the composite** — registering it at the top of the proof makes the lattice $\leq$ ambiguous (it gets re-resolved through the topology's `specializationPreorder`), which silently breaks every later `le_antisymm`/`calc`. The older inf-level partials `lemma_3_9_incl_inf`/`lemma_3_9_retr_inf` are now superseded auxiliaries. Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.8 Proposition 3.12 (the lattice of projections `J_D`) — `proposition_3_12` (`FunctionSpaces.lean`)

`J_D = { j ∈ [D → D] : j = j ∘ j ⊑ id }` (`IsProjection`) is a complete lattice realized as a $\sqcup$-closed subspace of `[D → D]`. The whole proof reduces, via the pointwise characterization `isProjection_iff` (idempotent **and** deflationary), to closure of `J_D` under arbitrary `sSup` (`isProjection_sSup`); a $\sqcup$-closed subset of a complete lattice is a complete lattice (`completeLatticeOfSup` on the subtype `Projections D`).

- *binary* (`isProjection_sup`): since `j x ⊔ k x ⊑ x`, monotonicity + idempotency pin `j (j x ⊔ k x) = j x` (and symmetrically for `k`), so `(j ⊔ k) ∘ (j ⊔ k) = j ⊔ k`. This is the one spot needing `sup_apply` — the new lemma that the `completeLatticeOfSup`-derived binary join of Scott maps is computed *pointwise* (`(f ⊔ g) x = f x ⊔ g x`, since `⊔ = sSup {·,·}` and `sSup` is pointwise).
- *directed* (`isProjection_directedSup`): continuity of each `k ∈ S` distributes `k ((⊔S) x) = ⊔ⱼ k (j x)` over the directed family `{ j x }`, and directedness + idempotency collapse the double sup `{ k (j x) }` back to `(⊔S) x`. (Continuity of `D` itself is *not* used — this works for any complete lattice `D`.)
- *arbitrary* (`isProjection_sSup`): reuse `finsetSupOf` (every `sSup` is the directed sup of finite sub-joins), with `isProjection_finsetSup` via `Finset.sup_induction` on $\bot$/binary.

Engineering notes / lessons from 3.12: the identity map is named `ScottMap.idMap`, **not** `id`, to avoid shadowing `_root_.id` (which `finsetSupOf`'s `Finset.sup id` relies on). The `Projections D` subtype must be an `abbrev` (not `def`) so the ambient `Subtype.partialOrder`/`SupSet` instances are found by typeclass resolution — the same reducibility lesson as `IdemFix` in 2.12. Footprint `[propext, Classical.choice, Quot.sound]`.

5.2.9 Proposition 3.13 (D is a projection of [D → D]) — proposition_3_13 (FunctionSpaces.lean)

Scott's con : D → [D → D], con x = (const x), and min : [D → D] → D, min f = f(⊥), form a projection: min (con x) = (const x)(⊥) = x (so min ∘ con = id, rfl), and con (min f) = const (f ⊥) ⊑ f pointwise because f(⊥) ⊑ f(y) by monotonicity (so con ∘ min ⊑ id). Both maps are Scott-continuous: con because suprema in [D → D] are pointwise (con (⊔S) = const (⊔S) and ⊔$_j$ const(j) = const(⊔S)), and min because it is evaluation at ⊥, which reads off the pointwise supremum (ScottMap.sSup_apply). The result packages as a term of the existing IsContinuousLatticeProjection D [D → D], so it immediately feeds Proposition 3.10's machinery. (Continuity of D is again unused; included only to match Scott's hypothesis.) Footprint [propext, Classical.choice, Quot.sound].

5.2.10 Proposition 3.14 (the fixed-point operator) — proposition_3_14 (FunctionSpaces.lean)

fix : [D → D] → D is Scott's least-fixed-point combinator: f (fix f) = fix f and f x ⊑ x ⟹ fix f ⊑ x, and it is the *unique* operator with these two properties. The **order content** is mathlib's OrderHom.lfp (fix f := (⟨f, f.monotone⟩ : D →o D).lfp), giving fix_eq (map_lfp), fix_le (lfp_le), and fix_unique (least element of the fixed-point set is unique) for free.

The **continuity** of fix (Scott's actual claim) is the work. Scott argues via Kleene's fix f = ⊔$_n$ f^n(⊥) ("pointwise lub of continuous functions"); we give a **direct lattice proof that avoids iteration entirely** (fix_preservesDirectedSup). For directed S ⊆ [D → D], set g = ⊔S and a = ⊔{fix f : f ∈ S}:

- a ⊑ fix g is just fix-monotonicity (fix_mono, itself a two-line fix_le).
- fix g ⊑ a: by fix_le it suffices that a is a pre-fixed point, g a ⊑ a. Pointwise sups give g a = ⊔_{f∈S} f a, and continuity of each f on the **directed** family {fix f' : f' ∈ S} gives f a = ⊔_{f'∈S} f (fix f'). For any f, f' ∈ S choose (directedness) h ∈ S above both: f (fix f') ⊑ h (fix f') ⊑ h (fix h) = fix h ⊑ a. Hence g a ⊑ a.

Engineering notes / lessons from 3.14: the direct argument is far shorter than building Kleene's theorem and only needs three ingredients already in hand — OrderHom.lfp monotonicity facts, ScottMap.sSup_apply (pointwise sups in [D → D]), and preservesDirectedSup_coe. Two small Lean traps: (1) sSup_le leaves the bound element as an un-β-reduced (fun f => ↑f (sSup T)) f, so a show (f : D → D) (sSup T) ≤ sSup T is needed before the rw; (2) in the uniqueness clause an *unannotated* binder ∀ f, (f : D → D) ... makes the ascription **fix the binder type to** D → D rather than coerce — the binders must be written ∀ f : ScottMap D D. Continuity of D is unused (works for any complete lattice). Footprint [propext, Classical.choice, Quot.sound].

5.3 Inverse limits (Scott §4)

Section §4 constructs D_∞ and proves Theorem 4.4. The adjoint route to Proposition 4.1 and the function-space tower scaffolding for 4.4 are the main engineering contributions beyond Scott's text.

5.3.1 Proposition 4.1 (inverse limit of projections is a continuous lattice) — proposition_4_1 (InverseLimits.lean)

D∞ = { x : ∀n, D$_n$ // ∀n, j$_n$(x$_{n+1}$) = x$_n$ } for an ω-system of continuous lattices with projection bonding maps j$_n$: D_{n+1} → D$_n$. Scott proves continuity *topologically* (show D∞ is an injective T$_0$-space, then Theorem 2.12), using the maximal extension 3.8 and the compatibility 3.9. We realize the **same retraction order-theoretically, with no topology**, which

sidesteps a genuine soundness trap (the subspace Scott topology on D_∞ need not equal its own Scott topology, so the inclusion is not obviously a Scott embedding — the hypothesis 3.8/3.9 silently need).

The key observation: each projection is an **adjunction**. From $j_n \circ i_n = \mathrm{id}$ and $i_n \circ j_n \sqsubseteq \mathrm{id}$ we get `GaloisConnection` $i_n\ j_n$ (`projection_galoisConnection`), so j_n (the upper adjoint) preserves arbitrary infima (`retr_sInf`). Hence:

- the compatibility predicate is closed under **pointwise** `sInf` (`compatible_sInf`), so D_∞ is a complete lattice by `completeLatticeOfInf`;
- the inclusion $D_\infty \hookrightarrow \prod D_n$ preserves infima, so it has a **left adjoint** `r : ` $\prod D_n \to D_\infty$, `r y = ⊓{ x ∈ D∞ : y ⊑ x }` (`invLimRetr`, `invLimRetr_galoisConnection`); a left adjoint preserves *all* suprema (`GaloisConnection.l_sSup`), in particular directed ones, so `r` is Scott-continuous, and `r∘incl = id` (`invLimRetr_incl`);
- the inclusion itself is Scott-continuous because directed sups of compatible sequences are pointwise (each j_n is Scott-continuous), so D_∞'s directed sups agree with the ambient ones (`coe_sSup_of_directed`).

Thus D_∞ is a Scott-continuous **retract** of $\prod D_n$, which is a continuous lattice (Prop 2.9a), so Prop 2.10a gives `IsContinuousLattice` D_∞. This `r` is exactly the retraction Scott's injectivity argument constructs (extend `id_{D∞}` along the inclusion), here obtained directly as an adjoint.

Engineering notes / lessons from 4.1: `IsContinuousLattice` is purely order-theoretic and 2.10a transfers it across a *Scott-continuous retraction* with no topology, which is what makes the adjoint route viable. Two friction points: coordinatewise `sInf`/`sSup` of a product are reached through `sInf_apply_eq_sInf_image`/`sSup_apply_eq_sSup_image`, and the resulting set equalities are best closed with `Set.image_image` + `Set.image_congr` (using compatibility pointwise) rather than `ext` (whose membership unfolds to `Function.eval` with the wrong orientation). The directed-sup-is-pointwise lemma is proved by exhibiting the pointwise sup as an explicit `IsLUB` and invoking `(isLUB_sSup S).unique`. Footprint `[propext, Classical.choice, Quot.sound]`.

5.3.2 Proposition 4.2 (the maps `j_{∞n}` are projections) — `proposition_4_2` (`InverseLimits.lean`)

`j_{∞n}` : $D_\infty \to D_n$ is evaluation $x \mapsto x_n$. Scott constructs the inverse embedding `i_{n∞}` : $D_n \to D_\infty$ componentwise: `i_{n∞}(x)_m = x` at `m = n`, climbs by i_k `= (P k).incl` for `m > n`, and descends by j_k `= (P k).retr` for `m < n`. We realize this with two `Nat.leRecOn` towers:

- `embLE (h : n ≤ m) : D_n → D_m` (climb, `= i_{m-1}∘…∘i_n`) and `projLE (h : m ≤ n) : D_n → D_m` (descend, `= j_m∘…∘j_{n-1}`), with the computation lemmas `embLE_self/_succ/_succ_left`, `projLE_self/_succ` reading off `Nat.leRecOn_self/_succ/_succ_left`;
- `iComp n x m = if n ≤ m then embLE … else projLE …` is the component map; `iComp_compatible` (case split on `n ≤ m`, `n = m+1`, `m+1 ≤ n`, the middle hinge being `projLE_retr`) shows the sequence is a genuine point of D_∞, and `iComp_self` gives `j_{∞n}∘i_{n∞} = id`.

Both towers are Scott-continuous (`embLE/projLE_preservesDirectedSup`, by `Nat.le_induction` + `ScottMap.preservesDirectedSup_comp`), hence each component `iComp n · m` is (`iComp_preservesDirectedSup`); since directed sups in D_∞ are pointwise (`coe_sSup_of_directed`), the bundled `embInf n : ScottMap` D_n D_∞ and `projInf n : ScottMap` D_∞ D_n are continuous. `proposition_4_2` packages ⟨`embInf, projInf`⟩ as an `IsContinuousLatticeProjection`: `retr_incl = iComp_self`, and `incl_retr_le` reduces coordinatewise (`Subtype.coe_le_coe`)

to iComp_incl_le — for m ≥ n climbing y_n stays below y_m (embLE_le, using incl∘retr ⊑ id and compatibility), for m < n it equals y_m (projLE_compatible).

Also formalized: the recursion equation i_{n∞} = i_{(n+1)∞}∘i_n (embInf_succ) and the monotone-lub identity x = $\bigcup_n$ i_{n∞}(x_n) (inverseLimit_eq_iSup); the family is monotone via embInf_succ + incl_retr_le (embInf_le_succ), so its range is directed and the lub is computed pointwise, where iComp_self pins the m-th coordinate to x_m.

Engineering notes / lessons from 4.2: Nat.leRecOn (and Nat.le_induction) is the clean way to build/induct on the two dependently-typed towers without Nat-subtraction casts; the descend tower uses the *function* motive C k := D k → D_m. Nat.leRecOn is @[elab_as_elim], so its computation lemmas must be applied after unfolding the wrapper (simp only [embLE] / simp only [projLE]) — a bare term-mode := Nat.leRecOn_self x fails with "failed to elaborate eliminator". Lean 4's definitional proof irrelevance means the towers do not depend on *which* ≤ proof is supplied, so the rw chains match across le_refl/Nat.le_succ_of_le/Nat.le_of_succ_le freely. The eliminator is invoked as induction n, h using Nat.le_induction. Footprint [propext, Classical.choice, Quot.sound].

5.3.3 Corollary 4.3 (D∞ is also the *direct* limit) — corollary_4_3 (InverseLimits.lean)

Where Prop 4.2 makes D∞ the *inverse* (projective) limit, 4.3 is the dual universal property: it is the *direct* (injective) limit along the embeddings i_n. Given any complete lattice D' and a **compatible cocone** of Scott maps f_n : D_n → D' with f_n = f_{n+1}∘i_n (hf), the mediating map is coconeInf f x = f∞(x) = $\bigcup_n$ f_n(x_n). We prove there is a **unique** continuous f∞ with f_n = f∞∘i_{n∞} (an ∃! over ScottMap (InverseLimit D P) D').

- *Factorization* coconeInf_comp_embInf: f∞(i_{n∞}(x)) = $\bigcup_m$ f_m(iComp n x m) = f_n(x) by le_antisymm. The ≥ direction is iComp_self at m = n. For ≤, the family m ↦ f_m(iComp n x m) is dominated by f_n(x): above n it is *constant* = f_n(x) (coconeInf_climb, Nat.le_induction collapsing f_{m+1}∘i_m = f_m), and below n it only decreases (coconeInf_descend: peel projLE via projLE_succ, then f_m∘j_m = f_{m+1}∘i_m∘j_m ⊑ f_{m+1} by incl_retr_le + monotonicity).

- *Continuity* coconeInf_preservesDirectedSup: needs **no** hf. For directed S, push the sup through each coordinate (eval_preservesDirectedSup) and through each continuous f_n (preservesDirectedSup_coe, image of S directed under evaluation), then commute the resulting double sup over ℕ × S with iSup_comm (rewriting images as subtype sups with sSup_image').

- *Uniqueness*: any continuous g with f_n = g∘i_{n∞} satisfies g(x) = g($\bigcup_n$ i_{n∞}(x_n)) = $\bigcup_n$ g(i_{n∞}(x_n)) = $\bigcup_n$ f_n(x_n) = f∞(x), using inverseLimit_eq_iSup (4.2), continuity of g on the directed family (embInf_family_directed), and ScottMap.ext.

Footprint [propext, Classical.choice, Quot.sound].

5.3.4 Lemma 4.5 and the functional equation — lemma_4_5, idInf_eq_iSup (InverseLimits.lean)

idInf_eq_iSup records Scott's "remark following 4.2": as Scott maps D_∞ → D_∞, id = $\bigcup_n$ (i_{n∞} ∘ j_{∞n}). Pointwise, ($\bigcup_n$ i_{n∞}∘j_{∞n})(x) = $\bigcup_n$ i_{n∞}(x_n) = x (ScottMap.sSup_apply to push the sup of maps through evaluation, then inverseLimit_eq_iSup).

lemma_4_5 is Scott's tool for *recognizing projections from limits*: if u : ∀ n, D_{n+1} obeys the shifted recursion j_{n+1}(u_{n+2}) = u_{n+1}, then u_∞ = $\bigcup_n$ i_{(n+1)∞}(u_n) has j_{∞(n+1)}(u_∞) = u_n. The trick is to *extend* u to a genuinely compatible sequence w (w_0 = j_0(u_0), w_{k+1} = u_k; compatibility at k=0 is rfl, at k+1 it is the hypothesis), so w ∈ D_∞. Since the family k ↦ i_{k∞}(w_k) is monotone (embInf_le_succ), dropping its 0-th term

leaves the lub unchanged (`Monotone.iSup_nat_add ... 1`), giving `u_∞ = ⨆ₖ i_{k∞}(w_k) = w` by `inverseLimit_eq_iSup`; hence `j_{∞(n+1)}(u_∞) = w_{n+1} = uₙ` by definitional unfolding of `w`.

5.3.5 Theorem 4.4 scaffolding — `FunctionSpaceTower.lean`

The capstone needs the *concrete* recursion `D_{n+1} = [Dₙ → Dₙ]`, `j_{n+1} = [jₙ → jₙ]` — the first place in §4 where the levels are genuine function spaces. Because the type at level `n+1` depends on the *lattice structure* at level `n`, we bundle carrier + instance in `CLat` and recurse (`towerCLat`); `towerType`/`towerCompleteLattice` project out the type and its `CompleteLattice`, and crucially `towerType_succ : D_{n+1} = [Dₙ→Dₙ]` holds by `rfl`, with a `CoeFun` (`towerCoeFun`) letting us apply a `D_{n+1}` element directly as a function `Dₙ → Dₙ`.

The bonding maps come from a continuous form of Proposition 3.7: `conjMap post pre` (`f ↦ post∘f∘pre`) is Scott-continuous (directed sups in `[Y→Y]` are pointwise, so the conjugate commutes with them), whence `IsContinuousLatticeProjection.functionSpace` makes `[D→D]` a projection of `[D'→D']` from a projection `D ◁ D'`. Iterating from a chosen base `j₀ : [D₀→D₀] ◁ D₀` (Proposition 3.13 supplies one) gives the projection tower `towerProj`. The Scott recursion/algebra laws are then definitional: `towerProj_succ_incl_apply` (`i_{n+1}(x)=iₙ∘x∘jₙ`), `towerProj_succ_retr_apply` (`j_{n+1}=jₙ∘·∘iₙ`), and `towerProj_incl_apply` (`iₙ(f(x))=i_{n+1}(f)(iₙ(x))`, application preserved one level up).

Thm 4.4(a) — `embInfInf` / `projInfInf`. With `DInf := InverseLimit (towerType D₀) (towerProj D₀ j₀)` (a continuous lattice by Proposition 4.1) and `DInfFn := [D∞ → D∞]`, Scott's limit pair is written down directly:

```
i inf (x) =  bigcup _n (i_{n inf }  comp  x_{n+1}  comp  j_{ inf n})       : D inf   ->  [D inf
       ->  D inf ]
j inf (f) =  bigcup _n i_{(n+1) inf }(j_{ inf n}  comp  f  comp  i_{n inf })   : [D inf   ->  D
     inf ]  ->  D inf
```

The engineering payoff: **each summand is already a `ScottMap`.** The `n`-th summand of `i∞`, `iInfTerm n`, is the composite `conjMap (i_{n∞}, j_{∞n}) ∘ j_{∞(n+1)}` (conjugation by the Prop 4.2 projection pair, precomposed with the component projection `j_{∞(n+1)} : D∞ → D_{n+1}` reading off `x_{n+1}`); the `n`-th summand of `j∞`, `jInfTerm n`, is `i_{(n+1)∞} ∘ conjMap (j_{∞n}, i_{n∞})`. Both are honest Scott maps because `conjMap`, `embInf`, `projInf`, and `.comp` are. Consequently `i∞` and `j∞` are *suprema of Scott maps* — `⨆ₙ iInfTerm n`, `⨆ₙ jInfTerm n` — taken in the complete lattices `[D∞ → [D∞→D∞]]` and `[[D∞→D∞] → D∞]` (Theorem 3.3), so they are Scott-continuous *for free*: no bespoke directed-sup/`iSup_comm` argument is needed (contrast the `coconeInf` template). The pointwise unfolding `embInfInf_apply : i∞(x) = ⨆ₙ iInfTerm n x` (and `projInfInf_apply`) follows from `ScottMap.sSup_apply` + `Set.range_comp`, and the `*_apply` reductions of the summands hold by `rfl` (riding on `towerType_succ` defeq). `*_preservesDirectedSup` is then immediate from `.continuous` via Proposition 2.5. Footprint `[propext, Classical.choice, Quot.sound]`. Theorem 4.4 subgoals **(a)**–**(d)** are all complete:

- **(a)** `embInfInf` / `projInfInf`: define i_∞/j_∞ as Scott maps (suprema of Scott maps).
- **(b)** `projInfInf_comp_embInfInf`: $j_\infty \circ i_\infty = \mathrm{id}$ on D_∞.
- **(c)** `embInfInf_comp_projInfInf`: $i_\infty \circ j_\infty = \mathrm{id}$ on $[D_\infty \to D_\infty]$.
- **(d)** `theorem_4_4`, `theorem_4_4_orderIso`: capstone packaging $D_\infty \cong [D_\infty \to D_\infty]$.

Thm 4.4(b) — `projInfInf_comp_embInfInf`. Goal: $j_\infty \circ i_\infty = \mathrm{id}$ on D_∞. Following Scott's calculation, expand `j∞(i∞(x)) = ⨆ₙ jInfTerm n (i∞ x)`. Pushing the two conjugations through the inner/outer suprema (`conjMap_iSup`, `embInf_succ_iSup` — each just

preservation of directed sups by the relevant `ScottMap`, since the summand families are monotone in `m`) rewrites the `n`-th term as $\bigcup_m$ `g n m` with `g n m = i_{(n+1)∞}(conjMap (j_{∞n}, i_{n∞})(iInfTerm m x))`. The double sup $\bigcup_n \bigcup_m$ `g n m` collapses to the diagonal $\bigcup_n$ `g n n` (`iSup₂_monotone_eq_diagonal`); monotonicity in `m` is routine, and monotonicity in `n` is the one piece of real content — `conjMap_incl_le_conjMap_succ`, the inequality `i_{n+1}(conjMap (j_{∞n}, i_{n∞}) f) ⊑ conjMap (j_{∞(n+1)}, i_{(n+1)∞}) f` in `D_{n+2}`, built from `embInf_succ`, `incl_retr_le`, and `i_{n∞}(y_n) ⊑ y_{n+1}` (`incl_projInf_le_projInf_succ`). On the diagonal, `conj_iInfTerm_eq` is exactly the function-space retraction `j_{[·]} ∘ i_{[·]} = id` of the Prop 4.2 projection pair, giving `g n n = i_{(n+1)∞}(x_{n+1})`; an index shift (`Monotone.iSup_nat_add`) plus `inverseLimit_eq_iSup` recognizes the result as `x`. Footprint `[propext, Classical.choice, Quot.sound]`.

Thm 4.4(c) — `embInfInf_comp_projInfInf.` Goal: $i_\infty \circ j_\infty = \mathrm{id}$ on $[D_\infty \to D_\infty]$. The restrictions `u_n = j_{∞n} ∘ f ∘ i_{n∞} = conjMap (j_{∞n}, i_{n∞}) f ∈ D_{n+1}` satisfy the Lemma-4.5 recursion `j_{n+1}(u_{n+2}) = u_{n+1}` — proved as `towerProj_retr_conjMap_succ`, the equality sibling of (b)'s `conjMap_incl_le_conjMap_succ` (unfold `(towerProj (n+1)).retr` as the function-space `conjMap`, then `embInf_succ` and the compatibility equation `x.2 n`). Hence `lemma_4_5` gives the components `(j∞ f).(n+1) = u_n` (`hcoord`). Evaluating `i∞(j∞ f)` pointwise (`embInfInf_apply`, then `ScottMap.sSup_apply` for the pointwise lub) and rewriting each summand with `hcoord` + `conjMap_apply` reduces the `n`-th term to `r_n (f (r_n z))` with `r_n = i_{n∞} ∘ j_{∞n}`. The analytic step (Scott ˜1326–1334) confines the lub via continuity of `f` and the functional equation `id = ⋃_n r_n` (here just `inverseLimit_eq_iSup`, since `r_n z = i_{n∞}(z_n)`): `f z = ⋃_k r_k (f z) = ⋃_k r_k (f (⋃_m r_m z)) = ⋃_k ⋃_m r_k (f (r_m z))`, and the monotone double sup collapses to the diagonal $\bigcup_n$ `r_n (f (r_n z))` (`iSup₂_monotone_eq_diagonal`), which is exactly the evaluated `i∞(j∞ f) z`. Footprint `[propext, Classical.choice, Quot.sound]`.

Thm 4.4(d) — `theorem_4_4.` Capstone packaging of (b)+(c): `theorem_4_4` bundles the two composition identities (`projInfInf_comp_embInfInf`, `embInfInf_comp_projInfInf`); helper lemmas `projInfInf_embInfInf` / `embInfInf_projInfInf` apply the `ScottMap` equalities pointwise. `theorem_4_4_orderIso : D∞ ≃o [D∞ → D∞]` is built via `Equiv.toOrderIso` from the same inverse pair (both directions monotone Scott maps, hence Scott-continuous). Footprint `[propext, Classical.choice, Quot.sound]`. **Scott §4 is complete.**

---

6 Reproducibility

Inventory source of truth: this file (`arxiv.md`). Do not use generated `arxiv_with_code.md` or `arxiv.tex` (they inline Lean sources and mermaid figures for review/PDF packaging and are stale whenever older than `arxiv.md` or any listed `.lean` file).

The repository pins Lean / mathlib **v4.30.0** (`lean-toolchain`).

```
lake exe cache get
lake build Scott1972
bash scripts/generate_arxiv_with_code.sh   #  ->  arxiv_with_code.md (gitignored)
bash scripts/build_arxiv_tex.sh            #  ->  arxiv.tex + lean-listings/ + figures/ (
    gitignored)
bash scripts/build_arxiv_pdf.sh            #  ->  arxiv.pdf (LuaLaTeX; tracked) + dist/
    arxiv_submit.zip
bash scripts/package_arxiv_submit.sh       #  ->  dist/arxiv_submit.zip only (skips PDF)
```

---

7 Acknowledgments

- **Dana Scott** — *Continuous Lattices* **[Sco72]**, the paper this development formalizes.
- **Robin Milner** — the March 1972 correction to **[Sco72]**, without which Propositions 2.9, 2.10, and 3.3 would be wrong as originally stated.

7.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools **[COPE24]**), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools:

- **Cursor [Cur26]** — agent-assisted editing in the Cursor IDE: formalizing Scott's 1972 continuous-lattice layer in Lean 4 / mathlib, `lake build` repair, vision-OCR transcription, drafting this narrative (`arxiv.md`), and tracking the formalized inventory. Generated Lean was provisional until it compiled under the pinned toolchain.
- **Cursor Composer 2.5 Fast [Cmp25]** — routine multi-step work: module scaffolding, dependency-ordered wiring of `Scott1972/ContinuousLattice/`, documentation and Mermaid blueprints, and medium proof obligations where the strategy was already fixed. Per its model card, Composer 2.5 is optimized for codebase navigation rather than open-ended topological proof design; the Milner-block results (2.9–2.11, full 3.3) were not delegated to it alone.
- **Anthropic Claude Opus 4.8 (high reasoning) [Ant26]** — selective use for the heaviest proof work (Propositions 2.9–2.11, Theorem 2.12, Theorem 3.3, Propositions 3.8–3.10, Theorem 4.4). Every emitted proof term was checked by the Lean kernel.
- **Google Gemini 3.5 Flash [Gem25]** — exploratory passes on Scott's typographic conventions (ambient vs subspace joins in the Milner correction) and scope decisions.

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

7.2 Artifact availability

The development **[SR72]** is at `github.com/catskillsresearch/scott1972`. Run `lake build Scott1972` for the sorry-free formalization; `scripts/generate_arxiv_with_code.sh` builds `arxiv_with_code.md` from this file plus the complete Lean source.

---

8 References

- **[Sco69]** D. S. Scott. *Lattice-theoretic models for the λ-calculus* (unpublished manuscript). University of Oxford, 1969.
- **[Sco70]** D. S. Scott. Outline of a mathematical theory of computation. In *Proceedings of the Fourth Annual Princeton Conference on Information Sciences and Systems* (pp. 169–176). Princeton University, 1970.

- **[Sco72]** D. S. Scott. Continuous lattices. In F. W. Lawvere (Ed.), *Toposes, Algebraic Geometry and Logic* (Lecture Notes in Mathematics, Vol. 274, pp. 97–136). Springer, Berlin, Heidelberg, 1972. URL: http://upol.cz
- **[GHKLMS03]** G. Gierz, K. H. Hofmann, K. Keimel, J. D. Lawson, M. Mislove, and D. S. Scott. *Continuous Lattices and Domains*. Cambridge University Press, 2003.
- **[Kel55]** J. L. Kelley. *General Topology*. D. Van Nostrand Company, 1955.
- **[SR72]** Catskills Research. *scott1972* (this work). https://github.com/catskillsresearch/scott1972.
- **[COPE24]** Committee on Publication Ethics (COPE). *Authorship and AI tools: COPE position statement*. 2024. https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools
- **[Cur26]** Anysphere, Inc. *Cursor: AI-native code editor and agent environment*. https://cursor.com (accessed 2026).
- **[Cmp25]** Anysphere, Inc. *Composer 2.5*. Model announcement and documentation, https://cursor.com/blog/composer-2-5; model card as integrated in Cursor, https://cursor.com/docs/models (accessed 2026).
- **[Ant26]** Anthropic. *Claude Opus 4.8* (high thinking/reasoning variant). System card and announcement, https://www.anthropic.com/news/claude-opus-4-8; model documentation as integrated in Cursor, https://cursor.com/docs/models/claude-opus-4-8 (accessed 2026).
- **[Gem25]** Google DeepMind. *Gemini 3.5 Flash*. Technical documentation and model cards. https://ai.google.dev/gemini-api/docs/models (accessed 2026).

A Complete Lean source

Role	File
Root import graph	`Scott1972.lean`
Scott §1	`Scott1972/ContinuousLattice/Injective.lean`
Scott §2	`Scott1972/ContinuousLattice/WayBelow.lean`
Scott §2	`Scott1972/ContinuousLattice/Specialization.lean`
Scott §2	`Scott1972/ContinuousLattice/ScottMaps.lean`
March 1972 correction	`Scott1972/ContinuousLattice/MilnerCorrection.lean`
Scott §2.8–2.12	`Scott1972/ContinuousLattice/Constructions.lean`
Scott §3	`Scott1972/ContinuousLattice/FunctionSpaces.lean`
Theorem 2.12	`Scott1972/ContinuousLattice/Theorem212.lean`
Scott §4	`Scott1972/ContinuousLattice/InverseLimits.lean`
Theorem 4.4	`Scott1972/ContinuousLattice/FunctionSpaceTower.lean`

Primary source (OCR plain text): `sources/ScottContinLatt1972.md` — transcription of **[Sco72]** for use in the Lean development (see §2).

Files appear in `Scott1972.lean` import order. Each block is a verbatim copy of the repository file at generation time.

A.1 Scott1972.lean

10 lines.

Lean 4 source

```
import Scott1972.ContinuousLattice.Injective
import Scott1972.ContinuousLattice.WayBelow
import Scott1972.ContinuousLattice.Specialization
import Scott1972.ContinuousLattice.ScottMaps
import Scott1972.ContinuousLattice.MilnerCorrection
import Scott1972.ContinuousLattice.Constructions
import Scott1972.ContinuousLattice.FunctionSpaces
import Scott1972.ContinuousLattice.Theorem212
import Scott1972.ContinuousLattice.InverseLimits
import Scott1972.ContinuousLattice.FunctionSpaceTower
```

A.2 Scott1972/ContinuousLattice/Injective.lean

125 lines.

Lean 4 source

```
import Mathlib.Topology.ContinuousMap.TOSierpinski
import Mathlib.Topology.ContinuousMap.Basic
import Mathlib.Topology.Sets.Opens

/-!
# Injective spaces (Scott 1972, S1)

Scott's S1 introduces *injective* `T_0`-spaces -- those with the extension property for
continuous maps along subspace embeddings -- and shows the two-point Sierpinski space is
injective, that injectivity is preserved by products and by retracts, and (the embedding
theorem) that every `T_0`-space embeds in a power of the Sierpinski space.

## Main definitions / results

* `IsInjectiveSpace` -- Scott's Definition 1.1.
* `proposition_1_2` ... `proposition_1_5` -- Scott's Propositions 1.2-1.5.
* `corollary_1_6`, `corollary_1_7` -- Scott's Corollaries 1.6 and 1.7.
-/

/-- Scott's two-point Sierpinski space  O : `Prop` with the Sierpinski topology. -/
abbrev Sierpinski := Prop

open Topology TopologicalSpace

universe u v w

/-- **Scott 1972, Definition 1.1.** A topological space `D` is *injective* when, for every
topological embedding `e : X  ->  Y` and every continuous `f : X  ->  D`, there is a continuous
`g : Y  ->  D` extending `f` along `e`. -/
def IsInjectiveSpace (D : Type v) [TopologicalSpace D] : Prop :=
  forall  {X Y : Type u} [TopologicalSpace X] [TopologicalSpace Y] (e : X  ->  Y),
    IsEmbedding e  ->  forall  f : C(X, D), exists  g : C(Y, D), forall  x, g (e x) = f x

/-- **Scott 1972, Proposition 1.2.** The Sierpinski space is injective. -/
theorem isInjectiveSpace_sierpinski : IsInjectiveSpace.{u} Prop := by
  intro X Y _ _ e he f
  have hfopen : IsOpen {x | f x} := continuous_Prop.1 f.continuous
  rw [he.isInducing.isOpen_iff] at hfopen
  obtain <V, hVopen, hVeq> := hfopen
```

```
  refine <<fun y => y  in  V, continuous_Prop.2 hVopen>, fun x => ?_>
  have : (e x  in  V) = (x  in  {x | f x}) := by rw [ <-  hVeq]; rfl
  simpa using this

theorem proposition_1_2 : IsInjectiveSpace.{u} Sierpinski :=
  isInjectiveSpace_sierpinski

/-- **Scott 1972, Proposition 1.3.** Products of injective spaces are injective. -/
theorem IsInjectiveSpace.pi {? : Type w} (D : ?  ->  Type v) [forall  i, TopologicalSpace (D i)
    ]
    (h : forall  i, IsInjectiveSpace.{u} (D i)) : IsInjectiveSpace.{u} (forall  i, D i) := by
  intro X Y _ _ e he f
  choose g hg using fun i =>
    h i e he <fun x => f x i, (continuous_apply i).comp f.continuous>
  refine <<fun y i => g i y, continuous_pi fun i => (g i).continuous>, fun x => ?_>
  funext i
  exact hg i x

theorem proposition_1_3 {? : Type w} (D : ?  ->  Type v) [forall  i, TopologicalSpace (D i)]
    (h : forall  i, IsInjectiveSpace.{u} (D i)) : IsInjectiveSpace.{u} (forall  i, D i) :=
  IsInjectiveSpace.pi D h

/-- **Scott 1972, Proposition 1.4.** Retracts of injective spaces are injective. -/
theorem IsInjectiveSpace.of_retract {D : Type v} {D' : Type v}
    [TopologicalSpace D] [TopologicalSpace D'] (hD : IsInjectiveSpace.{u} D)
    (s : C(D', D)) (r : C(D, D')) (hrs : forall  d, r (s d) = d) :
    IsInjectiveSpace.{u} D' := by
  intro X Y _ _ e he f
  obtain <g, hg> := hD e he (s.comp f)
  refine <r.comp g, fun x => ?_>
  show r (g (e x)) = f x
  rw [hg x]
  exact hrs (f x)

theorem proposition_1_4 {D : Type v} {D' : Type v} [TopologicalSpace D] [TopologicalSpace D']
    (hD : IsInjectiveSpace.{u} D) (s : C(D', D)) (r : C(D, D')) (hrs : forall  d, r (s d) = d)
    :
    IsInjectiveSpace.{u} D' :=
  IsInjectiveSpace.of_retract hD s r hrs

theorem isInjectiveSpace_sierpinski_power (? : Type w) :
    IsInjectiveSpace.{u} (?  ->  Prop) :=
  proposition_1_3 (fun (_ : ?) => Prop) (fun _ => proposition_1_2)

/-- **Scott 1972, Proposition 1.5.** Every `T_0`-space embeds into a power of  0 , and that
power is injective. -/
theorem proposition_1_5 (X : Type u) [TopologicalSpace X] [T0Space X] :
    IsInjectiveSpace.{u,u} (Opens X  ->  Prop)  /\
      IsEmbedding (productOfMemOpens X) := by
  refine <?_, productOfMemOpens_isEmbedding X>
  exact isInjectiveSpace_sierpinski_power (Opens X)

theorem t0_isEmbedding_into_sierpinski_power (X : Type u) [TopologicalSpace X] [T0Space X] :
    IsEmbedding (productOfMemOpens X) :=
  (proposition_1_5 X).2

/-- Scott's notion of retract: a subspace with a continuous retraction. -/
structure IsRetractSubspace (D' D : Type u) [TopologicalSpace D'] [TopologicalSpace D] where
  section' : C(D', D)
  isEmbedding_section : IsEmbedding section'
  retraction : C(D, D')
  retraction_section : forall  d, retraction (section' d) = d
```

```
/-- **Scott 1972, Corollary 1.6.** Injective spaces are exactly retracts of powers of  O . -/
theorem corollary_1_6 (D : Type u) [TopologicalSpace D] [T0Space D] :
    IsInjectiveSpace.{u,u} D  <->  exists  (? : Type u), Nonempty (IsRetractSubspace D (?  ->
    Prop)) := by
  constructor
  * intro hD
    obtain <r, hr> := hD (productOfMemOpens D) (proposition_1_5 D).2 (ContinuousMap.id D)
    refine <Opens D, <<productOfMemOpens D, (proposition_1_5 D).2, r, hr>>>
  * rintro <?, <R>>
    exact IsInjectiveSpace.of_retract (isInjectiveSpace_sierpinski_power ?) R.section' R.
    retraction
      R.retraction_section

/-- **Scott 1972, Corollary 1.7.** A space is injective iff it is a retract of every
super-space of which it is a subspace. -/
theorem corollary_1_7 (D : Type u) [TopologicalSpace D] [T0Space D] :
    IsInjectiveSpace.{u,u} D  <->
      forall  {Y : Type u} [TopologicalSpace Y] (e : D  ->  Y), IsEmbedding e  ->
        exists  r : C(Y, D), forall  d, r (e d) = d := by
  constructor
  * intro hD Y _ e he
    obtain <r, hr> := hD e he (ContinuousMap.id D)
    exact <r, hr>
  * intro hD
    obtain <r, hr> := hD (productOfMemOpens D) (proposition_1_5 D).2
    exact IsInjectiveSpace.of_retract (isInjectiveSpace_sierpinski_power (Opens D))
      (productOfMemOpens D) r hr
```

A.3 Scott1972/ContinuousLattice/WayBelow.lean

229 lines.

Lean 4 source

```
import Mathlib.Order.CompleteLattice.Basic
import Mathlib.Order.UpperLower.Basic
import Mathlib.Order.Directed

/-!
# The induced (Scott) topology and the way-below relation (Scott 1972, S2)

This file formalizes the core of Scott's S2 *Continuous Lattices*: the topology a complete
lattice carries intrinsically (Scott's "induced topology", today the **Scott topology**),
the **way-below relation** ` << `, the basic properties of ` << ` (Scott's Proposition 2.2),
    and
the definition of a **continuous lattice** (Scott's Definition 2.3).

Scott defines the induced topology on a partially ordered set `D` by declaring `U` open iff

* (i)  `U` is an upper set; and
* (ii) whenever `S  subseteq  D` is directed (and, in this paper, non-empty), ` sqsup S` exists
     and
       ` sqsup S  in  U`, then `S  I  U  !=  {}`.

He then defines `x  <<  y` ("`x` is way below `y`") to mean `y  in  interior {z | x  sqsub  z
    }`, the
interior being taken in this induced topology. We encode "Scott-open" as the predicate
`ScottOpen` and ` << ` as `WayBelow`, witnessing the interior by a Scott-open neighbourhood
contained in the principal up-set `Set.Ici x = {z | x  <=  z}`. This is faithful to Scott's
topological definition (rather than the order-theoretic shortcut), and as a result Scott's
Proposition 2.2 (vi) and (vii) fall straight out of the open-set axioms.
```

```
This is the classical/topological version of the theory, so we reason classically.
-/

namespace Scott1972.ContinuousLattice

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, S2, the induced topology.** `U` is *Scott-open* when it is an upper set and
is inaccessible by suprema of non-empty directed sets: if a non-empty directed `S` has its
supremum in `U`, then some member of `S` already lies in `U`. -/
def ScottOpen (U : Set D) : Prop :=
  IsUpperSet U ∧
    ∀ ⦃S : Set D⦄, S.Nonempty → DirectedOn (· ≤ ·) S → sSup S ∈ U → (S ∩
     U).Nonempty

theorem ScottOpen.isUpperSet {U : Set D} (h : ScottOpen U) : IsUpperSet U := h.1

theorem scottOpen_univ : ScottOpen (Set.univ : Set D) := by
  refine ⟨isUpperSet_univ, fun S hS _ _ => ?_⟩
  obtain ⟨s, hs⟩ := hS
  exact ⟨s, hs, Set.mem_univ s⟩

theorem scottOpen_inter {U V : Set D} (hU : ScottOpen U) (hV : ScottOpen V) :
    ScottOpen (U ∩ V) := by
  refine ⟨hU.1.inter hV.1, fun S hS hSdir hmem => ?_⟩
  obtain ⟨s_1, hs_1S, hs_1U⟩ := hU.2 hS hSdir hmem.1
  obtain ⟨s_2, hs_2S, hs_2V⟩ := hV.2 hS hSdir hmem.2
  obtain ⟨s_3, hs_3S, h_1, h_2⟩ := hSdir s_1 hs_1S s_2 hs_2S
  exact ⟨s_3, hs_3S, hU.1 h_1 hs_1U, hV.1 h_2 hs_2V⟩

theorem scottOpen_sUnion {C : Set (Set D)} (hC : ∀ U ∈ C, ScottOpen U) :
    ScottOpen (⋃₀ C) := by
  refine ⟨isUpperSet_sUnion fun U hU => (hC U hU).1, fun S hS hSdir hmem => ?_⟩
  obtain ⟨U, hUC, hUmem⟩ := hmem
  obtain ⟨s, hsS, hsU⟩ := (hC U hUC).2 hS hSdir hUmem
  exact ⟨s, hsS, U, hUC, hsU⟩

/-- **Scott 1972, S2.** The *way-below* relation: `x ≪ y` iff `y` lies in the interior of
    the
principal up-set `Set.Ici x` for the induced topology, witnessed by a Scott-open
neighbourhood of `y` contained in `Set.Ici x`. -/
def WayBelow (x y : D) : Prop :=
  ∃ U : Set D, ScottOpen U ∧ y ∈ U ∧ U ⊆ Set.Ici x

@[inherit_doc] scoped infix:50 " ≪ " => WayBelow

/-- **Scott 1972, Proposition 2.2(v).** `x ≪ y` implies `x ≤ y`. -/
theorem WayBelow.le {x y : D} (h : x ≪ y) : x ≤ y := by
  obtain ⟨U, _, hyU, hsub⟩ := h
  exact hsub hyU

/-- **Scott 1972, Proposition 2.2(i).** `False ≪ x` for every `x`. -/
theorem bot_wayBelow (x : D) : (False : D) ≪ x :=
  ⟨Set.univ, scottOpen_univ, Set.mem_univ x, fun _ _ => bot_le⟩

/-- **Scott 1972, Proposition 2.2(iii).** `x ≪ y` and `y ≤ z` imply `x ≪ z` (monotone
    on the
right). -/
theorem WayBelow.trans_le {x y z : D} (h : x ≪ y) (hyz : y ≤ z) : x ≪ z := by
  obtain ⟨U, hU, hyU, hsub⟩ := h
  exact ⟨U, hU, hU.1 hyz hyU, hsub⟩

/-- **Scott 1972, Proposition 2.2(iv).** `x ≤ y` and `y ≪ z` imply `x ≪ z` (monotone
```

```
    on the
left). -/
theorem WayBelow.le_trans {x y z : D} (hxy : x  <=  y) (h : y  <<  z) : x  <<  z := by
  obtain <U, hU, hzU, hsub> := h
  refine <U, hU, hzU, fun w hw => ?_>
  have hyw : y  <=  w := Set.mem_Ici.1 (hsub hw)
  exact Set.mem_Ici.2 (hxy.trans hyw)

/-- **Scott 1972, Proposition 2.2(ii).** `x  <<  z` and `y  <<  z` imply `x  sqsup  y  <<  z`.
    -/
theorem WayBelow.sup {x y z : D} (hx : x  <<  z) (hy : y  <<  z) : x  sqsup  y  <<  z := by
  obtain <U, hU, hzU, hUsub> := hx
  obtain <V, hV, hzV, hVsub> := hy
  refine <U  I  V, scottOpen_inter hU hV, <hzU, hzV>, fun w hw => ?_>
  exact Set.mem_Ici.2 (sup_le (hUsub hw.1) (hVsub hw.2))

/-- Auxiliary: the up-set `{z | x  <<  z}` is itself Scott-open. (This is *not* Scott's
Proposition 2.2(vi) -- see `wayBelow_self_iff_scottOpen_Ici` for that -- but it is a standard
and useful fact, the openness of the sets ` Up x`.) -/
theorem scottOpen_wayBelow (x : D) : ScottOpen {z | x  <<  z} := by
  refine <fun a b hab ha => ha.trans_le hab, fun S hS hSdir hmem => ?_>
  obtain <U, hU, hsupU, hsub> := hmem
  obtain <s, hsS, hsU> := hU.2 hS hSdir hsupU
  exact <s, hsS, <U, hU, hsU, hsub>>

/-- **Scott 1972, Proposition 2.2(vi).** `x  <<  x` iff the principal up-set `{z | x  sqsub  z
    }` (i.e.
`Set.Ici x`) is Scott-open. This characterizes the *compact* (finite, isolated) elements:
`x` is compact exactly when `x` is open. -/
theorem wayBelow_self_iff_scottOpen_Ici {x : D} : x  <<  x  <->  ScottOpen (Set.Ici x) := by
  constructor
  * rintro <U, hU, hxU, hsub>
    -- `Ici x = U`: `Ici x  subseteq  U` since `U` is upper and `x  in  U`; `U  subseteq  Ici x
    ` is `hsub`.
    have hIci : Set.Ici x = U :=
      le_antisymm (fun w hw => hU.1 (Set.mem_Ici.1 hw) hxU) hsub
    rw [hIci]; exact hU
  * intro hopen
    exact <Set.Ici x, hopen, Set.self_mem_Ici, le_refl _>

/-- **Scott 1972, Proposition 2.2(vii).** For a non-empty directed set `S`, `x  <<   sqsup S`
    iff
`x  <<  y` for some `y  in  S`. The forward direction is exactly inaccessibility of a Scott-
    open
set; the backward direction is monotonicity on the right. -/
theorem wayBelow_sSup_iff {x : D} {S : Set D} (hS : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) : x  <<  sSup S  <->  exists  y  in  S, x  <<  y := by
  constructor
  * rintro <U, hU, hsupU, hsub>
    obtain <s, hsS, hsU> := hU.2 hS hSdir hsupU
    exact <s, hsS, U, hU, hsU, hsub>
  * rintro <y, hyS, hxy>
    exact hxy.trans_le (le_sSup hyS)

/-- **Scott 1972, Definition 2.3.** A complete lattice `D` is a *continuous lattice* when every
element is the supremum of the elements way below it: `y =  sqsup  {x | x  <<  y}`. -/
def IsContinuousLattice (D : Type*) [CompleteLattice D] : Prop :=
  forall  y : D, IsLUB {x | x  <<  y} y

/-- In a continuous lattice, `y` is the actual supremum of `{x | x  <<  y}`. -/
theorem IsContinuousLattice.sSup_wayBelow (h : IsContinuousLattice D) (y : D) :
    sSup {x | x  <<  y} = y :=
  (h y).sSup_eq
```

```
/-- The set `{x | x  <<  y}` of elements way below `y` is directed: it is closed under binary
joins by Proposition 2.2(ii) (`WayBelow.sup`). This holds in *any* complete lattice. -/
theorem directedOn_wayBelow (y : D) : DirectedOn (*  <=  *) {x | x  <<  y} :=
  fun a ha b hb => <a  sqsup  b, ha.sup hb, le_sup_left, le_sup_right>

/-- **Interpolation property of ` << `.** In a continuous lattice, the way-below relation
interpolates: `a  <<  c` implies there is some `b` with `a  <<  b  <<  c`.

The proof runs Scott's standard argument: the set `M = {m | exists  x, m  <<  x  /\  x  <<  c}`
     is directed
(using directedness of `{*  <<  x}` twice) and has supremum `c` (using continuity twice). Hence
`a  <<  c =  sqsup M` with `M` directed forces `a  <<  m` for some `m  in  M`, say `m  <<  x
    <<  c`; then
`a  <<  m  <=  x` gives `a  <<  x  <<  c`, so `b := x` works. -/
theorem wayBelow_interpolate (hD : IsContinuousLattice D) {a c : D} (hac : a  <<  c) :
    exists  b, a  <<  b  /\  b  <<  c := by
  set M : Set D := {m | exists  x, m  <<  x  /\  x  <<  c} with hM
  have hMdir : DirectedOn (*  <=  *) M := by
    rintro m_1 <x_1, hm_1x_1, hx_1c> m_2 <x_2, hm_2x_2, hx_2c>
    obtain <x_3, hx_3c, hx_1_3, hx_2_3> := directedOn_wayBelow c x_1 hx_1c x_2 hx_2c
    obtain <m_3, hm_3x_3, hm_1m_3, hm_2m_3> :=
      directedOn_wayBelow x_3 m_1 (hm_1x_1.trans_le hx_1_3) m_2 (hm_2x_2.trans_le hx_2_3)
    exact <m_3, <x_3, hm_3x_3, hx_3c>, hm_1m_3, hm_2m_3>
  have hMne : M.Nonempty := <False, a, bot_wayBelow a, hac>
  have hsupM : sSup M = c := by
    refine le_antisymm (sSup_le ?_) ?_
    * rintro m <x, hmx, hxc>
      exact hmx.le.trans hxc.le
    * rw [ <-  hD.sSup_wayBelow c]
      refine sSup_le fun x hxc => ?_
      rw [ <-  hD.sSup_wayBelow x]
      exact sSup_le fun m hmx => le_sSup <x, hmx, hxc>
  rw [ <-  hsupM] at hac
  obtain <m, <x, hmx, hxc>, ham> := (wayBelow_sSup_iff hMne hMdir).1 hac
  exact <x, ham.trans_le hmx.le, hxc>

/-- In a continuous lattice the sets ` Up a = {z | a  <<  z}` form a basis of the Scott
    topology:
every Scott-open `U` containing `z` contains some basic neighbourhood ` Up a` of `z`. Indeed
`z =  sqsup {a | a  <<  z}` is a directed supremum lying in the open set `U`, so some `a  <<  z
    ` already
lies in `U`, and then ` Up a  subseteq  a  subseteq  U`. -/
theorem exists_wayBelow_subset (hD : IsContinuousLattice D) {U : Set D} (hU : ScottOpen U)
    {z : D} (hz : z  in  U) : exists  a, a  <<  z  /\  {w | a  <<  w}  subseteq  U := by
  have hne : {a | a  <<  z}.Nonempty := <False, bot_wayBelow z>
  have hsup : sSup {a | a  <<  z}  in  U := by rw [hD.sSup_wayBelow z]; exact hz
  obtain <a, haz, haU> := hU.2 hne (directedOn_wayBelow z) hsup
  exact <a, haz, fun w hw => hU.1 hw.le haU>

/-- A strengthening of `exists_wayBelow_subset`: the witness `a  <<  z` can be taken with the
    whole
principal up-set `Set.Ici a` (not merely ` Up a`) inside `U`. The element `a` produced lies in
    the
open `U`, which is upper, so `a  subseteq  U`. -/
theorem exists_wayBelow_Ici_subset (hD : IsContinuousLattice D) {U : Set D} (hU : ScottOpen U)
    {z : D} (hz : z  in  U) : exists  a, a  <<  z  /\  Set.Ici a  subseteq  U := by
  have hne : {a | a  <<  z}.Nonempty := <False, bot_wayBelow z>
  have hsup : sSup {a | a  <<  z}  in  U := by rw [hD.sSup_wayBelow z]; exact hz
  obtain <a, haz, haU> := hU.2 hne (directedOn_wayBelow z) hsup
  exact <a, haz, fun w hw => hU.1 (Set.mem_Ici.1 hw) haU>

/-- The infimum of a Scott-open neighbourhood of `y` is way below `y`: the open set is itself
```

```
the required witness. Scott uses this in moving between Definition 2.3 and Proposition 2.4. -/
theorem sInf_wayBelow {U : Set D} (hU : ScottOpen U) {y : D} (hy : y  in  U) :
    sInf U  <<  y :=
  <U, hU, hy, fun _ hz => Set.mem_Ici.2 (sInf_le hz)>

/-- **Scott 1972, Proposition 2.4.** A complete lattice is continuous iff every element is the
supremum of the infima of its open neighbourhoods: `y =  sqsup  { sqcap U : y  in  U open}`.
    This is Scott's
alternate form of Definition 2.3. -/
theorem isContinuousLattice_iff_isLUB_sInf_nhds :
    IsContinuousLattice D  <->
      forall  y : D, IsLUB {a : D | exists  U, ScottOpen U  /\  y  in  U  /\  a = sInf U} y :=
    by
  constructor
  * intro h y
    refine <?_, ?_>
    * rintro a <U, hU, hyU, rfl>
      exact (sInf_wayBelow hU hyU).le
    * intro b hb
      refine (h y).2 ?_
      intro x hx
      obtain <U, hU, hyU, hsub> := hx
      have hxle : x  <=  sInf U := le_sInf fun _ hz => Set.mem_Ici.1 (hsub hz)
      exact hxle.trans (hb <U, hU, hyU, rfl>)
  * intro h y
    refine <fun x hx => hx.le, ?_>
    intro b hb
    refine (h y).2 ?_
    rintro a <U, hU, hyU, rfl>
    exact hb (sInf_wayBelow hU hyU)

end Scott1972.ContinuousLattice
```

A.4 Scott1972/ContinuousLattice/Specialization.lean

125 lines.

Lean 4 source

```
import Scott1972.ContinuousLattice.WayBelow
import Mathlib.Topology.Inseparable
import Mathlib.Topology.Separation.Basic
import Mathlib.Topology.Order.ScottTopology
import Mathlib.Order.DirSupClosed

/-!
# Specialization order and Scott topology (Scott 1972, S2 opening)

Scott's S2 begins with the specialization order on a `T_0`-space and the induced (Scott)
topology on a complete lattice. Proposition 2.1 (monotone nets and least upper bounds) is
split into its two directions; the convergence-to-below direction is the mathematically
heavier half and is recorded as `proposition_2_1_of_le`.
-/

namespace Scott1972.ContinuousLattice

open Topology Set

universe u

variable {X D : Type*} [TopologicalSpace X] [CompleteLattice D]
```

```
/-! ### Specialization order -/

/-- **Scott 1972, S2.** The *specialization order*: `x  sqsub  y` when `x  in  U` open implies
    `y  in  U`. -/
def SpecializationLe (x y : X) : Prop :=
  forall  U, IsOpen U  ->  x  in  U  ->  y  in  U

instance specializationPreorder : Preorder X where
  le := SpecializationLe
  le_refl x := fun _ _ hx => hx
  le_trans x y z hxy hyz U hU hxU := hyz U hU (hxy U hU hxU)

theorem specializationLe_antisymm [TOSpace X] {x y : X}
    (hxy : SpecializationLe x y) (hyx : SpecializationLe y x) : x = y :=
  Inseparable.eq (inseparable_iff_specializes_and.2
    <specializes_iff_forall_open.2 fun s hs hy => hyx s hs hy,
     specializes_iff_forall_open.2 fun s hs hx => hxy s hs hx>)

/-! ### Scott topology from `ScottOpen` -/

/-- Scott's induced topology on a complete lattice, realized as mathlib's Scott topology. -/
@[reducible] noncomputable def scottTopologicalSpace : TopologicalSpace D :=
  Topology.scott D univ

theorem ScottOpen_iff_dirSupInacc {U : Set D} : ScottOpen U  <->  IsUpperSet U  /\  DirSupInacc
     U := by
  constructor
  * intro <hU, hU'>
    refine <hU, fun d hd_1 hd_2 a ha hmem => ?_>
    rw [ <-  IsLUB.sSup_eq ha] at hmem
    obtain <s, hs, hsU> := hU' hd_1 hd_2 hmem
    exact <s, hs, hsU>
  * intro <hU, hU'>
    refine <hU, fun d hd_1 hd_2 hmem => ?_>
    obtain <s, hs, hsU> := hU' hd_1 hd_2 (isLUB_sSup d) hmem
    exact <s, hs, hsU>

theorem isOpen_iff_scottOpen {U : Set D} : @IsOpen D scottTopologicalSpace U  <->  ScottOpen U
    := by
  rw [ScottOpen_iff_dirSupInacc, scottTopologicalSpace]
  haveI : IsScott (WithScott D) univ := inferInstance
  rw [show @IsOpen D (Topology.scott D univ) U = @IsOpen (WithScott D) inferInstance U from rfl
    ,
    IsScott.isOpen_iff_isUpperSet_and_dirSupInaccOn (a := WithScott D) (D := univ),
    dirSupInaccOn_univ]
  rfl

/-- Scott-open sets in our sense agree with mathlib's Scott topology (alias). -/
theorem isOpen_scott_iff_scottOpen {U : Set D} :
    @IsOpen D (Topology.scott D univ) U  <->  ScottOpen U :=
  isOpen_iff_scottOpen

/-! ### Monotone nets and Proposition 2.1 -/

variable {? : Type u} [Preorder ?] [IsDirected ? (*  <=  *)]

def IsMonotoneNet (x : ?  ->  D) : Prop :=
  Monotone x

def ScottConvergesTo (x : ?  ->  D) (y : D) : Prop :=
  forall  U, ScottOpen U  ->  y  in  U  ->  exists  i, forall  j  >=  i, x j  in  U

variable {x : ?  ->  D} {L y : D}
```

```
/-- **Scott 1972, Proposition 2.1 (backward).** If `y ≤ L` and `L` is the lub of a monotone
net, then the net converges to `y` in the Scott topology. -/
theorem proposition_2_1_of_le [Nonempty ?] (hx : IsMonotoneNet x) (hL : IsLUB (range x) L)
    (hyL : y ≤ L) : ScottConvergesTo x y := by
  intro U hU hyU
  have hdir : DirectedOn (· ≤ ·) (range x) := by
    rintro _ ⟨i, rfl⟩ _ ⟨j, rfl⟩
    obtain ⟨k, hik, hjk⟩ := IsDirected.directed (r := (· ≤ ·)) i j
    exact ⟨x k, ⟨k, rfl⟩, hx hik, hx hjk⟩
  have hLU : sSup (range x) ∈ U := by
    rw [IsLUB.sSup_eq hL]
    exact hU.1 hyL hyU
  obtain ⟨s, hsS, hsU⟩ := hU.2 (Set.range_nonempty x) hdir hLU
  obtain ⟨i_0, rfl⟩ := hsS
  refine ⟨i_0, fun j hj => hU.1 (hx hj) hsU⟩

/-- The complement of a principal lower set `Iic L` is Scott-open: it is an upper set, and it
is inaccessible by directed suprema because if every member of a directed `S` lies below `L`
then so does ` sqsup S`. -/
theorem scottOpen_not_le (L : D) : ScottOpen {z : D | ¬ z ≤ L} := by
  refine ⟨fun a b hab ha hb => ha (le_trans hab hb), fun S hSne hSdir hmem => ?_⟩
  by_contra hcon
  refine hmem (sSup_le fun s hs => ?_)
  by_contra hsL
  exact hcon ⟨s, hs, hsL⟩

omit [IsDirected ? (· ≤ ·)] in
/-- **Scott 1972, Proposition 2.1 (forward).** If a monotone net converges to `y` in the Scott
topology and `L` is its least upper bound, then `y ≤ L`. -/
theorem proposition_2_1_le_of_converges (hL : IsLUB (range x) L)
    (hconv : ScottConvergesTo x y) : y ≤ L := by
  by_contra hyL
  obtain ⟨i, hi⟩ := hconv {z : D | ¬ z ≤ L} (scottOpen_not_le L) hyL
  exact hi i le_rfl (hL.1 ⟨i, rfl⟩)

/-- **Scott 1972, Proposition 2.1.** A monotone net with least upper bound `L` converges to
`y` in the Scott topology iff `y ⊑ L = ⊔ {x_i}`. -/
theorem proposition_2_1 [Nonempty ?] (hx : IsMonotoneNet x) (hL : IsLUB (range x) L) :
    ScottConvergesTo x y ↔ y ≤ L :=
  ⟨fun hconv => proposition_2_1_le_of_converges hL hconv, proposition_2_1_of_le hx hL⟩

end Scott1972.ContinuousLattice
```

A.5 Scott1972/ContinuousLattice/ScottMaps.lean

204 lines.

Lean 4 source

```
import Scott1972.ContinuousLattice.Specialization
import Mathlib.Order.ScottContinuity
import Mathlib.Topology.Order.ScottTopology

/-!
# Scott-continuous maps (Scott 1972, §2.5-2.7)
-/

namespace Scott1972.ContinuousLattice

open Set Topology
```

```
variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

def PreservesDirectedSup (f : D → D') : Prop :=
  forall ?S : Set D?, S.Nonempty → DirectedOn (* <= *) S → f (sSup S) = sSup (f '' S)

theorem preservesDirectedSup_monotone {f : D → D'} (hf : PreservesDirectedSup f) :
    Monotone f := by
  intro x y hxy
  have hdir : DirectedOn (* <= *) ({x, y} : Set D) := directedOn_pair hxy
  have hS : ({x, y} : Set D).Nonempty := <x, Set.mem_insert _ _>
  have hsup : sSup ({x, y} : Set D) = y := by
    calc sSup ({x, y} : Set D) = x ⊔ y := sSup_pair
      _ = y := by apply le_antisymm; exact sup_le hxy le_rfl; exact le_sup_right
  have heq := hf hS hdir
  rw [hsup, Set.image_pair] at heq
  exact le_trans (le_sSup (Set.mem_insert _ _)) heq.symm.le

theorem scottOpen_preimage {f : D → D'} (hf : PreservesDirectedSup f) {U : Set D'}
    (hU : ScottOpen U) : ScottOpen (f ⁻¹' U) := by
  have hmono := preservesDirectedSup_monotone hf
  refine <fun a b hab ha => hU.1 (hmono hab) ha, fun S hS hSdir hmem => ?_>
  rw [Set.mem_preimage] at hmem
  have hmem' : sSup (f '' S) ∈ U := by rw [ <- hf hS hSdir]; exact hmem
  obtain <s, hsS, hsU> := hU.2 (Set.image_nonempty.2 hS)
    (fun s hs t ht => by
      obtain <a, haS, rfl> := hs
      obtain <b, hbS, rfl> := ht
      obtain <c, hcS, hac, hbc> := hSdir a haS b hbS
      exact <f c, Set.mem_image_of_mem f hcS, hmono hac, hmono hbc>) hmem'
  obtain <a, haS, rfl> := hsS
  exact <a, haS, Set.mem_preimage.2 hsU>

theorem continuous_of_preservesDirectedSup {f : D → D'} (hf : PreservesDirectedSup f) :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace f := by
  rw [continuous_def]
  intro U hU
  rw [isOpen_iff_scottOpen] at hU ⊢
  exact scottOpen_preimage hf hU

theorem continuous_preservesDirectedSup {f : D → D'}
    (hf : @Continuous D D' scottTopologicalSpace scottTopologicalSpace f) :
    PreservesDirectedSup f := by
  have hf' :
      @Continuous (WithScott D) (WithScott D') _ _ (WithScott.toScott ∘ f ∘ WithScott
    .ofScott) := by
    simpa [WithScott.toScott, WithScott.ofScott] using hf
  have hsc : ScottContinuous (WithScott.toScott ∘ f ∘ WithScott.ofScott) :=
    scottContinuousOn_univ.1 <|
      (Topology.IsScott.scottContinuousOn_iff_continuous (a := WithScott D) (D := univ)
        (fun _ _ _ => trivial)).2 hf'
  intro S hS hSdir
  have h := hsc hS hSdir (isLUB_sSup S)
  simp only [Function.comp_def, WithScott.toScott, WithScott.ofScott] at h
  exact h.sSup_eq.symm

/-- **Scott 1972, Proposition 2.5.** Scott continuity ↔ preservation of directed suprema.
    -/
theorem proposition_2_5 (f : D → D') :
    (@Continuous D D' scottTopologicalSpace scottTopologicalSpace f) ↔
      PreservesDirectedSup f :=
  <continuous_preservesDirectedSup, fun hf => continuous_of_preservesDirectedSup hf>

/-! ### Proposition 2.6 -/
```

```
/-- **Scott 1972, Proposition 2.6.** A function of several variables between complete lattices
    is
(Scott-)continuous jointly iff it is continuous in each variable separately. Continuity is
    phrased
as preservation of directed suprema (Proposition 2.5); it suffices to treat two variables, the
product `D  x  D'` carrying the componentwise complete-lattice structure (whose induced
    topology is
the product topology). -/
theorem proposition_2_6 (f : D  x  D'  ->  D'') :
    PreservesDirectedSup f  <->
      (forall  y, PreservesDirectedSup fun x => f (x, y))  /\
        (forall  x, PreservesDirectedSup fun y => f (x, y)) := by
  constructor
  * -- joint continuity ? separate continuity (precompose with the continuous slice maps)
    intro hf
    refine <fun y S hS hSdir => ?_, fun x S hS hSdir => ?_>
    * set T : Set (D  x  D') := (fun x => (x, y)) '' S with hT
      have hTne : T.Nonempty := hS.image _
      have hTdir : DirectedOn (*  <=  *) T := by
        rintro _ <a, ha, rfl> _ <b, hb, rfl>
        obtain <c, hc, hac, hbc> := hSdir a ha b hb
        exact <(c, y), Set.mem_image_of_mem _ hc, <hac, le_rfl>, <hbc, le_rfl>>
      have hfst : Prod.fst '' T = S := by rw [hT, Set.image_image]; simp
      have hsnd : Prod.snd '' T = {y} := by rw [hT, Set.image_image]; exact hS.image_const y
      have hsupT : sSup T = (sSup S, y) := by
        have e1 : (sSup T).1 = sSup S := by rw [Prod.fst_sSup, hfst]
        have e2 : (sSup T).2 = y := by rw [Prod.snd_sSup, hsnd, sSup_singleton]
        exact Prod.ext_iff.mpr <e1, e2>
      have h := hf hTne hTdir
      rw [hsupT, hT, Set.image_image] at h
      simpa using h
    * set T : Set (D  x  D') := (fun y => (x, y)) '' S with hT
      have hTne : T.Nonempty := hS.image _
      have hTdir : DirectedOn (*  <=  *) T := by
        rintro _ <a, ha, rfl> _ <b, hb, rfl>
        obtain <c, hc, hac, hbc> := hSdir a ha b hb
        exact <(x, c), Set.mem_image_of_mem _ hc, <le_rfl, hac>, <le_rfl, hbc>>
      have hfst : Prod.fst '' T = {x} := by rw [hT, Set.image_image]; exact hS.image_const x
      have hsnd : Prod.snd '' T = S := by rw [hT, Set.image_image]; simp
      have hsupT : sSup T = (x, sSup S) := by
        have e1 : (sSup T).1 = x := by rw [Prod.fst_sSup, hfst, sSup_singleton]
        have e2 : (sSup T).2 = sSup S := by rw [Prod.snd_sSup, hsnd]
        exact Prod.ext_iff.mpr <e1, e2>
      have h := hf hTne hTdir
      rw [hsupT, hT, Set.image_image] at h
      simpa using h
  * -- separate continuity ? joint continuity (Scott 1972's directedness argument)
    rintro <h1, h2> Sstar hne hdir
    have hmono1 : forall  y, Monotone fun x => f (x, y) := fun y =>
    preservesDirectedSup_monotone (h1 y)
    have hmono2 : forall  x, Monotone fun y => f (x, y) := fun x =>
    preservesDirectedSup_monotone (h2 x)
    have hmono : Monotone f := by
      rintro <a, b> <c, d> hpq
      exact le_trans (hmono1 b hpq.1) (hmono2 c hpq.2)
    have hSne : (Prod.fst '' Sstar).Nonempty := hne.image _
    have hS'ne : (Prod.snd '' Sstar).Nonempty := hne.image _
    have hSdir : DirectedOn (*  <=  *) (Prod.fst '' Sstar) := hdir.fst
    have hS'dir : DirectedOn (*  <=  *) (Prod.snd '' Sstar) := hdir.snd
    have hsupStar : sSup Sstar = (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar)) :=
      Prod.ext_iff.mpr <Prod.fst_sSup _, Prod.snd_sSup _>
    apply le_antisymm
```

```
    * rw [hsupStar]
      have e1 : f (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
          = sSup ((fun x => f (x, sSup (Prod.snd '' Sstar))) '' (Prod.fst '' Sstar)) :=
        h1 (sSup (Prod.snd '' Sstar)) hSne hSdir
      rw [e1]
      apply sSup_le
      rintro _ <x, hx, rfl>
      show f (x, sSup (Prod.snd '' Sstar))  <=  sSup (f '' Sstar)
      have e2 : f (x, sSup (Prod.snd '' Sstar))
          = sSup ((fun y => f (x, y)) '' (Prod.snd '' Sstar)) :=
        h2 x hS'ne hS'dir
      rw [e2]
      apply sSup_le
      rintro _ <y, hy, rfl>
      show f (x, y)  <=  sSup (f '' Sstar)
      obtain <p, hpS, hpfst> := hx
      obtain <q, hqS, hqsnd> := hy
      obtain <r, hrS, hpr, hqr> := hdir p hpS q hqS
      have hxr : x  <=  r.1 := hpfst  |>  hpr.1
      have hyr : y  <=  r.2 := hqsnd  |>  hqr.2
      calc f (x, y)  <=  f (r.1, r.2) := hmono <hxr, hyr>
        _ = f r := by rw [Prod.mk.eta]
        _  <=  sSup (f '' Sstar) := le_sSup (Set.mem_image_of_mem f hrS)
    * apply sSup_le
      rintro _ <p, hp, rfl>
      exact hmono (le_sSup hp)

/-! ### Proposition 2.7 -/

/-- **Scott 1972, Proposition 2.7 (join).** Binary join is Scott-continuous on every complete
lattice; this is the ` sqsup `-part of Scott's 2.7. -/
theorem proposition_2_7_sup :
    @Continuous (D  x  D) D scottTopologicalSpace scottTopologicalSpace (fun p : D  x  D => p.1
      sqsup  p.2) :=
  continuous_of_preservesDirectedSup <| by
    intro S hS hSdir
    simpa using (ScottContinuous.sup_2 (b := D) (d := S) hS hSdir (isLUB_sSup S)).sSup_eq.symm

theorem meet_preservesDirectedSup (x : D) (hD : IsContinuousLattice D) :
    PreservesDirectedSup (fun z => x  sqcap  z) := by
  intro S hS hSdir
  apply le_antisymm
  * have hle : sSup {t | t  <<  x  sqcap  sSup S}  <=  sSup (Set.image (fun z => x  sqcap  z) S
    ) := by
      apply sSup_le
      intro t ht
      obtain <w, hwS, ht_w> := (wayBelow_sSup_iff hS hSdir).1 (ht.trans_le inf_le_right)
      have hmem : x  sqcap  w  in  Set.image (fun z : D => x  sqcap  z) S :=
        Set.mem_image_of_mem (fun z => x  sqcap  z) hwS
      have hbound : x  sqcap  w  <=  sSup (Set.image (fun z => x  sqcap  z) S) := le_sSup hmem
      exact (le_inf (ht.le.trans inf_le_left) ht_w.le).trans hbound
    dsimp only
    rw [ <-  hD.sSup_wayBelow (x  sqcap  sSup S)]
    exact hle
  * apply sSup_le
    intro z hz
    obtain <w, hwS, rfl> := hz
    exact inf_le_inf le_rfl (le_sSup hwS)

/-- **Scott 1972, Proposition 2.7 (meet, separate).** On a continuous lattice, `x  |->  x
    sqcap  y`
and `y  |->  x  sqcap  y` are Scott-continuous; Scott's full 2.7 also covers joint continuity
    on the
```

```
product via Proposition 2.6. -/
theorem proposition_2_7_inf_left (hD : IsContinuousLattice D) (y : D) :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace (fun x => x  sqcap  y) :=
  continuous_of_preservesDirectedSup <| by
    intro S' hS' hSdir'
    rw [show (fun x => x  sqcap  y) = fun z => y  sqcap  z from funext fun x => inf_comm x y]
    have h := meet_preservesDirectedSup y hD
    exact h (S := S') hS' hSdir'

theorem proposition_2_7_inf_right (hD : IsContinuousLattice D) (x : D) :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace (fun y => x  sqcap  y) :=
  continuous_of_preservesDirectedSup (meet_preservesDirectedSup x hD)

end Scott1972.ContinuousLattice
```

A.6 Scott1972/ContinuousLattice/MilnerCorrection.lean

56 lines.

Lean 4 source

```
import Scott1972.ContinuousLattice.Specialization

/-!
# March 1972 correction (Scott, pp. 135-136)

Robin Milner observed that Scott's remark before Proposition 2.5 was wrong: a `T_0`-topology
on a complete lattice need not have every open set Scott-open. The two extreme `T_0`-topologies
inducing the same partial order are generated by

* `{x | x  sqsubneq  y}` (the *lower* subbasic sets), and
* `{x | y  sqsub  x}` (principal up-sets).

The corrected proofs of Propositions **2.9**, **2.10**, and **3.3** assume the given
topology is *contained in* the Scott (lattice) topology: every open set of the given
topology is Scott-open.

For Proposition **2.10**, Scott distinguishes ambient joins in `D'` from subspace joins in
a retract `D  subseteq  D'`: the prime in ` sqsup S'` marks the *index* (join taken in `D'`),
    not a
"primed join" operator. The retraction identity used in the corrected argument is
`j( sqsup S') =  sqsup S` for directed `S  subseteq  D` (see `IsContinuousLatticeRetraction.
    retr_ambientSSup_eq_sSup`
in `FunctionSpaces.lean`).

Page **136** adds that function spaces pose no subspace/sup mismatch: lubs are computed
pointwise, so the product (relativized) topology is contained in the Scott topology once
the Milner hypothesis is in place.
-/

namespace Scott1972.ContinuousLattice

open Set Topology

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

/-! ### Extreme `T_0`-topologies inducing the order -/

/-- Lower subbasic set `{x | x  sqsubneq  y}` from Scott's March 1972 correction. -/
def scottLowerSubbasisSet (y : D) : Set D :=
  { x | not  x  <=  y }
```

```
/-- Upper subbasic set `{x | y  sqsub  x}` (principal up-set) from the correction. -/
def scottPrincipalUpSet (y : D) : Set D :=
  Set.Ici y

/-! ### Milner hypothesis for 2.9, 2.10, 3.3 -/

/-- **March 1972 correction.** The given topology `tau` is coarser than (or equal to) the Scott
topology: every set open in `tau` is Scott-open. Equivalently `Opens(tau)  subseteq  Opens(
    Scott)`,
i.e. `scottTopologicalSpace  <=  tau` in mathlib's order on topologies. -/
def CoarserThanScottTopology (tau : TopologicalSpace D) : Prop :=
  scottTopologicalSpace  <=  tau

theorem scottOpen_of_coarserThanScott {tau : TopologicalSpace D} (htau :
    CoarserThanScottTopology tau)
    {U : Set D} (hU : @IsOpen D tau U) : ScottOpen U :=
  isOpen_iff_scottOpen.mp (htau U hU)

end Scott1972.ContinuousLattice
```

A.7 Scott1972/ContinuousLattice/Constructions.lean

508 lines.

Lean 4 source (lines 1–400)

```
import Scott1972.ContinuousLattice.MilnerCorrection
import Scott1972.ContinuousLattice.ScottMaps
import Scott1972.ContinuousLattice.Injective
import Mathlib.Order.Preorder.Finite

/-!
# Continuous lattice constructions (Scott 1972, S2.8-2.12)

The Milner correction (March 1972, pp. 135-136) is in `MilnerCorrection.lean`. Full proofs of
2.8-2.12 under `CoarserThanScottTopology` are the remaining 1972 items; the Sierpinski
injectivity base case (1.2) is already complete.

This file additionally proves Scott's first two example/closure results, the order-theoretic
content of Propositions 2.8 (finite lattices are continuous) and 2.9 (products of continuous
lattices are continuous). The accompanying topological claims of 2.9-2.10 ("the induced
topology agrees with the product / subspace topology") are the parts that require the Milner
correction and remain open.
-/

namespace Scott1972.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- A non-empty **finite** directed set attains its supremum: ` sqsup S  in  S`. A maximal
    element of
`S` (which exists by finiteness) is, by directedness, the greatest element, hence the
supremum. This is the order-theoretic heart of "finite ? continuous" (Scott 1972, 2.8). -/
theorem directedOn_finite_sSup_mem {S : Set D} (hSfin : S.Finite) (hSne : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) : sSup S  in  S := by
  obtain <m, hm> := hSfin.exists_maximal hSne
  have hub : m  in  upperBounds S := by
    intro s hs
    obtain <c, hcS, hmc, hsc> := hSdir m hm.1 s hs
    exact hsc.trans (hm.2 hcS hmc)
  have hlub : IsLUB S m := <hub, fun b hb => hb hm.1>
```

```
  rw [hlub.sSup_eq]
  exact hm.1

/-- **Scott 1972, Proposition 2.8.** A finite lattice is a continuous lattice. In a finite
lattice every principal up-set `Set.Ici y` is Scott-open (a non-empty directed set is finite,
so it attains its supremum), hence `y  <<  y`; therefore `y` is the supremum of `{x | x  <<  y
    }`. -/
theorem proposition_2_8 [Finite D] : IsContinuousLattice D := by
  intro y
  have hopen : ScottOpen (Set.Ici y) := by
    refine <isUpperSet_Ici y, fun S hSne hSdir hmem => ?_>
    exact <sSup S, directedOn_finite_sSup_mem (Set.toFinite S) hSne hSdir, hmem>
  have hyy : y  <<  y := wayBelow_self_iff_scottOpen_Ici.mpr hopen
  exact <fun x hx => hx.le, fun b hb => hb hyy>

/-- **Scott 1972, Proposition 2.9(a) (order-theoretic content).** The Cartesian product of any
family of continuous lattices is a continuous lattice. (Part (b), that the induced topology of
the product agrees with the product topology, is `proposition_2_9_b` below; `proposition_2_9`
bundles the two.)

The key step is that if `a  <<  y_i` in the factor `D_i`, then the cylinder element `[a]?`
(equal to `a` in coordinate `i` and `False` elsewhere) is way below `y` in the product: the
preimage `{z | z_i  in  U}` of a Scott-open witness `U  subseteq  D_i` is Scott-open in the
    product
(suprema are computed coordinatewise). Any upper bound `b` of `{x | x  <<  y}` therefore
dominates every `[a]?`, so `a = ([a]?)_i  <=  b_i`; ranging over `a  <<  y_i` and using
    continuity of
`D_i` gives `y_i  <=  b_i` for all `i`, i.e. `y  <=  b`. -/
theorem proposition_2_9_a {? : Type*} (E : ?  ->  Type*) [forall  i, CompleteLattice (E i)]
    (hE : forall  i, IsContinuousLattice (E i)) : IsContinuousLattice (forall  i, E i) := by
  classical
  intro y
  refine <fun x hx => hx.le, fun b hb => ?_>
  rw [Pi.le_def]
  intro i
  rw [ <-  (hE i).sSup_wayBelow (y i)]
  apply sSup_le
  intro a ha
  set e : (forall  j, E j) := Function.update (False : forall  j, E j) i a with he
  have hei : e i = a := by rw [he]; exact Function.update_self i a _
  have hcyl : e  <<  y := by
    obtain <U, hU, hyiU, hUsub> := ha
    refine <{z : forall  j, E j | z i  in  U}, ?_, hyiU, ?_>
    * refine <fun z w hzw hz => hU.1 (hzw i) hz, fun S hSne hSdir hmem => ?_>
      rw [Set.mem_setOf_eq, sSup_apply_eq_sSup_image] at hmem
      have hdir' : DirectedOn (*  <=  *) (Function.eval i '' S) := by
        rintro _ <f, hf, rfl> _ <g, hg, rfl>
        obtain <h, hhS, hfh, hgh> := hSdir f hf g hg
        exact <Function.eval i h, <h, hhS, rfl>, hfh i, hgh i>
      obtain <t, htimg, htU> := hU.2 (hSne.image _) hdir' hmem
      obtain <f, hfS, rfl> := htimg
      exact <f, hfS, htU>
    * intro z hz
      rw [Set.mem_Ici, Pi.le_def]
      intro j
      rcases eq_or_ne j i with rfl | hji
      * rw [hei]; exact Set.mem_Ici.1 (hUsub hz)
      * rw [he, Function.update_of_ne hji]; exact bot_le
  have hle : e i  <=  b i := (hb hcyl) i
  rw [hei] at hle
  exact hle

/-! ### Proposition 2.9(b): the induced topology of a product is the product topology
```

```
Scott 1972, Proposition 2.9 also asserts that the Scott topology of the product `?_i D_i` of
continuous lattices coincides with the topological product of the Scott topologies of the
    factors.
We prove the two inclusions:

* **Product  subseteq  Scott** (`scottTopologicalSpace  <=  Pi.topologicalSpace`): every
    projection
  `eval i` preserves directed suprema (they are computed coordinatewise), hence is Scott-
    continuous,
  hence the Scott topology of the product is finer than each induced topology, i.e. finer than
    their
  infimum (the product topology).
* **Scott  subseteq  Product** (`Pi.topologicalSpace  <=  scottTopologicalSpace`): given a
    Scott-open `U` and a
  point `z  in  U`, the ` Up a` basis (`exists_wayBelow_Ici_subset`) yields `a  <<  z` with `a
     subseteq  U`. A
  way-below element of a product has **finite support** (`wayBelow_finite_support`) and is
  way-below in each coordinate (`wayBelow_proj`); the resulting finite box
  `?_{i  in  F} eval i ^-1' ( Up a_i-neighbourhood of z_i)` is a product-open neighbourhood of
    `z` inside
  `a  subseteq  U`. -/

/-- Plugging a value into a fixed coordinate, `v  |->  Function.update z i v`, preserves
    directed
suprema: away from `i` the result is the constant `z j`, and at `i` it is the identity. -/
theorem update_preservesDirectedSup {? : Type*} [DecidableEq ?] {E : ?  ->  Type*}
    [forall  i, CompleteLattice (E i)] (z : forall  i, E i) (i : ?) :
    PreservesDirectedSup (fun v : E i => Function.update z i v) := by
  intro T hTne _
  show Function.update z i (sSup T) = sSup ((fun v : E i => Function.update z i v) '' T)
  funext j
  rw [sSup_apply_eq_sSup_image, Set.image_image]
  rcases eq_or_ne j i with hji | hji
  * rw [hji, Function.update_self]
    have h : (fun v : E i => Function.eval i (Function.update z i v)) = id := by
      funext v; simp [Function.eval, Function.update_self]
    rw [h, Set.image_id]
  * rw [Function.update_of_ne hji]
    have h : (fun v : E i => Function.eval j (Function.update z i v)) = fun _ => z j := by
      funext v; simp [Function.eval, Function.update_of_ne hji]
    rw [h, hTne.image_const, sSup_singleton]

/-- A way-below relation in a product projects to each coordinate: if `a  <<  z` in `?_i E_i`
    then
`a_i  <<  z_i`. The witnessing Scott-open neighbourhood of `z_i` is the preimage of a product
    Scott-open
witness under `v  |->  Function.update z i v`, which is Scott-open by `
    update_preservesDirectedSup`. -/
theorem wayBelow_proj {? : Type*} {E : ?  ->  Type*} [forall  i, CompleteLattice (E i)]
    {a z : forall  i, E i} (h : a  <<  z) (i : ?) : a i  <<  z i := by
  classical
  obtain <W, hW, hzW, hWsub> := h
  refine <(fun v : E i => Function.update z i v) ^-1' W, ?_, ?_, ?_>
  * exact scottOpen_preimage (update_preservesDirectedSup z i) hW
  * show Function.update z i (z i)  in  W
    rw [Function.update_eq_self]; exact hzW
  * intro v hv
    have hav : a  <=  Function.update z i v := Set.mem_Ici.1 (hWsub hv)
    have := hav i
    rwa [Function.update_self] at this

/-- A way-below element of a product has finite support: if `a  <<  z` in `?_i E_i` then `a? =
```

```
    False` for
all but finitely many `j`. The finite truncations `Z F = (fun j => if j  in  F then z j else
    False)` form a
directed family with supremum `z`; since `a  <<  z =  sqsup _F Z F`, already `a  <=  Z F` for
    some finite `F`,
forcing `a? = False` off `F`. -/
theorem wayBelow_finite_support {? : Type*} {E : ?  ->  Type*} [forall  i, CompleteLattice (E i
    )]
    {a z : forall  i, E i} (h : a  <<  z) : exists  F : Finset ?, forall  j  notin  F, a j =
    False := by
  classical
  set Z : Finset ?  ->  (forall  j, E j) := fun F j => if j  in  F then z j else False with hZ
  set ? : Set (forall  j, E j) := Set.range Z with h?
  have hmono : Monotone Z := by
    intro F G hFG
    rw [Pi.le_def]; intro j
    simp only [hZ]
    by_cases hjF : j  in  F
    * rw [if_pos hjF, if_pos (hFG hjF)]
    * rw [if_neg hjF]; exact bot_le
  have h?ne : ?.Nonempty := <Z {}, {}, rfl>
  have h?dir : DirectedOn (*  <=  *) ? := by
    rintro _ <F, rfl> _ <G, rfl>
    exact <Z (F  U  G), <F  U  G, rfl>, hmono Finset.subset_union_left,
      hmono Finset.subset_union_right>
  have hsup : sSup ? = z := by
    apply le_antisymm
    * apply sSup_le
      rintro _ <F, rfl>
      rw [Pi.le_def]; intro j
      simp only [hZ]
      by_cases hjF : j  in  F
      * rw [if_pos hjF]
      * rw [if_neg hjF]; exact bot_le
    * rw [Pi.le_def]; intro j
      rw [sSup_apply_eq_sSup_image]
      refine le_sSup <Z {j}, <{j}, rfl>, ?_>
      simp [hZ, Function.eval]
  rw [ <-  hsup] at h
  obtain <d, hd?, had> := (wayBelow_sSup_iff h?ne h?dir).1 h
  obtain <F, rfl> := hd?
  refine <F, fun j hjF => ?_>
  have hj := had.le j
  simp only [hZ, if_neg hjF] at hj
  exact le_bot_iff.1 hj

/-- **Scott 1972, Proposition 2.9(b).** For a family of continuous lattices, the Scott topology
     of
the product coincides with the product of the Scott topologies of the factors. -/
theorem proposition_2_9_b {? : Type*} (E : ?  ->  Type*) [forall  i, CompleteLattice (E i)]
    (hE : forall  i, IsContinuousLattice (E i)) :
    (scottTopologicalSpace : TopologicalSpace (forall  i, E i)) =
      @Pi.topologicalSpace ? E (fun _ => scottTopologicalSpace) := by
  classical
  have hprod : IsContinuousLattice (forall  i, E i) := proposition_2_9_a E hE
  apply le_antisymm
  * -- Product  subseteq  Scott: projections preserve directed suprema, hence are Scott-
    continuous.
    refine le_iInf fun i => ?_
    rw [ <-  continuous_iff_le_induced]
    exact continuous_of_preservesDirectedSup (fun _ _ _ => sSup_apply_eq_sSup_image)
  * -- Scott  subseteq  Product: every Scott-open set is a union of finite product-open boxes.
    intro U hU
```

```
    rw [isOpen_iff_scottOpen] at hU
    rw [@isOpen_iff_forall_mem_open _ (@Pi.topologicalSpace ? E (fun _ => scottTopologicalSpace
    ))]
    intro z hz
    obtain <a, haz, haIci> := exists_wayBelow_Ici_subset hprod hU hz
    obtain <F, hF> := wayBelow_finite_support haz
    have hproj : forall  i, a i  <<  z i := fun i => wayBelow_proj haz i
    choose V hVopen hzV hVsub using hproj
    refine <? i  in  F, (fun w : forall  j, E j => w i) ^-1' V i, ?_, ?_, ?_>
    * -- the box lies inside `a  subseteq  U`
      intro w hw
      rw [Set.mem_iInter_2] at hw
      refine haIci (Set.mem_Ici.2 fun j => ?_)
      by_cases hjF : j  in  F
      * exact Set.mem_Ici.1 (hVsub j (hw j hjF))
      * rw [hF j hjF]; exact bot_le
    * -- the box is product-open: a finite intersection of cylinders over Scott-open factors
      refine @isOpen_biInter_finset (forall  j, E j) ?
        (@Pi.topologicalSpace ? E (fun _ => scottTopologicalSpace)) F
        (fun i => (fun w : forall  j, E j => w i) ^-1' V i) (fun i _ => ?_)
      have hVi : @IsOpen (E i) scottTopologicalSpace (V i) := isOpen_iff_scottOpen.mpr (hVopen
    i)
      have hind : @IsOpen _
          (TopologicalSpace.induced (fun w : forall  j, E j => w i) scottTopologicalSpace)
          ((fun w : forall  j, E j => w i) ^-1' V i) :=
        (isOpen_induced_iff (t := scottTopologicalSpace)).mpr <V i, hVi, rfl>
      exact iInf_le
        (fun i => TopologicalSpace.induced (fun w : forall  j, E j => w i)
    scottTopologicalSpace) i _ hind
    * -- the box contains `z`
      rw [Set.mem_iInter_2]
      exact fun i _ => hzV i

/-- **Scott 1972, Proposition 2.9 (full statement).** The product of a family of continuous
    lattices
is again a continuous lattice (`proposition_2_9_a`) whose Scott topology agrees with the
    product
topology (`proposition_2_9_b`). -/
theorem proposition_2_9 {? : Type*} (E : ?  ->  Type*) [forall  i, CompleteLattice (E i)]
    (hE : forall  i, IsContinuousLattice (E i)) :
    IsContinuousLattice (forall  i, E i)  /\
      (scottTopologicalSpace : TopologicalSpace (forall  i, E i)) =
        @Pi.topologicalSpace ? E (fun _ => scottTopologicalSpace) :=
  <proposition_2_9_a E hE, proposition_2_9_b E hE>

/-! ### Proposition 2.11: continuous lattices are injective

The substance of Scott's Theorem 2.12. We give the explicit extension operator
`g(y) =  sqsup _{V ? y open}  sqcap  f''(e^-1V)` and prove (a) it extends `f` along an
    embedding `e`
(using continuity of `D` to interpolate from below) and (b) it is Scott-continuous (using the
` Up a` basis of the Scott topology). -/

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [CompleteLattice E
    ]

/-- Scott's canonical extension of `f : X  ->  E` along `e : X  ->  Y` (Scott 1972, proof of
    2.11):
`y  |->   sqsup  {  sqcap  f''(e^-1V) : V an open neighbourhood of y }`. No topology on `E` is
    needed to *state*
the operator -- it is purely order-theoretic. -/
```

```
def scottExtend (e : X  ->  Y) (f : X  ->  E) (y : Y) : E :=
  sSup {d | exists  V : Set Y, IsOpen V  /\  y  in  V  /\  d = sInf (f '' (e ^-1' V))}

theorem scottExtend_aux_nonempty (e : X  ->  Y) (f : X  ->  E) (y : Y) :
    {d | exists  V : Set Y, IsOpen V  /\  y  in  V  /\  d = sInf (f '' (e ^-1' V))}.Nonempty :=
  <_, Set.univ, isOpen_univ, Set.mem_univ y, rfl>

/-- The defining family of `scottExtend` is directed: open neighbourhoods are closed under
intersection, and ` sqcap  f''(e^-1 *)` is monotone in the neighbourhood (smaller set, larger
    inf). -/
theorem scottExtend_aux_directed (e : X  ->  Y) (f : X  ->  E) (y : Y) :
    DirectedOn (*  <=  *) {d | exists  V : Set Y, IsOpen V  /\  y  in  V  /\  d = sInf (f '' (e
     ^-1' V))} := by
  rintro _ <V_1, hV_1o, hyV_1, rfl> _ <V_2, hV_2o, hyV_2, rfl>
  refine <sInf (f '' (e ^-1' (V_1  I  V_2))), <V_1  I  V_2, hV_1o.inter hV_2o, <hyV_1, hyV_2>,
    rfl>, ?_, ?_>
  * exact sInf_le_sInf (Set.image_mono (Set.preimage_mono Set.inter_subset_left))
  * exact sInf_le_sInf (Set.image_mono (Set.preimage_mono Set.inter_subset_right))

/-- The extension agrees with `f` on the (embedded) subspace. The ` <= ` direction is immediate
(`f x_0` is one of the values being met); the ` >= ` direction uses continuity of `E`: for each
`a  <<  f x_0` the Scott-open ` Up a` pulls back along the continuous `f` and, by the embedding
    , to an
open `V  subseteq  Y` on whose `e`-preimage `f  >=  a`, so `a  <=  sqcap  f''(e^-1V)  <=  g(e
    x_0)`. -/
theorem scottExtend_eq_of_continuous (hE : IsContinuousLattice E) (e : X  ->  Y)
    (he : IsEmbedding e) (f : X  ->  E) (hf : @Continuous X E _ scottTopologicalSpace f) (x_0 :
     X) :
    scottExtend e f (e x_0) = f x_0 := by
  apply le_antisymm
  * refine sSup_le ?_
    rintro d <V, hVo, hex_0V, rfl>
    exact sInf_le <x_0, hex_0V, rfl>
  * rw [ <-  hE.sSup_wayBelow (f x_0)]
    refine sSup_le fun a ha => ?_
    have hWopen : @IsOpen E scottTopologicalSpace {z : E | a  <<  z} :=
      isOpen_iff_scottOpen.mpr (scottOpen_wayBelow a)
    have hpre : IsOpen (f ^-1' {z : E | a  <<  z}) := continuous_def.mp hf _ hWopen
    rw [he.isInducing.isOpen_iff] at hpre
    obtain <V, hVo, hVeq> := hpre
    have hx_0V : x_0  in  e ^-1' V := by rw [hVeq]; exact ha
    refine le_trans ?_ (le_sSup <V, hVo, hx_0V, rfl>)
    refine le_sInf ?_
    rintro w <x, hxV, rfl>
    have hxW : x  in  f ^-1' {z : E | a  <<  z} := by rw [ <-  hVeq]; exact hxV
    exact (hxW : a  <<  f x).le

/-- The extension is Scott-continuous. For a Scott-open `U` and a point `y_0` with `g y_0  in
    U`, the
basis lemma gives `a  <<  g y_0` with ` Up a  subseteq  U`; since `g y_0` is a directed
    supremum, `a  <<  sqcap  f''(e^-1V)`
for some open `V ? y_0`, and that value is ` <=  g y'` for every `y'  in  V`, so `V  subseteq
    g^-1U`. -/
theorem scottExtend_continuous (hE : IsContinuousLattice E) (e : X  ->  Y) (f : X  ->  E) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f) := by
  letI : TopologicalSpace E := scottTopologicalSpace
  rw [continuous_def]
  intro U hU
  rw [isOpen_iff_scottOpen] at hU
  rw [isOpen_iff_forall_mem_open]
  intro y_0 hy_0
  have hgy_0U : scottExtend e f y_0  in  U := hy_0
  obtain <a, haz, hasub> := exists_wayBelow_subset hE hU hgy_0U
```

```
  obtain <d, hd, had> := (wayBelow_sSup_iff (scottExtend_aux_nonempty e f y_0)
    (scottExtend_aux_directed e f y_0)).1 haz
  obtain <V, hVo, hy_0V, rfl> := hd
  refine <V, ?_, hVo, hy_0V>
  intro y' hy'V
  show scottExtend e f y'  in  U
  refine hasub ?_
  show a  <<  scottExtend e f y'
  exact had.trans_le (le_sSup <V, hVo, hy'V, rfl>)

/-- For a continuous `f' : Y  ->  E` into a continuous lattice `E` (with its Scott topology),
    the
value `f' y` is reconstructed as the supremum over open neighbourhoods `U ? y` of the meets
` sqcap  f''(U)`. This is the order-theoretic content behind the maximality clause of
    Proposition 3.8:
the ` <= ` direction interpolates from below using `f' y =  sqsup  {a | a  <<  f' y}` and the
    openness of each
`f'^-1( Up a)`; the ` >= ` direction is immediate since `f' y  in  f''(U)` whenever `y  in  U`.
     -/
theorem continuous_eq_sSup_openInfs (hE : IsContinuousLattice E) {f' : Y  ->  E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (y : Y) :
    f' y = sSup {d | exists  U : Set Y, IsOpen U  /\  y  in  U  /\  d = sInf (f' '' U)} := by
  apply le_antisymm
  * rw [ <-  hE.sSup_wayBelow (f' y)]
    refine sSup_le fun a ha => ?_
    have hWopen : @IsOpen E scottTopologicalSpace {z : E | a  <<  z} :=
      isOpen_iff_scottOpen.mpr (scottOpen_wayBelow a)
    have hpre : IsOpen (f' ^-1' {z : E | a  <<  z}) := continuous_def.mp hf' _ hWopen
    have hyU : y  in  f' ^-1' {z : E | a  <<  z} := ha
    refine le_trans ?_ (le_sSup <f' ^-1' {z : E | a  <<  z}, hpre, hyU, rfl>)
    refine le_sInf ?_
    rintro w <z, hzU, rfl>
    exact (hzU : a  <<  f' z).le
  * refine sSup_le ?_
    rintro d <U, _hUo, hyU, rfl>
    exact sInf_le <y, hyU, rfl>

/-- **Maximality clause of Scott 1972, Proposition 3.8.** Any continuous solution `f'` of
`f'  comp  e = f` lies below `scottExtend e f`. Following Scott: expand `f'` via
`continuous_eq_sSup_openInfs`, restrict each meet from `U` to the embedded subspace `e(X)  I  U
    `
(only enlarging the meet), and recognize the result as a defining term of `scottExtend`. -/
theorem scottExtend_maximal (hE : IsContinuousLattice E) (e : X  ->  Y) {f : X  ->  E} {f' : Y
     ->  E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (h_ext : forall  x, f' (e x) = f x) (y :
     Y) :
    f' y  <=  scottExtend e f y := by
  rw [continuous_eq_sSup_openInfs hE hf' y]
  refine sSup_le ?_
  rintro d <U, hUo, hyU, rfl>
  refine le_trans ?_ (le_sSup <U, hUo, hyU, rfl>)
  refine le_sInf ?_
  rintro w <x, hxU, rfl>
  rw [ <-  h_ext x]
  exact sInf_le <e x, hxU, rfl>

/-- **Scott 1972, remark following 3.8.** `scottExtend e g` is also the maximal *sub*-solution:
     any
continuous `f'` with `f'  comp  e  sqsub  g` satisfies `f'  sqsub  scottExtend e g`. Same proof
     as
`scottExtend_maximal`, replacing the final equality `f' (e x) = f x` by the inequality
`f' (e x)  <=  g x`. -/
theorem scottExtend_maximal_le (hE : IsContinuousLattice E) (e : X  ->  Y) {g : X  ->  E} {f' :
```

```
     Y  ->  E}
    (hf' : @Continuous Y E _ scottTopologicalSpace f') (h_le : forall  x, f' (e x)  <=  g x) (y
     : Y) :
    f' y  <=  scottExtend e g y := by
  rw [continuous_eq_sSup_openInfs hE hf' y]
  refine sSup_le ?_
  rintro d <U, hUo, hyU, rfl>
  refine le_trans ?_ (le_sSup <U, hUo, hyU, rfl>)
  refine le_sInf ?_
  rintro w <x, hxU, rfl>
  exact le_trans (sInf_le <e x, hxU, rfl>) (h_le x)

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X  ->  Y` a
    subspace
embedding, then for each continuous `f : X  ->  E` the explicit formula `scottExtend e f` is
    the
*maximal extension* of `f` to `[Y  ->  E]`: it is Scott-continuous (`scottExtend_continuous`),
    it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every continuous
solution of `f'  comp  e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : IsContinuousLattice E) (e : X  ->  Y) (he : IsEmbedding e)
    (f : X  ->  E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
       /\  (forall  x, scottExtend e f (e x) = f x)
       /\  (forall  f' : Y  ->  E, @Continuous Y E _ scottTopologicalSpace f'  ->  (forall  x,
    f' (e x) = f x)  ->
          forall  y, f' y  <=  scottExtend e f y) :=
  <scottExtend_continuous hE e f,
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y>

end InjectiveExtension

/-- **Scott 1972, Proposition 2.11.** Every continuous lattice is an injective space under its
induced (Scott) topology. The witness is `scottExtend`, which extends any continuous `f` along
any embedding `e` (`scottExtend_eq_of_continuous`) and is itself continuous
(`scottExtend_continuous`). -/
theorem proposition_2_11 {E : Type*} [CompleteLattice E] (hE : IsContinuousLattice E) :
    @IsInjectiveSpace E scottTopologicalSpace := by
  letI : TopologicalSpace E := scottTopologicalSpace
```

Lean 4 source (lines 401–508)

```
  intro X Y _ _ e he f
  exact <<scottExtend e f, scottExtend_continuous hE e f>,
    fun x => scottExtend_eq_of_continuous hE e he f f.continuous x>

/-- The Sierpinski space `Prop` (Scott's ` O `) is a continuous lattice: it is a finite
    complete
lattice, so Proposition 2.8 applies. -/
theorem isContinuousLattice_prop : IsContinuousLattice Prop :=
  proposition_2_8

/-- The Scott topology on Scott's two-point space ` O  = Prop` is exactly the Sierpinski
    topology
(`generateFrom {{True}}`). The Scott-open sets of `Prop` are the upper sets `{}`, `{True}`, `
    univ`,
which are precisely the Sierpinski-open sets. This is the topological identification underlying
Theorem 2.12: the building block ` O ` carries its Scott topology. -/
theorem scottTopology_prop :
    (scottTopologicalSpace : TopologicalSpace Prop) = sierpinskiSpace := by
  apply le_antisymm
  * -- `scott  <=  sierpinski`: the single Sierpinski sub-basic open `{True}` is Scott-open.
    show (scottTopologicalSpace : TopologicalSpace Prop)  <=  TopologicalSpace.generateFrom {{
```

```
    True}}
    apply le_generateFrom
    intro s hs
    rw [Set.mem_singleton_iff] at hs
    subst hs
    rw [isOpen_iff_scottOpen]
    refine <?_, ?_>
    * intro a b hab ha
      rw [Set.mem_singleton_iff] at ha  |-
      exact le_antisymm le_top (ha  |>  hab)
    * intro S _ _ hmem
      rw [Set.mem_singleton_iff] at hmem
      have hex : exists  p  in  S, p := by rw [ <-  sSup_Prop_eq, hmem]; trivial
      obtain <p, hpS, hp> := hex
      exact <p, hpS, by rw [Set.mem_singleton_iff]; exact eq_true hp>
  * -- `sierpinski  <=  scott`: every Scott-open upper set of `Prop` is `{}`, `{True}`, or `
    univ`.
    rw [TopologicalSpace.le_def]
    intro U hU
    rw [isOpen_iff_scottOpen] at hU
    by_cases hT : True  in  U
    * by_cases hF : False  in  U
      * have hUuniv : U = Set.univ := by
          ext p
          simp only [Set.mem_univ, iff_true]
          by_cases hp : p
          * rwa [eq_true hp]
          * rwa [eq_false hp]
        rw [hUuniv]; exact isOpen_univ
      * have hUtrue : U = {True} := by
          ext p
          rw [Set.mem_singleton_iff]
          constructor
          * intro hpU
            by_cases hp : p
            * exact eq_true hp
            * exact absurd (eq_false hp  |>  hpU) hF
          * intro hp; rw [hp]; exact hT
        rw [hUtrue]; exact isOpen_singleton_true
    * have hUempty : U = {} := by
        ext p
        simp only [Set.mem_empty_iff_false, iff_false]
        intro hpU
        exact hT (hU.1 le_top hpU)
      rw [hUempty]; exact isOpen_empty

/-- A power of Scott's two-point space ` O  = Prop` is a continuous lattice: `Prop` is a
    continuous
lattice (`isContinuousLattice_prop`) and products of continuous lattices are continuous
(Proposition 2.9(a)). This is the order-theoretic content of "a Cartesian power of ` O ` is a
continuous lattice", the construction Theorem 2.12 retracts onto. -/
theorem sierpinskiPower_isContinuousLattice (? : Type*) : IsContinuousLattice (?  ->  Prop) :=
  proposition_2_9_a (fun _ => Prop) (fun _ => isContinuousLattice_prop)

/-- The Scott topology on a power `?  ->   O ` of the Sierpinski space coincides with the
    product
(= Sierpinski power) topology. Combine Proposition 2.9(b) (the Scott topology of a product is
    the
product of the factors' Scott topologies) with `scottTopology_prop` (each factor's Scott
    topology
is the Sierpinski topology). -/
theorem scottTopology_sierpinskiPower (? : Type*) :
    (scottTopologicalSpace : TopologicalSpace (?  ->  Prop)) =
```

```
      (inferInstance : TopologicalSpace (?  ->  Prop)) := by
  rw [proposition_2_9_b (fun _ => Prop) (fun _ => isContinuousLattice_prop)]
  show (@Pi.topologicalSpace ? (fun _ => Prop) (fun _ => scottTopologicalSpace)) =
    @Pi.topologicalSpace ? (fun _ => Prop) (fun _ => sierpinskiSpace)
  congr 1
  funext _
  exact scottTopology_prop

/-- **Scott 1972, Theorem 2.12 (forward direction).** Every continuous lattice is an injective
space under its Scott topology. This is the substantial half of the equivalence "injective
spaces = continuous lattices", and is exactly Proposition 2.11. -/
theorem theorem_2_12_forward {E : Type*} [CompleteLattice E] (hE : IsContinuousLattice E) :
    @IsInjectiveSpace E scottTopologicalSpace :=
  proposition_2_11 hE

/-- **Scott 1972, Theorem 2.12 (backward, Sierpinski base case).** `Prop` is a continuous
lattice (`isContinuousLattice_prop`), so its injectivity (Proposition 1.2) is an instance of
    the
equivalence. -/
theorem theorem_2_12_sierpinski_backward (_h : IsContinuousLattice Prop) : IsInjectiveSpace
    Prop :=
  proposition_1_2

/-- **Scott 1972, Theorem 2.12 (injectivity half, unconditional).** `Prop` is injective
(Sierpinski); the continuous-lattice half is now `isContinuousLattice_prop`. -/
theorem theorem_2_12_injective_half : IsInjectiveSpace Prop :=
  proposition_1_2

/-- The Sierpinski space ` O  = Prop` realizes the smallest case of Theorem 2.12: it is both
injective (1.2) and a continuous lattice (2.8). -/
theorem sierpinski_isInjective_and_isContinuousLattice :
    IsInjectiveSpace Prop  /\  IsContinuousLattice Prop :=
  <proposition_1_2, isContinuousLattice_prop>

end Scott1972.ContinuousLattice
```

A.8 Scott1972/ContinuousLattice/FunctionSpaces.lean

1626 lines.

Lean 4 source (lines 1–400)

```
import Scott1972.ContinuousLattice.ScottMaps
import Mathlib.Data.Finset.Lattice.Fold
import Mathlib.Topology.ContinuousMap.Basic
import Mathlib.Topology.Order
import Mathlib.Order.CompleteLattice.Basic
import Mathlib.Order.FixedPoints

/-!
# Function spaces on continuous lattices (Scott 1972, S3)

Scott's S3 studies spaces of continuous maps `[X  ->  Y]` with the topology of pointwise
convergence (subbasic sets `{f | f x  in  U}`), the pointwise specialization order (3.2),
and the fact that `[D  ->  D']` is a continuous lattice when `D  ->  D']` is (3.3).

## March 1972 correction (Milner)

Scott's remark before Proposition 2.5 was mistaken: a `T_0`-topology on a complete lattice
need not have every open set Scott-open. The two extreme `T_0`-topologies inducing the same
order are generated by `{x | x  sqsubneq  y}` (lower) and `{x | y  sqsub  x}` (principal up-
    sets).
The corrected proofs of 2.9, 2.10, and 3.3 assume the given topology is *coarser* than
```

```
the Scott (lattice) topology (`scottTopologicalSpace  <=  tau` in mathlib). We revisit that
hypothesis when formalizing those results.

In the March 1972 correction (p. 135), Scott writes ` sqsup S'` (prime on the index, not on the
join): for `S  subseteq  D` a subspace of ambient `D'`, ` sqsup S'` is the supremum of `S`
    computed in `D'`,
while ` sqsup S` is the supremum in the subspace `D`; the retraction identity is `j( sqsup S')
    =  sqsup S`.
-/

namespace Scott1972.ContinuousLattice

open Set Topology TopologicalSpace

universe u v

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [CompleteLattice
    D_1']

/-! ### Definition 3.1 -/

/-- Subbasic sets for Scott's function-space topology (pointwise convergence). -/
def scottFunctionEvalSubbasis (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] :
    Set (Set (C(X, Y))) :=
  { W | exists  (x : X) (U : Set Y), IsOpen U  /\  W = {f : C(X, Y) | f x  in  U} }

/-- **Scott 1972, Definition 3.1.** The function space `[X  ->  Y]` carries the topology
generated by `{f : f x  in  U | x  in  X, U open in Y}`. -/
@[reducible] noncomputable def scottFunctionTopology (X Y : Type*) [TopologicalSpace X]
    [TopologicalSpace Y] : TopologicalSpace (C(X, Y)) :=
  generateFrom (scottFunctionEvalSubbasis X Y)

/-- Scott's notation `[X  ->  Y]`: continuous maps with the pointwise (product) topology. -/
abbrev ScottFunctionSpace (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] :=
  C(X, Y)

/-! ### Proposition 3.2 -/

theorem scottFunctionSubbasis_isOpen {x : X} {U : Set Y} (hU : IsOpen U) :
    @IsOpen (C(X, Y)) (scottFunctionTopology X Y) {f : C(X, Y) | f x  in  U} :=
  isOpen_generateFrom_of_mem (s := {f : C(X, Y) | f x  in  U}) <x, U, hU, rfl>

def specializationLe_generateOpen (f g : C(X, Y)) (hfg : forall  x, SpecializationLe (f x) (g x
    ))
    {V : Set (C(X, Y))} (hV : GenerateOpen (scottFunctionEvalSubbasis X Y) V) (hf : f  in  V) :
    g  in  V :=
  match hV with
  | .basic W hW => by
    obtain <x, U, hU, rfl> := hW
    exact hfg x U hU hf
  | .univ => hf
  | .inter s t hs ht => by
    rcases hf with <hfs, hft>
    exact Set.mem_inter (specializationLe_generateOpen f g hfg hs hfs)
      (specializationLe_generateOpen f g hfg ht hft)
  | .sUnion S hS => by
    obtain <W, hWS, hfW> := Set.mem_sUnion.mp hf
    exact Set.mem_sUnion.mpr <W, hWS, specializationLe_generateOpen f g hfg (hS W hWS) hfW>

theorem scottFunctionSpecializationLe {f g : C(X, Y)} :
```

```
    letI := scottFunctionTopology X Y
    SpecializationLe f g  <->  forall  x, SpecializationLe (f x) (g x) := by
  letI : TopologicalSpace (C(X, Y)) := scottFunctionTopology X Y
  constructor
  * intro hfg x U hU hfU
    exact hfg {h | h x  in  U} (scottFunctionSubbasis_isOpen (X := X) (Y := Y) hU) hfU
  * intro hfg U hU hfU
    exact specializationLe_generateOpen f g hfg hU hfU

/-- **Scott 1972, Proposition 3.2.** The specialization order on `[X  ->  Y]` is pointwise. -/
theorem proposition_3_2 {f g : C(X, Y)} :
    letI := scottFunctionTopology X Y
    (SpecializationLe f g  <->  forall  x, SpecializationLe (f x) (g x)) :=
  scottFunctionSpecializationLe

/-! ### S3 on continuous lattices: pointwise lattice structure -/

/-- Continuous maps `D  ->  D'` with Scott topologies on domain and codomain. -/
abbrev ScottC (D D' : Type*) [CompleteLattice D] [CompleteLattice D'] :=
  @ContinuousMap D D' scottTopologicalSpace scottTopologicalSpace

/-- Continuous maps between complete lattices with Scott's induced topologies. -/
def ScottMap (D D' : Type*) [CompleteLattice D] [CompleteLattice D'] : Type _ :=
  { f : D  ->  D' // @Continuous D D' scottTopologicalSpace scottTopologicalSpace f }

namespace ScottMap

instance : CoeFun (ScottMap D D') (fun _ => D  ->  D') where
  coe f := f.1

@[ext]
theorem ext {f g : ScottMap D D'} (h : forall  x, f x = g x) : f = g :=
  Subtype.ext (funext h)

theorem continuous (f : ScottMap D D') :
    @Continuous D D' scottTopologicalSpace scottTopologicalSpace f :=
  f.2

theorem preservesDirectedSup_coe (f : ScottMap D D') (S : Set D) (hS : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) :
    (f : D  ->  D') (sSup S) = sSup (Set.image (f : D  ->  D') S) := by
  have h := (proposition_2_5 (Subtype.val f)).mp f.property
  exact h hS hSdir

theorem monotone (f : ScottMap D D') : Monotone (f : D  ->  D') :=
  preservesDirectedSup_monotone ((proposition_2_5 (Subtype.val f)).mp f.property)

theorem preservesDirectedSup_comp {f : D'  ->  D''} {g : D  ->  D'} (hf : PreservesDirectedSup
    f)
    (hg : PreservesDirectedSup g) : PreservesDirectedSup (f  comp  g) := by
  intro S hS hSdir
  have hmono : Monotone g := preservesDirectedSup_monotone hg
  have hdir' : DirectedOn (*  <=  *) (g '' S) := by
    intro p hp q hq
    obtain <a, ha, rfl> := hp
    obtain <b, hb, rfl> := hq
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <g c, Set.mem_image_of_mem g hc, hmono hac, hmono hbc>
  have hne : (g '' S).Nonempty := Set.image_nonempty.2 hS
  rw [Function.comp_apply, hg hS hSdir, hf hne hdir']
  congr 1
  exact (Set.image_comp f g S).symm
```

```
def comp (f : ScottMap D' D'') (g : ScottMap D D') : ScottMap D D'' :=
  <f  comp  g, continuous_of_preservesDirectedSup (preservesDirectedSup_comp
    ((proposition_2_5 (Subtype.val f)).mp f.property)
    ((proposition_2_5 (Subtype.val g)).mp g.property))>

def const (c : D') : ScottMap D D' :=
  <fun _ => c, continuous_of_preservesDirectedSup fun S hS _ => by
    obtain <x, hx> := hS
    apply le_antisymm
    * exact le_sSup <x, hx, rfl>
    * apply sSup_le
      intro b hb
      obtain <y, hy, rfl> := hb
      exact le_rfl>

theorem pointwise_sup_preservesDirectedSup (f g : ScottMap D D') :
    PreservesDirectedSup (fun x => f x  sqsup  g x) := by
  intro S hS hSdir
  show f (sSup S)  sqsup  g (sSup S) = sSup (Set.image (fun x => f x  sqsup  g x) S)
  rw [f.preservesDirectedSup_coe S hS hSdir, g.preservesDirectedSup_coe S hS hSdir]
  apply le_antisymm
  * apply sup_le
    * apply sSup_le
      intro b hb
      obtain <s, hsS, rfl> := hb
      exact le_trans le_sup_left (le_sSup (Set.mem_image_of_mem (fun z => f z  sqsup  g z) hsS)
    )
    * apply sSup_le
      intro b hb
      obtain <s, hsS, rfl> := hb
      exact le_trans le_sup_right (le_sSup (Set.mem_image_of_mem (fun z => f z  sqsup  g z) hsS
    ))
  * apply sSup_le
    intro b hb
    obtain <s, hsS, rfl> := hb
    exact sup_le (le_trans (le_sSup (Set.mem_image_of_mem f hsS)) le_sup_left)
      (le_trans (le_sSup (Set.mem_image_of_mem g hsS)) le_sup_right)

noncomputable def bot : ScottMap D D' :=
  const False

def le (f g : ScottMap D D') : Prop :=
  forall  x, f x  <=  g x

noncomputable def sup (f g : ScottMap D D') : ScottMap D D' :=
  <fun x => f x  sqsup  g x, continuous_of_preservesDirectedSup (
    pointwise_sup_preservesDirectedSup f g)>

theorem pointwise_sSup_preservesDirectedSup (F : Set (ScottMap D D')) :
    PreservesDirectedSup (fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D  ->  D') x)
     F)) := by
  intro S hS hSdir
  set H := fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D  ->  D') x) F)
  show H (sSup S) = sSup (Set.image H S)
  apply le_antisymm
  * apply sSup_le
    intro b hb
    obtain <f, hfF, rfl> := hb
    change f (sSup S)  <=  sSup (Set.image H S)
    rw [f.preservesDirectedSup_coe S hS hSdir]
    apply sSup_le
    intro c hc
    obtain <s, hsS, rfl> := hc
```

```
    exact le_trans (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D  ->  D') s)
    hfF))
      (le_sSup (Set.mem_image_of_mem H hsS))
  * apply sSup_le
    intro b hb
    obtain <s, hsS, rfl> := hb
    apply sSup_le
    intro c hc
    obtain <f, hfF, rfl> := hc
    exact le_trans (ScottMap.monotone f (le_sSup hsS))
      (le_sSup (Set.mem_image_of_mem (fun g : ScottMap D D' => (g : D  ->  D') (sSup S)) hfF))

noncomputable def sSupMaps (F : Set (ScottMap D D')) : ScottMap D D' :=
  <fun x => sSup (Set.image (fun f : ScottMap D D' => (f : D  ->  D') x) F),
    continuous_of_preservesDirectedSup (pointwise_sSup_preservesDirectedSup F)>

theorem sSupMaps_apply (F : Set (ScottMap D D')) (x : D) :
    (sSupMaps F : D  ->  D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D  ->  D') x) F) :=
  rfl

/-! ### The complete lattice `[D  ->  D']` (Theorem 3.3, order content)

The pointwise order makes `ScottMap D D'` a partial order; `sSupMaps` gives suprema (pointwise,
since directed *and* arbitrary suprema of Scott maps are computed pointwise -- Theorem 3.3). By
`completeLatticeOfSup` this is a complete lattice. Note infima are *not* pointwise (Scott's
warning); `completeLatticeOfSup` derives them as `sSup` of lower bounds. -/

instance instPartialOrder : PartialOrder (ScottMap D D') where
  le := ScottMap.le
  le_refl _ _ := le_refl _
  le_trans _ _ _ hfg hgh x := le_trans (hfg x) (hgh x)
  le_antisymm _ _ hfg hgf := ScottMap.ext fun x => le_antisymm (hfg x) (hgf x)

theorem le_def {f g : ScottMap D D'} : f  <=  g  <->  forall  x, (f : D  ->  D') x  <=  g x :=
    Iff.rfl

noncomputable instance instSupSet : SupSet (ScottMap D D') := <sSupMaps>

theorem sSup_apply (F : Set (ScottMap D D')) (x : D) :
    ((sSup F : ScottMap D D') : D  ->  D') x =
      sSup (Set.image (fun f : ScottMap D D' => (f : D  ->  D') x) F) :=
  rfl

theorem isLUB_sSup (F : Set (ScottMap D D')) : IsLUB F (sSup F) := by
  constructor
  * intro f hf
    rw [le_def]
    intro x
    rw [sSup_apply]
    exact le_sSup (Set.mem_image_of_mem _ hf)
  * intro g hg
    rw [le_def]
    intro x
    rw [sSup_apply]
    refine sSup_le ?_
    rintro _ <f, hfF, rfl>
    exact (hg hfF) x

noncomputable instance instCompleteLattice : CompleteLattice (ScottMap D D') :=
  completeLatticeOfSup (ScottMap D D') isLUB_sSup

/-- The identity Scott map. -/
```

```
def idMap : ScottMap D D :=
  <fun x => x, continuous_of_preservesDirectedSup fun S _ _ => by
    show sSup S = sSup (Set.image (fun x => x) S)
    rw [Set.image_id']>

@[simp] theorem idMap_apply (x : D) : (idMap : ScottMap D D) x = x := rfl

@[simp] theorem comp_apply (f : ScottMap D' D'') (g : ScottMap D D') (x : D) :
    (f.comp g : D  ->  D'') x = f (g x) := rfl

/-- The (completeLatticeOfSup-derived) binary join of Scott maps is computed pointwise. -/
theorem sup_apply (f g : ScottMap D D') (x : D) :
    ((f  sqsup  g : ScottMap D D') : D  ->  D') x = f x  sqsup  g x := by
  have h : (f  sqsup  g : ScottMap D D') = sSup ({f, g} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_pair]
  exact sSup_pair

/-- The bottom Scott map is the constant `False`. -/
theorem bot_apply (x : D) : ((False : ScottMap D D') : D  ->  D') x = False := by
  have h : (False : ScottMap D D') = sSup ({} : Set (ScottMap D D')) := rfl
  rw [h, sSup_apply, Set.image_empty, sSup_empty]

@[simp] theorem const_apply (c : D') (x : D) : (ScottMap.const c : D  ->  D') x = c := rfl

end ScottMap

/-! ### Theorem 3.3 (core) -/

/-- **Scott 1972, Theorem 3.3 (directed sup).** Pointwise suprema of Scott-continuous maps are
Scott-continuous; this is the heart of Scott's proof that `[D  ->  D']` is a complete lattice.
    -/
noncomputable def theorem_3_3_sSup (F : Set (ScottMap D D')) : ScottMap D D' :=
  ScottMap.sSupMaps F

/-- **Scott 1972, Theorem 3.3 (binary join).** Pointwise join of Scott-continuous maps is
Scott-continuous. -/
noncomputable def theorem_3_3_sup (f g : ScottMap D D') : ScottMap D D' :=
  ScottMap.sup f g

/-! ### Theorem 3.3 (`[D  ->  D']` is a continuous lattice)

Scott's *step functions* `e[e, e']` are the building blocks: `e[e,e'](x) = e'` if `e  <<  x`,
    else
`False`. We encode the conditional value as ` bigcup  _ : e  <<  x, e'` (which is `e'` when `e
     <<  x` and `False`
otherwise), avoiding any decidability assumption. Each step function is Scott-continuous (the
way-above set `{x | e  <<  x}` is Scott-open), is way below `f` whenever `e'  <<  f e`, and `f`
     is the
supremum of the step functions below it -- whence `[D  ->  D']` is a continuous lattice. -/

/-- The (value of the) step function `e[e, e']` at `x`: `e'` if `e  <<  x`, else `False`. -/
noncomputable def stepFun (e : D) (e' : D') (x : D) : D' :=  bigcup  _ : e  <<  x, e'

theorem stepFun_of_wayBelow {e : D} {e' : D'} {x : D} (h : e  <<  x) : stepFun e e' x = e' :=
  iSup_pos h

theorem stepFun_of_not_wayBelow {e : D} {e' : D'} {x : D} (h : not  e  <<  x) :
    stepFun e e' x = False :=
  iSup_neg h

/-- Step functions are Scott-continuous: `{x | e  <<  x}` is Scott-open, so the conditional
    commutes
with directed suprema. -/
```

```
theorem stepFun_preservesDirectedSup (e : D) (e' : D') :
    PreservesDirectedSup (stepFun e e') := by
  intro S hS hSdir
  show stepFun e e' (sSup S) = sSup (Set.image (stepFun e e') S)
  by_cases h : e  <<  sSup S
  * rw [stepFun_of_wayBelow h]
    obtain <s, hsS, hes> := (scottOpen_wayBelow e).2 hS hSdir h
    apply le_antisymm
    * refine le_sSup_of_le (Set.mem_image_of_mem _ hsS) ?_
      rw [stepFun_of_wayBelow hes]
    * refine sSup_le ?_
      rintro _ <x, _, rfl>
      exact iSup_le fun _ => le_rfl
  * rw [stepFun_of_not_wayBelow h]
    refine (sSup_eq_bot.2 ?_).symm
    rintro _ <x, hxS, rfl>
    refine stepFun_of_not_wayBelow (fun hex => ?_)
    exact h (hex.trans_le (le_sSup hxS))

/-- The step function `e[e, e']` as a Scott map. -/
noncomputable def stepMap (e : D) (e' : D') : ScottMap D D' :=
  <stepFun e e', continuous_of_preservesDirectedSup (stepFun_preservesDirectedSup e e')>

theorem stepMap_apply_of_wayBelow {e : D} {e' : D'} {x : D} (h : e  <<  x) :
    (stepMap e e' : D  ->  D') x = e' :=
  stepFun_of_wayBelow h

/-- If `e'  <<  g e` then the step function `e[e, e']` lies below `g` (a pointwise argument).
    -/
theorem stepMap_le_of_wayBelow {e : D} {e' : D'} {g : ScottMap D D'}
    (h : e'  <<  (g : D  ->  D') e) : stepMap e e'  <=  g := by
  rw [ScottMap.le_def]
  intro x
  by_cases hex : e  <<  x
  * rw [stepMap_apply_of_wayBelow hex]
    exact h.le.trans (g.monotone hex.le)
  * show stepFun e e' x  <=  (g : D  ->  D') x
    rw [stepFun_of_not_wayBelow hex]
    exact bot_le

/-- **The step function is way below `f`.** If `e'  <<  f e`, then `e[e, e']  <<  f` in `[D  ->
      D']`,
witnessed by the Scott-open set `{g | e'  <<  g e}` (open because suprema in `[D  ->  D']` are
    pointwise
and `{*  <<  *}` is inaccessible by directed suprema). -/
theorem stepMap_wayBelow {e : D} {e' : D'} {f : ScottMap D D'}
    (he' : e'  <<  (f : D  ->  D') e) : stepMap e e'  <<  f := by
  refine <{g : ScottMap D D' | e'  <<  (g : D  ->  D') e}, <?_, ?_>, he', ?_>
  * intro g g' hgg' hg
    exact hg.trans_le (hgg' e)
  * intro F hFne hFdir hmem
    simp only [Set.mem_setOf_eq, ScottMap.sSup_apply] at hmem
    have hne : (Set.image (fun g : ScottMap D D' => (g : D  ->  D') e) F).Nonempty := hFne.
    image _
    have hdir : DirectedOn (*  <=  *) (Set.image (fun g : ScottMap D D' => (g : D  ->  D') e) F
    ) := by
      rintro _ <a, ha, rfl> _ <b, hb, rfl>
      obtain <c, hc, hac, hbc> := hFdir a ha b hb
      exact <(c : D  ->  D') e, Set.mem_image_of_mem _ hc, hac e, hbc e>
    obtain <_, <g, hgF, rfl>, hg> := (wayBelow_sSup_iff hne hdir).1 hmem
    exact <g, hgF, hg>
  * intro g hg
    exact Set.mem_Ici.2 (stepMap_le_of_wayBelow hg)
```

```
/-- **Pointwise reconstruction.** For a Scott map `f`, the supremum of the values of the step
functions below `f` recovers `f x`: `f x =  sqsup  {e' | exists  e  <<  x, e'  <<  f e}`. Uses
    continuity of `D`
(`x =  sqsup {e | e  <<  x}`), Scott-continuity of `f`, and continuity of `D'`. -/
theorem stepMap_pointwise_sSup (hD : IsContinuousLattice D) (hD' : IsContinuousLattice D')
    (f : ScottMap D D') (x : D) :
    sSup {e' : D' | exists  e, e  <<  x  /\  e'  <<  (f : D  ->  D') e} = (f : D  ->  D') x :=
    by
  apply le_antisymm
  * refine sSup_le ?_
    rintro e' <e, hex, he'>
    exact he'.le.trans (f.monotone hex.le)
  * rw [ <-  hD'.sSup_wayBelow ((f : D  ->  D') x)]
    refine sSup_le ?_
    intro w hw
    have hfx : (f : D  ->  D') x = sSup (Set.image (f : D  ->  D') {e | e  <<  x}) := by
      rw [ <-  f.preservesDirectedSup_coe {e | e  <<  x} <False, bot_wayBelow x> (
    directedOn_wayBelow x),
        hD.sSup_wayBelow x]
    rw [hfx] at hw
    have hne : (Set.image (f : D  ->  D') {e | e  <<  x}).Nonempty := <f False, False,
    bot_wayBelow x, rfl>
    have hdir : DirectedOn (*  <=  *) (Set.image (f : D  ->  D') {e | e  <<  x}) := by
      rintro _ <a, ha, rfl> _ <b, hb, rfl>
      obtain <c, hc, hac, hbc> := directedOn_wayBelow x a ha b hb
      exact <f c, Set.mem_image_of_mem _ hc, f.monotone hac, f.monotone hbc>
```

Lean 4 source (lines 401–800)

```
    obtain <_, <e, hex, rfl>, hwe> := (wayBelow_sSup_iff hne hdir).1 hw
    exact le_sSup <e, hex, hwe>

/-- **Scott 1972, Theorem 3.3 (order content).** If `D` and `D'` are continuous lattices, then
    so
is `[D  ->  D']` under the pointwise order. Every `f` is the supremum of the step functions way
     below
it: `f =  sqsup  {e[e,e'] | e'  <<  f e}`, and each such step function is way below `f`. -/
theorem theorem_3_3_isContinuousLattice (hD : IsContinuousLattice D)
    (hD' : IsContinuousLattice D') : IsContinuousLattice (ScottMap D D') := by
  intro f
  refine <fun h hh => hh.le, fun g hg => ?_>
  rw [ScottMap.le_def]
  intro x
  rw [ <-  stepMap_pointwise_sSup hD hD' f x]
  refine sSup_le ?_
  rintro e' <e, hex, he'>
  have hle : stepMap e e'  <=  g := hg (stepMap_wayBelow he')
  have hx := (ScottMap.le_def.1 hle) x
  rwa [stepMap_apply_of_wayBelow hex] at hx

/-! ### Theorem 3.3(b): the lattice topology = the topology of pointwise convergence

The Scott topology of the continuous lattice `[D  ->  D']` coincides with the product (
    pointwise
convergence) topology, whose subbasis is `{f | f x  in  U}` (`U` Scott-open in `D'`). One
    inclusion
(pointwise  subseteq  Scott) is immediate: each subbasic set is Scott-open in the lattice (
    joins are
pointwise). The other (Scott  subseteq  pointwise) is Scott's argument via the ` Up phi` basis
    of a continuous
lattice: `phi  <<  g` forces `phi  <=  sqsup _i e[e_i,e_i']` for finitely many pairs with `e_i
    '  <<  g e_i`, and the
finite intersection `?_i {h | e_i'  <<  h e_i}` is a pointwise-open neighbourhood of `g` inside
```

```
     ` Up phi`. -/

/-- A finite sup of elements way below `g` is way below `g`. -/
theorem wayBelow_finset_sup {? L : Type*} [CompleteLattice L] {s : Finset ?} {f : ?  ->  L} {g
    : L}
    (h : forall  i  in  s, f i  <<  g) : s.sup f  <<  g :=
  Finset.sup_induction (p := fun a => a  <<  g) (bot_wayBelow g) (fun _ ha _ hb => ha.sup hb) h

/-- Subbasic sets of the pointwise (product) topology on `[D  ->  D']`: `{f | f x  in  U}` for
    `U`
Scott-open in `D'`. -/
def scottMapPointwiseSubbasis (D D' : Type*) [CompleteLattice D] [CompleteLattice D'] :
    Set (Set (ScottMap D D')) :=
  { W | exists  (x : D) (U : Set D'),
      @IsOpen D' scottTopologicalSpace U  /\  W = {f : ScottMap D D' | (f : D  ->  D') x  in  U
    } }

/-- **Scott 1972, Definition 3.1 (on lattices).** The topology of pointwise convergence on
`[D  ->  D']`. -/
@[reducible] noncomputable def scottMapPointwiseTopology (D D' : Type*)
    [CompleteLattice D] [CompleteLattice D'] : TopologicalSpace (ScottMap D D') :=
  generateFrom (scottMapPointwiseSubbasis D D')

theorem pointwiseSubbasic_isOpen (x : D) {U : Set D'} (hU : @IsOpen D' scottTopologicalSpace U)
     :
    @IsOpen (ScottMap D D') (scottMapPointwiseTopology D D')
      {f : ScottMap D D' | (f : D  ->  D') x  in  U} :=
  isOpen_generateFrom_of_mem <x, U, hU, rfl>

/-- Each pointwise-subbasic set `{f | f x  in  U}` (`U` Scott-open) is Scott-open in the
    lattice
`[D  ->  D']`, because suprema there are pointwise. This is the easy inclusion pointwise
    subseteq  Scott. -/
theorem pointwiseSubbasic_scottOpen (x : D) {U : Set D'} (hU : ScottOpen U) :
    ScottOpen {f : ScottMap D D' | (f : D  ->  D') x  in  U} := by
  refine <fun f f' hff' hf => hU.1 (hff' x) hf, fun F hFne hFdir hmem => ?_>
  simp only [Set.mem_setOf_eq, ScottMap.sSup_apply] at hmem
  have hne : (Set.image (fun g : ScottMap D D' => (g : D  ->  D') x) F).Nonempty := hFne.image
    _
  have hdir : DirectedOn (*  <=  *) (Set.image (fun g : ScottMap D D' => (g : D  ->  D') x) F)
    := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hFdir a ha b hb
    exact <(c : D  ->  D') x, Set.mem_image_of_mem _ hc, hac x, hbc x>
  obtain <_, <g, hgF, rfl>, hg> := hU.2 hne hdir hmem
  exact <g, hgF, hg>

/-- **Step-function decomposition of ` << `.** If `phi  <<  g` in `[D  ->  D']`, then `phi`
    lies below a finite
join of step functions `e[e_i,e_i']` with `e_i'  <<  g e_i`. The finite joins of such step
    functions form
a directed family with supremum `g`, so `wayBelow_sSup_iff` produces one above `phi`. -/
theorem wayBelow_le_finset_sup_step (hD : IsContinuousLattice D) (hD' : IsContinuousLattice D')
    {phi g : ScottMap D D'} (h : phi  <<  g) :
    exists  F : Finset (D  x  D'), (forall  p  in  F, (p.2 : D')  <<  (g : D  ->  D') p.1)  /\
      phi  <=  F.sup (fun p => stepMap p.1 p.2) := by
  classical
  set Sg : Set (ScottMap D D') :=
    (fun F : Finset (D  x  D') => F.sup (fun p => stepMap p.1 p.2)) ''
      {F | forall  p  in  F, (p.2 : D')  <<  (g : D  ->  D') p.1} with hSg
  have hSgne : Sg.Nonempty := by
    refine <_, {}, ?_, rfl>
    intro p hp
```

```
    exact absurd hp (Finset.notMem_empty p)
  have hSgdir : DirectedOn (*  <=  *) Sg := by
    rintro _ <F_1, hF_1, rfl> _ <F_2, hF_2, rfl>
    refine <(F_1  U  F_2).sup (fun p => stepMap p.1 p.2), <F_1  U  F_2, fun p hp => ?_, rfl>,
      Finset.sup_mono Finset.subset_union_left, Finset.sup_mono Finset.subset_union_right>
    rcases Finset.mem_union.1 hp with hp | hp
    * exact hF_1 p hp
    * exact hF_2 p hp
  have hSgsup : sSup Sg = g := by
    apply le_antisymm
    * refine sSup_le ?_
      rintro _ <F, hF, rfl>
      exact Finset.sup_le fun p hp => stepMap_le_of_wayBelow (hF p hp)
    * rw [ScottMap.le_def]
      intro x
      rw [ <-  stepMap_pointwise_sSup hD hD' g x]
      refine sSup_le ?_
      rintro e' <e, hex, he'>
      have hmemSg : stepMap e e'  in  Sg := by
        refine <{(e, e')}, fun p hp => ?_, ?_>
        * rw [Finset.mem_singleton] at hp; subst hp; exact he'
        * show ({(e, e')} : Finset (D  x  D')).sup (fun p => stepMap p.1 p.2) = stepMap e e'
          rw [Finset.sup_singleton]
      have hx : (stepMap e e' : D  ->  D') x  <=  ((sSup Sg : ScottMap D D') : D  ->  D') x :=
    by
        rw [ScottMap.sSup_apply]
        exact le_sSup <stepMap e e', hmemSg, rfl>
      rwa [stepMap_apply_of_wayBelow hex] at hx
  rw [ <-  hSgsup] at h
  obtain <_, <F, hF, rfl>, hphis> := (wayBelow_sSup_iff hSgne hSgdir).1 h
  exact <F, hF, hphis.le>

/-- **Scott 1972, Theorem 3.3(b).** The Scott (lattice) topology on `[D  ->  D']` agrees with
    the
topology of pointwise convergence. -/
theorem theorem_3_3_topology (hD : IsContinuousLattice D) (hD' : IsContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) = scottMapPointwiseTopology D D'
     := by
  have hL : IsContinuousLattice (ScottMap D D') := theorem_3_3_isContinuousLattice hD hD'
  apply le_antisymm
  * -- pointwise  subseteq  Scott: each subbasic set is Scott-open in the lattice
    apply le_generateFrom_iff_subset_isOpen.2
    rintro W <x, U, hUopen, rfl>
    exact isOpen_iff_scottOpen.mpr (pointwiseSubbasic_scottOpen x (isOpen_iff_scottOpen.mp
    hUopen))
  * -- Scott  subseteq  pointwise: each Scott-open lattice set is pointwise-open, via the ` Up
    phi` basis
    intro U hU
    rw [isOpen_iff_scottOpen] at hU
    rw [@isOpen_iff_forall_mem_open (ScottMap D D') (scottMapPointwiseTopology D D')]
    intro g hgU
    obtain <phi, hphig, hphisub> := exists_wayBelow_subset hL hU hgU
    obtain <F, hF, hphiF> := wayBelow_le_finset_sup_step hD hD' hphig
    refine <? p  in  F, {h : ScottMap D D' | (p.2 : D')  <<  (h : D  ->  D') p.1}, ?_, ?_, ?_>
    * intro h hh
      refine hphisub (WayBelow.le_trans hphiF (wayBelow_finset_sup fun p hp => ?_))
      exact stepMap_wayBelow (Set.mem_iInter_2.1 hh p hp)
    * exact @isOpen_biInter_finset (ScottMap D D') (D  x  D') (scottMapPointwiseTopology D D')
    F _
        (fun p _ => pointwiseSubbasic_isOpen p.1 (isOpen_iff_scottOpen.mpr (scottOpen_wayBelow
    p.2)))
    * exact Set.mem_iInter_2.2 fun p hp => hF p hp
```

```
/-- **Scott 1972, Theorem 3.3 (full statement).** For continuous lattices `D`, `D'`, the
    function
space `[D  ->  D']` is a continuous lattice (`theorem_3_3_isContinuousLattice`) whose Scott
    topology
agrees with the topology of pointwise convergence (`theorem_3_3_topology`). -/
theorem theorem_3_3 (hD : IsContinuousLattice D) (hD' : IsContinuousLattice D') :
    IsContinuousLattice (ScottMap D D')  /\
      (scottTopologicalSpace : TopologicalSpace (ScottMap D D')) = scottMapPointwiseTopology D
    D' :=
  <theorem_3_3_isContinuousLattice hD hD', theorem_3_3_topology hD hD'>

/-! ### Corollary 3.4 -/

theorem scottFunctionSubbasis_isOpen_scott {x : D} {U : Set D'} (hU : @IsOpen D'
    scottTopologicalSpace U) :
    @IsOpen (ScottC D D') (@scottFunctionTopology D D' scottTopologicalSpace
    scottTopologicalSpace)
      {f : ScottC D D' | f x  in  U} :=
  isOpen_generateFrom_of_mem (s := {f : ScottC D D' | f x  in  U}) <x, U, hU, rfl>

theorem corollary_3_4_eval_on_C (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  continuous_def.2 fun _U hU => scottFunctionSubbasis_isOpen_scott (D := D) (D' := D') (x := x)
     hU

/-- **Scott 1972, Corollary 3.4 (fixed `x`).** Evaluation at fixed `x` is continuous on `[D  ->
      D']`
(with Scott topologies on `D` and `D'`); this is one half of the separate-continuity input to
joint continuity. -/
theorem corollary_3_4 (x : D) :
    @Continuous (ScottC D D') D'
      (@scottFunctionTopology D D' scottTopologicalSpace scottTopologicalSpace)
      scottTopologicalSpace (fun f : ScottC D D' => f x) :=
  corollary_3_4_eval_on_C (D := D) (D' := D') x

/-- **Scott 1972, Corollary 3.4 (joint continuity, core).** Evaluation `[D  ->  D']  x  D  ->
    D'`,
`(f, x)  |->  f x`, preserves directed suprema. By Proposition 2.6 it suffices to check
    separate
Scott-continuity: in `x` (with `f` fixed) it is `f`'s own continuity, and in `f` (with `x`
    fixed)
it is the pointwise formula for suprema in `[D  ->  D']` (`ScottMap.sSup_apply`). -/
theorem corollary_3_4_preservesDirectedSup :
    PreservesDirectedSup (fun p : ScottMap D D'  x  D => (p.1 : D  ->  D') p.2) := by
  rw [proposition_2_6]
  constructor
  * intro y S hS hSdir
    show ((sSup S : ScottMap D D') : D  ->  D') y
        = sSup ((fun x : ScottMap D D' => (x : D  ->  D') y) '' S)
    rw [ScottMap.sSup_apply]
  * intro x
    exact (proposition_2_5 _).mp x.continuous

/-- **Scott 1972, Corollary 3.4.** The evaluation map `eval : [D  ->  D']  x  D  ->  D'`, `(f,
    x)  |->  f x`,
is (jointly) Scott-continuous. Via Theorem 3.3(b) and Proposition 2.9(b) the Scott topology of
    the
product lattice is the product of the pointwise topology on `[D  ->  D']` and the Scott
    topology on
`D`, so this is exactly joint continuity for Scott's product topology. -/
theorem corollary_3_4_jointly_continuous :
```

```
    @Continuous (ScottMap D D'  x  D) D' scottTopologicalSpace scottTopologicalSpace
      (fun p : ScottMap D D'  x  D => (p.1 : D  ->  D') p.2) :=
  continuous_of_preservesDirectedSup corollary_3_4_preservesDirectedSup

/-! ### Proposition 3.5 (currying) -/

theorem sSup_image_prod_mk_left (x : D) (S : Set D') (hS : S.Nonempty) :
    sSup (Set.image (fun y => (x, y)) S) = (x, sSup S) := by
  have himage : Set.image (fun y => (x, y)) S = ({x} : Set D)  x ? S := by
    ext <a, b>
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    * rintro <y, hy, h>
      obtain <ha, hb> := Prod.ext_iff.mp h
      subst hb
      exact <ha.symm, hy>
    * rintro <ha, hb>
      refine <b, hb, Prod.ext_iff.mpr <ha.symm, rfl>>
  have hx : sSup ({x} : Set D) = x := by
    apply le_antisymm
    * exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    * exact le_sSup (Set.mem_singleton x)
  rw [himage, sSup_prod (hs := <x, Set.mem_singleton x>) (ht := hS), hx]

theorem curry_right_preservesDirectedSup (f : ScottMap (D  x  D') D'') (x : D) :
    PreservesDirectedSup (fun y => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (*  <=  *) (Set.image (fun y => (x, y)) S) := by
    intro p hp q hq
    obtain <a, ha, rfl> := hp
    obtain <b, hb, rfl> := hq
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <(x, c), Set.mem_image_of_mem _ hc, And.intro (And.intro le_rfl hac) (And.intro
    le_rfl hbc)>
  have hS'ne : (Set.image (fun y => (x, y)) S).Nonempty := by
    obtain <s, hs> := hS
    exact <(x, s), s, hs, rfl>
  rw [show (fun y => f (x, y)) (sSup S) = f (x, sSup S) from rfl,
     <-  sSup_image_prod_mk_left x S hS,
    f.preservesDirectedSup_coe (Set.image (fun y => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

/-- **Scott 1972, Proposition 3.5 (right).** Currying in the `y`-variable is Scott-continuous.
    -/
noncomputable def scottLambdaAt (f : ScottMap (D  x  D') D'') (x : D) : ScottMap D' D'' :=
  <fun y => f (x, y), continuous_of_preservesDirectedSup (curry_right_preservesDirectedSup f x)
    >

theorem sSup_image_prod_mk_right (y : D') (S : Set D) (hS : S.Nonempty) :
    sSup (Set.image (fun x => (x, y)) S) = (sSup S, y) := by
  have himage : Set.image (fun x => (x, y)) S = S  x ? ({y} : Set D') := by
    ext <a, b>
    simp only [Set.mem_image, Set.mem_prod, Set.mem_singleton_iff]
    constructor
    * rintro <x, hx, h>
      obtain <ha, hb> := Prod.ext_iff.mp h
      subst ha
      exact <hx, hb.symm>
    * rintro <ha, hb>
      exact <a, ha, Prod.ext_iff.mpr <rfl, hb.symm>>
  have hy : sSup ({y} : Set D') = y := by
    apply le_antisymm
```

```
    * exact sSup_le fun z hz => by rw [Set.mem_singleton_iff] at hz; rw [hz]
    * exact le_sSup (Set.mem_singleton y)
  rw [himage, sSup_prod (hs := hS) (ht := <y, Set.mem_singleton y>), hy]

/-- Currying in the `x`-variable: `x  |->  f (x, y)` is Scott-continuous (used for continuity
    of
`lambda f` as a map `D  ->  [D'  ->  D'']`). -/
theorem curry_left_preservesDirectedSup (f : ScottMap (D  x  D') D'') (y : D') :
    PreservesDirectedSup (fun x => f (x, y)) := by
  intro S hS hSdir
  have hS' : DirectedOn (*  <=  *) (Set.image (fun x => (x, y)) S) := by
    intro p hp q hq
    obtain <a, ha, rfl> := hp
    obtain <b, hb, rfl> := hq
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <(c, y), Set.mem_image_of_mem _ hc, And.intro (And.intro hac le_rfl) (And.intro hbc
    le_rfl)>
  have hS'ne : (Set.image (fun x => (x, y)) S).Nonempty := by
    obtain <s, hs> := hS
    exact <(s, y), s, hs, rfl>
  rw [show (fun x => f (x, y)) (sSup S) = f (sSup S, y) from rfl,
     <-  sSup_image_prod_mk_right y S hS,
    f.preservesDirectedSup_coe (Set.image (fun x => (x, y)) S) hS'ne hS']
  congr 1
  simp [Set.image_image]

/-- The outer half of currying: `x  |->  (y  |->  f (x, y))` preserves directed suprema (so `
    lambda f` is a
Scott map `D  ->  [D'  ->  D'']`). Equality in `[D'  ->  D'']` is pointwise, reducing to `
    curry_left`. -/
theorem lambda_outer_preservesDirectedSup (f : ScottMap (D  x  D') D'') :
    PreservesDirectedSup (fun x => scottLambdaAt f x) := by
  intro S hS hSdir
  apply ScottMap.ext
  intro y
  show f (sSup S, y)
      = ((sSup (Set.image (fun x => scottLambdaAt f x) S) : ScottMap D' D'') : D'  ->  D'') y
  rw [ScottMap.sSup_apply, Set.image_image]
  exact curry_left_preservesDirectedSup f y hS hSdir

/-- **Scott 1972, Proposition 3.5.** Functional abstraction
`lambda : [[D  x  D']  ->  D'']  ->  [D  ->  [D'  ->  D'']]`, `lambda f x y = f (x, y)`. By
    Theorem 3.3,
`[D  ->  [D'  ->  D'']]` is itself a continuous lattice, and `lambda f` is a Scott map. -/
noncomputable def scottLambda (f : ScottMap (D  x  D') D'') : ScottMap D (ScottMap D' D'') :=
  <fun x => scottLambdaAt f x,
    continuous_of_preservesDirectedSup (lambda_outer_preservesDirectedSup f)>

@[simp] theorem scottLambda_apply (f : ScottMap (D  x  D') D'') (x : D) (y : D') :
    ((scottLambda f x : ScottMap D' D'') : D'  ->  D'') y = (f : (D  x  D')  ->  D'') (x, y) :=
  rfl

/-- `lambda` preserves directed suprema: both sides evaluate, pointwise at `(x, y)`, to
` sqsup  {f (x, y) | f  in  ?}`, since suprema in every function lattice involved are pointwise
    . -/
theorem proposition_3_5_preservesDirectedSup :
    PreservesDirectedSup
      (scottLambda : ScottMap (D  x  D') D''  ->  ScottMap D (ScottMap D' D'')) := by
  intro ? h? h?dir
  apply ScottMap.ext
  intro x
  apply ScottMap.ext
  intro y
```

```
  rw [scottLambda_apply, ScottMap.sSup_apply, ScottMap.sSup_apply, ScottMap.sSup_apply,
    Set.image_image, Set.image_image]
  congr 1

/-- **Scott 1972, Proposition 3.5.** Functional abstraction `lambda` is Scott-continuous. -/
theorem proposition_3_5 :
    @Continuous (ScottMap (D  x  D') D'') (ScottMap D (ScottMap D' D''))
      scottTopologicalSpace scottTopologicalSpace scottLambda :=
  continuous_of_preservesDirectedSup proposition_3_5_preservesDirectedSup

/-! ### Definition 3.6 -/

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D D' : Type*) [CompleteLattice D] [CompleteLattice D']
    where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : forall  d, retr (incl d) = d

/-- **Scott 1972, Definition 3.6.** A *projection* of continuous lattices: a retract with
`i  comp  j  sqsub  id`. -/
structure IsContinuousLatticeProjection (D D' : Type*) [CompleteLattice D] [CompleteLattice D']
    extends IsContinuousLatticeRetraction D D' where
  incl_retr_le : forall  d, incl (retr d)  <=  d

namespace IsContinuousLatticeRetraction

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For `S  subseteq  D` a subspace
     of
ambient `D'`, write ` sqsup S'` for the supremum of `S` computed in `D'` (Scott's prime is on
    the
index, not the join). This is `sSup (i '' S)` in Lean. -/
noncomputable def ambientSSup (R : IsContinuousLatticeRetraction D D') (S : Set D) : D' :=
  sSup (Set.image (R.incl : D  ->  D') S)

/-- **Scott 1972, Prop 2.10 / March 1972 correction (p. 135).** For directed `S  subseteq  D`,
`j( sqsup S') =  sqsup S`: the retraction sends the ambient join back to the subspace join. -/
theorem retr_ambientSSup_eq_sSup (R : IsContinuousLatticeRetraction D D') (S : Set D)
    (hS : S.Nonempty) (hdir : DirectedOn (*  <=  *) S) :
    R.retr (ambientSSup R S) = sSup S := by
  unfold ambientSSup
  have hdir' : DirectedOn (*  <=  *) (Set.image (R.incl : D  ->  D') S) := by
    intro p hp q hq
    obtain <a, ha, rfl> := hp
    obtain <b, hb, rfl> := hq
    obtain <c, hc, hac, hbc> := hdir a ha b hb
    exact <R.incl c, Set.mem_image_of_mem _ hc, R.incl.monotone hac, R.incl.monotone hbc>
  have hne' : (Set.image (R.incl : D  ->  D') S).Nonempty := Set.image_nonempty.2 hS
  rw [R.retr.preservesDirectedSup_coe (Set.image (R.incl : D  ->  D') S) hne' hdir']
  rw [Set.image_image, Set.image_congr fun a _ => R.retr_incl a]
  simp [Set.image_id]

/-- The inclusion of a retraction preserves directed suprema (it is a Scott map). -/
theorem incl_preservesDirectedSup (R : IsContinuousLatticeRetraction D D') :
    PreservesDirectedSup (R.incl : D  ->  D') :=
  fun _ hS hSdir => R.incl.preservesDirectedSup_coe _ hS hSdir

/-- **Heart of Scott's proof of 2.10.** If `x'  <<  i(d)` in the ambient continuous lattice `D
    `,
then its image `j(x')` is way below `d` in the retract `D`. The Scott-open witness in `D` is
    the
preimage `i^-1V'` of an ambient Scott-open witness `V'` (Scott-open because `i` preserves
    directed
```

```
suprema); for `z  in  i^-1V'` we have `x'  sqsub  i(z)`, hence `j(x')  sqsub  j(i(z)) = z`. No
    projection
hypothesis is needed -- `j  comp  i = id` and monotonicity suffice. -/
theorem retr_wayBelow_of_wayBelow_incl (R : IsContinuousLatticeRetraction D D')
    {d : D} {x' : D'} (hx' : x'  <<  R.incl d) : R.retr x'  <<  d := by
  obtain <U', hU'open, hd_in, hU'sub> := hx'
  refine <(R.incl : D  ->  D') ^-1' U', scottOpen_preimage R.incl_preservesDirectedSup hU'open,
    hd_in, fun z hz => ?_>
  have hle : x'  <=  R.incl z := Set.mem_Ici.1 (hU'sub hz)
  have hjle := R.retr.monotone hle
  rw [R.retr_incl z] at hjle
  exact Set.mem_Ici.2 hjle

/-- For `d` in the retract, the ambient way-below set of `i(d)` pushed back by `j` is a
    directed
family whose supremum (computed in `D`) is `d`. This is the identity `j( sqsup S') =  sqsup S`
    applied to
`S = {j(x') : x'  <<  i(d)}`, combined with continuity of `D'`. -/
theorem sSup_image_retr_wayBelow (R : IsContinuousLatticeRetraction D D')
    (hD' : IsContinuousLattice D') (d : D) :
    sSup (Set.image (R.retr : D'  ->  D) {x' | x'  <<  R.incl d}) = d := by
  have hne : ({x' : D' | x'  <<  R.incl d}).Nonempty := <False, bot_wayBelow _>
  have hdir : DirectedOn (*  <=  *) {x' : D' | x'  <<  R.incl d} := directedOn_wayBelow _
  rw [ <-  R.retr.preservesDirectedSup_coe _ hne hdir, hD'.sSup_wayBelow (R.incl d), R.
    retr_incl d]

theorem image_retr_wayBelow_nonempty (R : IsContinuousLatticeRetraction D D') (d : D) :
    (Set.image (R.retr : D'  ->  D) {x' | x'  <<  R.incl d}).Nonempty :=
  <R.retr False, False, bot_wayBelow _, rfl>

theorem image_retr_wayBelow_directed (R : IsContinuousLatticeRetraction D D') (d : D) :
    DirectedOn (*  <=  *) (Set.image (R.retr : D'  ->  D) {x' | x'  <<  R.incl d}) := by
  rintro _ <a, ha, rfl> _ <b, hb, rfl>
  exact <R.retr (a  sqsup  b), <a  sqsup  b, WayBelow.sup ha hb, rfl>,
    R.retr.monotone le_sup_left, R.retr.monotone le_sup_right>

end IsContinuousLatticeRetraction

/-! ### Proposition 2.10: a retract of a continuous lattice is a continuous lattice -/

/-- **Scott 1972, Proposition 2.10(a).** A (Scott-continuous) retract of a continuous lattice
    is a
continuous lattice. For `d  in  D` we have, in the ambient `D'`, `i(d) =  sqsup {x' | x'  <<  i
    (d)}`; applying
the retraction `j` (which preserves this directed supremum) gives `d =  sqsup {j(x') | x'  <<
    i(d)}` in
`D`, and each `j(x')  <<  d` by `retr_wayBelow_of_wayBelow_incl`. Hence any upper bound of
```

Lean 4 source (lines 801–1200)

```
`{x | x  <<  d}` dominates `d`, so `d =  sqsup {x | x  <<  d}`. -/
theorem proposition_2_10_a (R : IsContinuousLatticeRetraction D D')
    (hD' : IsContinuousLattice D') : IsContinuousLattice D := by
  intro d
  refine <fun x hx => hx.le, fun b hb => ?_>
  rw [ <-  R.sSup_image_retr_wayBelow hD' d]
  exact sSup_le fun _ <x', hx', hxeq> => hxeq  |>  hb (R.retr_wayBelow_of_wayBelow_incl hx')

/-- **Scott 1972, Proposition 2.10(b) (March 1972 / Milner correction).** The Scott topology of
     the
retract `D` coincides with the subspace topology induced from the ambient `D'` along `i`.

* `scott  <=  induced`: each induced-open `i^-1V'` is Scott-open in `D` because `i` is a Scott
    map.
```

```
* `induced  <=  scott`: the sets `i^-1( Up x') = {z | x'  <<  i(z)}` (`x'  in  D'`) are a basis
     of `D`'s Scott
  topology -- given a Scott-open `U ? d`, the directed family `{j(x') | x'  <<  i(d)}` (sup `d
    `) meets
  `U` at some `j(x')`, and `i^-1( Up x')  subseteq  U` since `x'  <<  i(z) ? j(x')  sqsub  z`
    with `U` upper. Each such
  basic set is induced-open by construction, so every Scott-open of `D` is induced-open. -/
theorem proposition_2_10_b (R : IsContinuousLatticeRetraction D D')
    (hD' : IsContinuousLattice D') :
    (scottTopologicalSpace : TopologicalSpace D) =
      TopologicalSpace.induced (R.incl : D  ->  D') scottTopologicalSpace := by
  apply le_antisymm
  * intro U hU
    rw [isOpen_induced_iff (t := scottTopologicalSpace)] at hU
    obtain <V', hV'open, rfl> := hU
    rw [isOpen_iff_scottOpen] at hV'open  |-
    exact scottOpen_preimage R.incl_preservesDirectedSup hV'open
  * intro U hU
    rw [isOpen_iff_scottOpen] at hU
    rw [@isOpen_iff_forall_mem_open _
      (TopologicalSpace.induced (R.incl : D  ->  D') scottTopologicalSpace)]
    intro d hd
    have hmem : sSup (Set.image (R.retr : D'  ->  D) {x' | x'  <<  R.incl d})  in  U := by
      rw [R.sSup_image_retr_wayBelow hD' d]; exact hd
    obtain <p, hp, hpU> :=
      hU.2 (R.image_retr_wayBelow_nonempty d) (R.image_retr_wayBelow_directed d) hmem
    obtain <x', hx', rfl> := hp
    refine <(R.incl : D  ->  D') ^-1' {z' | x'  <<  z'}, fun z hz => ?_, ?_, hx'>
    * have hjle := R.retr.monotone (show x'  <=  R.incl z from (hz : x'  <<  R.incl z).le)
      rw [R.retr_incl z] at hjle
      exact hU.1 hjle hpU
    * rw [isOpen_induced_iff (t := scottTopologicalSpace)]
      exact <{z' | x'  <<  z'}, isOpen_iff_scottOpen.mpr (scottOpen_wayBelow x'), rfl>

/-- **Scott 1972, Proposition 2.10 (full statement).** A retract of a continuous lattice is a
continuous lattice (`proposition_2_10_a`) whose Scott topology agrees with the subspace
    topology
(`proposition_2_10_b`). -/
theorem proposition_2_10 (R : IsContinuousLatticeRetraction D D')
    (hD' : IsContinuousLattice D') :
    IsContinuousLattice D  /\
      (scottTopologicalSpace : TopologicalSpace D) =
        TopologicalSpace.induced (R.incl : D  ->  D') scottTopologicalSpace :=
  <proposition_2_10_a R hD', proposition_2_10_b R hD'>

/-! ### Proposition 3.7 -/

/-- **Scott 1972, Proposition 3.7 (retraction half).** If `D_n` is a retract of `D'_n`
(`n = 0,1`), then `[D_0  ->  D_1]` is a retract of `[D'_0  ->  D'_1]` via
`f  |->  i_1  comp  f  comp  j_0` and `f'  |->  j_1  comp  f'  comp  i_0`. -/
structure ScottMapRetraction (D_0 D_0' D_1 D_1' : Type*)
    [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [CompleteLattice D_1']
    where
  incl : ScottMap D_0 D_1  ->  ScottMap D_0' D_1'
  retr : ScottMap D_0' D_1'  ->  ScottMap D_0 D_1
  retr_incl : forall  f, retr (incl f) = f

/-- **Scott 1972, Proposition 3.7 (projection half).** Under the same hypotheses with
projections, the induced pair on function spaces is also a projection. -/
structure ScottMapProjection (D_0 D_0' D_1 D_1' : Type*)
    [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [CompleteLattice D_1']
    extends
    ScottMapRetraction D_0 D_0' D_1 D_1' where
```

```
  incl_retr_le : forall  f' x, (incl (retr f')) x  <=  f' x

namespace ScottMapRetraction

def functionSpace (R_0 : IsContinuousLatticeRetraction D_0 D_0') (R_1 :
    IsContinuousLatticeRetraction D_1 D_1') :
    ScottMapRetraction D_0 D_0' D_1 D_1' where
  incl f := ScottMap.comp R_1.incl (ScottMap.comp f R_0.retr)
  retr f' := ScottMap.comp R_1.retr (ScottMap.comp f' R_0.incl)
  retr_incl f := ScottMap.ext fun x => by simp [ScottMap.comp, R_0.retr_incl, R_1.retr_incl]

end ScottMapRetraction

namespace ScottMapProjection

def functionSpace (P_0 : IsContinuousLatticeProjection D_0 D_0') (P_1 :
    IsContinuousLatticeProjection D_1 D_1') :
    ScottMapProjection D_0 D_0' D_1 D_1' where
  toScottMapRetraction :=
    ScottMapRetraction.functionSpace P_0.toIsContinuousLatticeRetraction P_1.
    toIsContinuousLatticeRetraction
  incl_retr_le f' y := by
    dsimp only [ScottMapRetraction.functionSpace, ScottMap.comp, Function.comp_def]
    exact le_trans (P_1.incl_retr_le (f' (P_0.incl (P_0.retr y))))
      (ScottMap.monotone f' (P_0.incl_retr_le y))

end ScottMapProjection

/-- **Scott 1972, Proposition 3.7 (retraction).** Function spaces inherit retractions. -/
def proposition_3_7_retraction (R_0 : IsContinuousLatticeRetraction D_0 D_0')
    (R_1 : IsContinuousLatticeRetraction D_1 D_1') :
    ScottMapRetraction D_0 D_0' D_1 D_1' :=
  ScottMapRetraction.functionSpace R_0 R_1

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections. -/
def proposition_3_7_projection (P_0 : IsContinuousLatticeProjection D_0 D_0')
    (P_1 : IsContinuousLatticeProjection D_1 D_1') :
    ScottMapProjection D_0 D_0' D_1 D_1' :=
  ScottMapProjection.functionSpace P_0 P_1

/-! ### Proposition 3.10 -/

section Proposition310

open Finset

namespace IsContinuousLatticeProjection

/-- **Scott 1972, Proposition 3.10(ii).** The inclusion map of a projection is injective. -/
theorem incl_injective (P : IsContinuousLatticeProjection D D') {x y : D} (h : P.incl x = P.
    incl y) : x = y := by
  rw [ <-  P.retr_incl x,  <-  P.retr_incl y, h]

theorem retr_bot (P : IsContinuousLatticeProjection D D') : (P.retr : D'  ->  D) False = False
    := by
  have hle : (False : D')  <=  P.incl (False : D) := bot_le
  have heq : (P.retr : D'  ->  D) (P.incl (False : D)) = (False : D) := P.retr_incl (d := (
    False : D))
  exact le_antisymm (le_trans (P.retr.monotone hle) (le_of_eq heq)) bot_le

/-- **Scott 1972, Proposition 3.10(i), empty case.** `i(False) = False`. -/
theorem incl_bot (P : IsContinuousLatticeProjection D D') : P.incl (False : D) = False :=
  le_antisymm (by simpa [retr_bot P] using P.incl_retr_le (False : D')) bot_le
```

```
/-- **Scott 1972, Proposition 3.10(i), binary case.** `i(x  sqsup  y) = i(x)  sqsup  i(y)`. -/
theorem incl_sup (P : IsContinuousLatticeProjection D D') (x y : D) :
    P.incl (x  sqsup  y) = P.incl x  sqsup  P.incl y := by
  apply le_antisymm
  * have hx : x  <=  P.retr (P.incl x  sqsup  P.incl y) :=
      le_trans (le_of_eq (P.retr_incl x).symm) (P.retr.monotone le_sup_left)
    have hy : y  <=  P.retr (P.incl x  sqsup  P.incl y) :=
      le_trans (le_of_eq (P.retr_incl y).symm) (P.retr.monotone le_sup_right)
    exact le_trans (P.incl.monotone (sup_le hx hy)) (P.incl_retr_le (P.incl x  sqsup  P.incl y)
    )
  * exact sup_le (P.incl.monotone le_sup_left) (P.incl.monotone le_sup_right)

theorem incl_finset_sup (P : IsContinuousLatticeProjection D D') (T : Finset D) :
    P.incl (T.sup (fun x => x)) = T.sup (fun x => P.incl x) := by
  classical
  refine Finset.induction_on T ?base ?step
  * simp only [sup_empty]
    exact incl_bot P
  * intro a s _ ih
    simp only [sup_insert]
    rw [incl_sup P, ih]

/-- Finite sub-lubs used in Scott's proof of Proposition 3.10(i). -/
def finsetSupOf (S : Set D) : Set D :=
  { z | exists  T : Finset D, (forall  x  in  T, x  in  S)  /\  z = T.sup id }

theorem mem_finsetSupOf_of_mem {S : Set D} {x : D} (hx : x  in  S) :
    x  in  finsetSupOf S :=
  <{x}, by simp [hx], by simp>

theorem le_sSup_finsetSupOf (S : Set D) : sSup S  <=  sSup (finsetSupOf S) :=
  sSup_le fun _ hx => le_sSup (mem_finsetSupOf_of_mem hx)

theorem sSup_le_finsetSupOf (S : Set D) : sSup (finsetSupOf S)  <=  sSup S :=
  sSup_le fun z hz => by
    obtain <T, hTS, rfl> := hz
    exact Finset.sup_le fun t ht => le_sSup (hTS t ht)

theorem sSup_finsetSupOf (S : Set D) : sSup S = sSup (finsetSupOf S) :=
  le_antisymm (le_sSup_finsetSupOf S) (sSup_le_finsetSupOf S)

theorem directedOn_finsetSupOf (S : Set D) :
    DirectedOn (*  <=  *) (finsetSupOf S) := by
  classical
  intro a ha b hb
  obtain <T_1, hT_1, rfl> := ha
  obtain <T_2, hT_2, rfl> := hb
  have hunion : (T_1  U  T_2).sup id = T_1.sup id  sqsup  T_2.sup id := by
    simpa using Finset.sup_union (s_1 := T_1) (s_2 := T_2) (f := id)
  refine <T_1.sup id  sqsup  T_2.sup id, <T_1  U  T_2, ?_, hunion.symm>, le_sup_left,
    le_sup_right>
  intro x hx
  simp only [Finset.mem_union] at hx
  exact hx.elim (hT_1 x) (hT_2 x)

theorem directedOn_wayBelow (a : D) : DirectedOn (*  <=  *) { z | z  <<  a } := by
  intro p hp q hq
  exact <p  sqsup  q, WayBelow.sup hp hq, le_sup_left, le_sup_right>

/-- **Scott 1972, Proposition 3.10(i).** Projections preserve arbitrary suprema. -/
theorem incl_sSup (P : IsContinuousLatticeProjection D D') (S : Set D) :
    P.incl (sSup S) = sSup (Set.image (P.incl : D  ->  D') S) := by
```

```
  rcases S.eq_empty_or_nonempty with hS | hS
  * subst hS
    simp only [sSup_empty, Set.image_empty, incl_bot P]
  * rw [sSup_finsetSupOf S]
    have hdir := directedOn_finsetSupOf S
    have hne : (finsetSupOf S).Nonempty := by
      obtain <x, hx> := hS
      exact <x, mem_finsetSupOf_of_mem hx>
    rw [P.incl.preservesDirectedSup_coe (finsetSupOf S) hne hdir]
    apply le_antisymm
    * apply sSup_le
      intro b hb
      obtain <z, hz, rfl> := hb
      obtain <T, hT, rfl> := hz
      have h := incl_finset_sup P T
      conv_lhs => rw [show T.sup id = T.sup (fun x => x) from rfl]
      rw [h]
      apply Finset.sup_le
      intro t ht
      exact le_sSup (Set.mem_image_of_mem (P.incl : D  ->  D') (hT t ht))
    * apply sSup_le
      intro b hb
      obtain <x, hxS, rfl> := (Set.mem_image (P.incl : D  ->  D') S b).1 hb
      exact le_sSup (Set.mem_image_of_mem (P.incl : D  ->  D') (mem_finsetSupOf_of_mem hxS))

/-- **Scott 1972, Proposition 3.10(iii).** Projections preserve the way-below relation. -/
theorem incl_wayBelow (P : IsContinuousLatticeProjection D D') (hD' : IsContinuousLattice D')
    {x y : D} (h : x  <<  y) : @WayBelow D' _ (P.incl x) (P.incl y) := by
  set W : Set D' := { z' | @WayBelow D' _ z' (P.incl y) }
  have hWne : W.Nonempty := <(False : D'), @bot_wayBelow D' _ (P.incl y)>
  have hWdir : DirectedOn (*  <=  *) W := by
    intro p hp q hq
    exact <p  sqsup  q, @WayBelow.sup D' _ p q (P.incl y) hp hq, le_sup_left, le_sup_right>
  have hy : y = sSup (Set.image (P.retr : D'  ->  D) W) := by
    rw [ <-  P.retr.preservesDirectedSup_coe W hWne hWdir, @IsContinuousLattice.sSup_wayBelow D
    ' _ hD' (P.incl y),
      P.retr_incl y]
  have hImgne : (Set.image (P.retr : D'  ->  D) W).Nonempty := by
    obtain <z', hz'> := hWne
    exact <P.retr z', z', hz', rfl>
  have hImgdir : DirectedOn (*  <=  *) (Set.image (P.retr : D'  ->  D) W) := by
    intro p hp q hq
    obtain <a, ha, rfl> := hp
    obtain <b, hb, rfl> := hq
    refine <P.retr (a  sqsup  b), <a  sqsup  b, @WayBelow.sup D' _ a b (P.incl y) ha hb, rfl>,
    ?_, ?_>
    * exact P.retr.monotone le_sup_left
    * exact P.retr.monotone le_sup_right
  have hx : x  <<  sSup (Set.image (P.retr : D'  ->  D) W) := by rwa [ <-  hy]
  obtain <prz, hprz, hxpr> := (wayBelow_sSup_iff hImgne hImgdir).1 hx
  obtain <z', hz', rfl> := hprz
  exact @WayBelow.le_trans D' _ (P.incl x) z' (P.incl y)
    (le_trans (P.incl.monotone hxpr.le) (P.incl_retr_le z')) hz'

/-- **Scott 1972, Proposition 3.10(i)-(iii), bundled.** The inclusion `i` of a projection
    preserves
arbitrary suprema, is injective, and preserves the way-below relation. -/
theorem proposition_3_10_forward (P : IsContinuousLatticeProjection D D')
    (hD' : IsContinuousLattice D') :
    (forall  S : Set D, (P.incl : D  ->  D') (sSup S) = sSup (Set.image (P.incl : D  ->  D') S)
    )
       /\  Function.Injective (P.incl : D  ->  D')
       /\  (forall  x y : D, x  <<  y  ->  (P.incl : D  ->  D') x  <<  (P.incl : D  ->  D') y)
```

```
    :=
  <fun S => P.incl_sSup S, fun _ _ h => P.incl_injective h,
    fun _ _ h => P.incl_wayBelow hD' h>

/-- **Scott 1972, Proposition 3.10(iv), uniqueness.** The retraction of any projection is
    forced to
be Scott's formula `j(x') =  sqsup  { x | i(x)  sqsub  x' }`. ` <= `: `j(x')` itself satisfies
    `i(j x')  sqsub  x'`
(by `i  comp  j  sqsub  id`), so it is a member of the set; ` >= `: each member `x` (with `i x
     sqsub  x'`) satisfies
`x = j(i x)  sqsub  j(x')` by `j  comp  i = id` and monotonicity. -/
theorem retr_eq_sSup (P : IsContinuousLatticeProjection D D') (x' : D') :
    (P.retr : D'  ->  D) x' = sSup {x | (P.incl : D  ->  D') x  <=  x'} := by
  apply le_antisymm
  * exact le_sSup (show (P.incl : D  ->  D') (P.retr x')  <=  x' from P.incl_retr_le x')
  * refine sSup_le fun x hx => ?_
    calc x = (P.retr : D'  ->  D) (P.incl x) := (P.retr_incl x).symm
      _  <=  (P.retr : D'  ->  D) x' := P.retr.monotone hx

end IsContinuousLatticeProjection

/-! #### Proposition 3.10, converse direction

Given a map `i : D  ->  D'` satisfying (i)-(iii), Scott's formula (iv)
`j(x') =  sqsup  { x | i(x)  sqsub  x' }` is the unique continuous retraction making `D` a
    projection of `D'`. -/

variable {i : D  ->  D'}

/-- **Scott 1972, Proposition 3.10(iv).** Scott's candidate retraction `j(x') =  sqsup  { x | i
    (x)  sqsub  x' }`. -/
noncomputable def converseRetr (i : D  ->  D') (x' : D') : D :=
  sSup {x | i x  <=  x'}

theorem converseRetr_mono (i : D  ->  D') : Monotone (converseRetr i) := by
  intro a b hab
  exact sSup_le_sSup fun _ hx => le_trans hx hab

/-- From (i): `i` preserves binary joins (Scott checks (i) on two-element sets). -/
theorem incl_sup_of_preservesSSup (hi : forall  S : Set D, i (sSup S) = sSup (i '' S)) (x y : D
    ) :
    i (x  sqsup  y) = i x  sqsup  i y := by
  calc i (x  sqsup  y) = i (sSup {x, y}) := by rw [sSup_pair]
    _ = sSup (i '' {x, y}) := hi _
    _ = sSup {i x, i y} := by rw [Set.image_pair]
    _ = i x  sqsup  i y := sSup_pair

/-- From (i): `i` is monotone. -/
theorem incl_mono_of_preservesSSup (hi : forall  S : Set D, i (sSup S) = sSup (i '' S)) :
    Monotone i := by
  intro x y hxy
  have h := incl_sup_of_preservesSSup hi x y
  rw [sup_eq_right.mpr hxy] at h
  rw [h]; exact le_sup_left

/-- From (i)+(ii): `i` is order-reflecting (`x  sqsub  y  <->  i x  sqsub  i y`), since `x
    sqsub  y  <->  x  sqsup  y = y`. -/
theorem le_of_incl_le (hi : forall  S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) {x y : D} (h : i x  <=  i y) : x  <=  y := by
  have h1 : i (x  sqsup  y) = i y := by rw [incl_sup_of_preservesSSup hi, sup_eq_right.mpr h]
  exact sup_eq_right.mp (hinj h1)

/-- `i  comp  j  sqsub  id` (uses only (i)). -/
```

```
theorem incl_converseRetr_le (hi : forall  S : Set D, i (sSup S) = sSup (i '' S)) (x' : D') :
    i (converseRetr i x')  <=  x' := by
  show i (sSup {x | i x  <=  x'})  <=  x'
  rw [hi]
  exact sSup_le (by rintro _ <x, hx, rfl>; exact hx)

/-- `j  comp  i = id` (uses (i)+(ii)): `{x | i x  sqsub  i y} = Iic y`, whose join is `y`. -/
theorem converseRetr_incl (hi : forall  S : Set D, i (sSup S) = sSup (i '' S))
    (hinj : Function.Injective i) (y : D) : converseRetr i (i y) = y := by
  have hset : {x | i x  <=  i y} = Set.Iic y := by
    ext x
    simp only [Set.mem_setOf_eq, Set.mem_Iic]
    exact <le_of_incl_le hi hinj, fun hx => incl_mono_of_preservesSSup hi hx>
  show sSup {x | i x  <=  i y} = y
  rw [hset]
  exact le_antisymm (sSup_le fun _ hx => hx) (le_sSup le_rfl)

/-- `j` is Scott-continuous (uses (i)+(iii) and continuity of `D`). For directed `S'`:
    monotonicity
gives ` sqsup  j''S'  sqsub  j( sqsup S')`; conversely if `i x  sqsub   sqsup S'` then for
    every `z  <<  x` we have `i z  <<  i x  sqsub   sqsup S'`,
so `i z  sqsub  x'` for some `x'  in  S'` (directedness), whence `z  sqsub  j(x')  sqsub
    sqsup  j''S'`; continuity of `D`
then gives `x  sqsub   sqsup  j''S'`. -/
theorem converseRetr_preservesDirectedSup
    (hi : forall  S : Set D, i (sSup S) = sSup (i '' S))
    (hwb : forall  {x y : D}, x  <<  y  ->  i x  <<  i y) (hD : IsContinuousLattice D) :
    PreservesDirectedSup (converseRetr i) := by
  intro S' hS'ne hS'dir
  apply le_antisymm
  * show sSup {x | i x  <=  sSup S'}  <=  sSup (converseRetr i '' S')
    apply sSup_le
    intro x hx
    have hx' : i x  <=  sSup S' := hx
    rw [ <-  hD.sSup_wayBelow x]
    apply sSup_le
    intro z hz
    have hiz : i z  <<  sSup S' := (hwb hz).trans_le hx'
    obtain <x', hx'S', hzx'> := (wayBelow_sSup_iff hS'ne hS'dir).1 hiz
    have hz_mem : z  in  {x | i x  <=  x'} := hzx'.le
    exact le_trans (le_sSup hz_mem) (le_sSup <x', hx'S', rfl>)
  * apply sSup_le
    rintro _ <x', hx'S', rfl>
    exact converseRetr_mono i (le_sSup hx'S')

/-- The projection assembled from a map `i` satisfying 3.10(i)-(iii). -/
noncomputable def converseProjection
    (hi : forall  S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : forall  {x y : D}, x  <<  y  ->  i x  <<  i y) (hD : IsContinuousLattice D) :
    IsContinuousLatticeProjection D D' where
  incl := <i, continuous_of_preservesDirectedSup (fun _ _ _ => hi _)>
  retr := <converseRetr i, continuous_of_preservesDirectedSup
    (converseRetr_preservesDirectedSup hi hwb hD)>
  retr_incl := fun d => converseRetr_incl hi hinj d
  incl_retr_le := fun x' => incl_converseRetr_le hi x'

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D  ->  D'` (between continuous
    lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii) preservation of `
    << `,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given by Scott
    's
formula (iv) `j(x') =  sqsup  { x | i(x)  sqsub  x' }`. -/
```

```
theorem proposition_3_10_converse
    (hi : forall  S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : forall  {x y : D}, x  <<  y  ->  i x  <<  i y) (hD : IsContinuousLattice D) :
    exists  P : IsContinuousLatticeProjection D D',
      (forall  d, (P.incl : D  ->  D') d = i d)
        /\  (forall  x', (P.retr : D'  ->  D) x' = sSup {x | i x  <=  x'}) :=
  <conserveProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl>

end Proposition310

/-! ### Proposition 3.8 -/

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

/-- **Scott 1972, Proposition 3.8.** The infimum term ` sqcap  { f(x) : e(x)  in  U }` for open
     `U  subseteq  Y`. -/
noncomputable def scottSubspaceExtendInf (e : X  ->  Y) (f : X  ->  D) (U : Set Y) : D :=
  sInf (f '' (e ^-1' U))

/-- **Scott 1972, Proposition 3.8.** Scott's maximal subspace extension `f : Y  ->  D`. -/
noncomputable def scottSubspaceExtend (e : X  ->  Y) (f : X  ->  D) (y : Y) : D :=
  sSup { d | exists  U, @IsOpen Y scottTopologicalSpace U  /\  y  in  U  /\  d =
    scottSubspaceExtendInf e f U }

omit [TopologicalSpace Y] in
theorem scottSubspaceExtendInf_le_of_mem {f' : ScottMap Y D} {U : Set Y} {y : Y}
    (hyU : y  in  U) : sInf (Set.image (f' : Y  ->  D) U)  <=  f' y :=
  sInf_le (Set.mem_image_of_mem _ hyU)

theorem sInf_image_le_sInf_image_of_subset {a : Type*} {f : a  ->  D} {S T : Set a}
    (hST : S  subseteq  T) : sInf (Set.image f T)  <=  sInf (Set.image f S) :=
  sInf_le_sInf fun _ hd => by
    obtain <s, hs, rfl> := hd
    exact <s, hST hs, rfl>

theorem scottMap_eq_sSup_openInfs (hD : IsContinuousLattice D) (f' : ScottMap Y D) (y : Y) :
    (f' : Y  ->  D) y =
      sSup { d | exists  U, @IsOpen Y scottTopologicalSpace U  /\  y  in  U  /\
        d = sInf (Set.image (f' : Y  ->  D) U) } := by
  apply le_antisymm
  * rw [ <-  IsContinuousLattice.sSup_wayBelow hD (f' y)]
    apply sSup_le
```

Lean 4 source (lines 1201–1600)

```
    intro a ha
    obtain <V, hV, hf'V, hVsub> := ha
    have hVopen : @IsOpen D scottTopologicalSpace V := isOpen_iff_scottOpen.mpr hV
    have hyU : y  in  (f' : Y  ->  D) ^-1' V := hf'V
    have hUScott : ScottOpen ((f' : Y  ->  D) ^-1' V) :=
      scottOpen_preimage ((proposition_2_5 (Subtype.val f')).mp f'.property) hV
    have hUopen : @IsOpen Y scottTopologicalSpace ((f' : Y  ->  D) ^-1' V) :=
      isOpen_iff_scottOpen.mpr hUScott
    have ha' : a  <=  sInf (Set.image (f' : Y  ->  D) ((f' : Y  ->  D) ^-1' V)) := by
      apply le_sInf
      intro d hd
      obtain <z, hzV, rfl> := hd
      exact Set.mem_Ici.1 (hVsub hzV)
    have hmem : sInf (Set.image (f' : Y  ->  D) ((f' : Y  ->  D) ^-1' V))  in
        { d | exists  U, @IsOpen Y scottTopologicalSpace U  /\  y  in  U  /\
          d = sInf (Set.image (f' : Y  ->  D) U) } :=
      <(f' : Y  ->  D) ^-1' V, hUopen, hyU, rfl>
```

```
    exact le_trans ha' (le_sSup hmem)
  * apply sSup_le
    intro d hd
    obtain <U, _hUopen, hyU, rfl> := hd
    exact scottSubspaceExtendInf_le_of_mem (f' := f') hyU

theorem scottSubspaceExtendInf_eq_of_comp {e : X  ->  Y} {f : X  ->  D} {U : Set Y}
    (f' : ScottMap Y D) (h : forall  x, f' (e x) = f x) :
    scottSubspaceExtendInf e f U = sInf (Set.image (f' : Y  ->  D) (Set.image e Set.univ  I  U)
    ) := by
  have hset : f '' (e ^-1' U) = Set.image (f' : Y  ->  D) (Set.image e Set.univ  I  U) := by
    ext d
    simp only [Set.mem_image, Set.mem_preimage, Set.mem_inter_iff, Set.mem_univ, true_and]
    constructor
    * rintro <x, hxU, heq>
      refine <e x, <?_, hxU>, ?_>
      * exact <x, rfl>
      * rw [h x, heq]
    * rintro <y, <hex, hyU>, heq>
      obtain <x, _, rfl> := hex
      exact <x, hyU, (h x).symm.trans heq>
  rw [scottSubspaceExtendInf, hset]

/-- **Scott 1972, Proposition 3.8 (subspace variant).** `f` (with the Scott topology on `Y`) is
the maximal extension along a subspace embedding. The faithful statement (arbitrary topology on
`Y`) is `proposition_3_8` in `Constructions.lean`. -/
theorem scottSubspaceExtend_maximal (hD : IsContinuousLattice D) (e : X  ->  Y) (_he :
    IsEmbedding e)
    (f : X  ->  D) (f' : ScottMap Y D) (h_ext : forall  x, f' (e x) = f x) (y : Y) :
    (f' : Y  ->  D) y  <=  scottSubspaceExtend e f y := by
  have hEq := scottMap_eq_sSup_openInfs hD f' y
  have hmain :
      sSup { d | exists  U, @IsOpen Y scottTopologicalSpace U  /\  y  in  U  /\
        d = sInf (Set.image (f' : Y  ->  D) U) }  <=  scottSubspaceExtend e f y := by
    unfold scottSubspaceExtend
    apply sSup_le
    intro d hd
    obtain <U, hUopen, hyU, rfl> := hd
    have hmem : scottSubspaceExtendInf e f U  in
        { d | exists  U, @IsOpen Y scottTopologicalSpace U  /\  y  in  U  /\  d =
    scottSubspaceExtendInf e f U } :=
      <U, hUopen, hyU, rfl>
    refine le_trans ?_ (le_sSup hmem)
    have hrestrict :
        sInf (Set.image (f' : Y  ->  D) U)  <=
          sInf (Set.image (f' : Y  ->  D) (Set.image e Set.univ  I  U)) :=
      sInf_image_le_sInf_image_of_subset
        (show Set.image e Set.univ  I  U  subseteq  U from inter_subset_right)
    refine le_trans hrestrict (le_of_eq (scottSubspaceExtendInf_eq_of_comp (f' := f') h_ext).
    symm)
  exact le_trans (le_of_eq hEq) hmain

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
theorem scottSubspaceExtendInf_mono {e : X  ->  Y} {f g_0 : X  ->  D} (hfg : forall  x, f x  <=
      g_0 x) (U : Set Y) :
    scottSubspaceExtendInf e f U  <=  scottSubspaceExtendInf e g_0 U := by
  apply le_sInf
  intro d hd
  obtain <x, hxU, rfl> := hd
  exact le_trans (sInf_le <x, hxU, rfl>) (hfg x)

theorem scottSubspaceExtend_mono {e : X  ->  Y} {f g_0 : X  ->  D} (hfg : forall  x, f x  <=
    g_0 x) (y : Y) :
```

```
    scottSubspaceExtend e f y  <=  scottSubspaceExtend e g_0 y := by
  apply sSup_le
  intro d hd
  obtain <U, hUopen, hyU, rfl> := hd
  have hmem : scottSubspaceExtendInf e g_0 U  in
      { d | exists  U, @IsOpen Y scottTopologicalSpace U  /\  y  in  U  /\  d =
    scottSubspaceExtendInf e g_0 U } :=
    <U, hUopen, hyU, rfl>
  exact le_trans (scottSubspaceExtendInf_mono hfg U) (le_sSup hmem)

theorem scottSubspaceExtendInf_eq_of_ext {e : X  ->  Y} {f g' : X  ->  D} (h : forall  x, f x =
     g' x) (U : Set Y) :
    scottSubspaceExtendInf e f U = scottSubspaceExtendInf e g' U := by
  unfold scottSubspaceExtendInf
  congr 1
  ext d
  simp [Set.mem_image, Set.mem_preimage, h]

theorem scottSubspaceExtend_eq_of_ext {e : X  ->  Y} {f g' : X  ->  D} (h : forall  x, f x = g'
     x) (y : Y) :
    scottSubspaceExtend e f y = scottSubspaceExtend e g' y := by
  unfold scottSubspaceExtend
  congr 1
  ext d
  simp [Set.mem_setOf_eq, scottSubspaceExtendInf_eq_of_ext h]

theorem scottSubspaceExtendInf_mono_retr {e : X  ->  Y} {g : X  ->  D'}
    (P : IsContinuousLatticeProjection D D') (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U)  <=  scottSubspaceExtendInf e (fun x => P.retr (g x))
     U := by
  apply le_sInf
  intro d hd
  obtain <x, hxU, rfl> := hd
  have hx : x  in  e ^-1' U := Set.mem_preimage.mpr hxU
  exact (P.retr.monotone (sInf_le (Set.mem_image_of_mem g hx))).trans le_rfl

theorem scottSubspaceExtendInf_mono_incl {e : X  ->  Y} {f : X  ->  D} {g : X  ->  D'}
    (P : IsContinuousLatticeProjection D D') (hfg : forall  x, f x = P.retr (g x)) (U : Set Y)
    :
    scottSubspaceExtendInf e (fun x => P.incl (f x)) U  <=  scottSubspaceExtendInf e g U := by
  apply le_sInf
  intro d hd
  obtain <x, hxU, rfl> := hd
  have hx : x  in  e ^-1' U := Set.mem_preimage.mpr hxU
  have hmem : P.incl (f x)  in  (fun x => P.incl (f x)) '' (e ^-1' U) :=
    Set.mem_image_of_mem _ hx
  have hincl : P.incl (f x)  <=  g x := by rw [hfg x]; exact P.incl_retr_le (g x)
  exact le_trans (sInf_le hmem) hincl

theorem scottSubspaceExtendInf_mono_incl_apply {e : X  ->  Y} {f : X  ->  D} (U : Set Y)
    (P : IsContinuousLatticeProjection D D') :
    P.incl (scottSubspaceExtendInf e f U)  <=  scottSubspaceExtendInf e (fun x => P.incl (f x))
     U := by
  apply le_sInf
  intro d hd
  obtain <x, hxU, rfl> := hd
  exact P.incl.monotone (sInf_le (Set.mem_image_of_mem f (Set.mem_preimage.mpr hxU)))

/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X  ->  Y} (P : IsContinuousLatticeProjection D D')
    {f : X  ->  D} {g : X  ->  D'} (hfg : forall  x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U)  <=  scottSubspaceExtendInf e g U :=
```

```
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

/-- **Scott 1972, Lemma 3.9 (retraction at infimum level).** The global equality
`f = j  comp  g` assembles these infimum bounds via Proposition 3.10(i). -/
theorem lemma_3_9_retr_inf {e : X  ->  Y} (P : IsContinuousLatticeProjection D D') {g : X  ->
    D'}
    (U : Set Y) :
    P.retr (scottSubspaceExtendInf e g U)  <=  scottSubspaceExtendInf e (fun x => P.retr (g x))
     U :=
  scottSubspaceExtendInf_mono_retr (e := e) (g := g) P U

end SubspaceExtension

/-! ### Definition 3.11 / Proposition 3.12: the lattice of projections `J_D` -/

section Projections

/-- **Scott 1972, Definition 3.11.** `J_D = { j  in  [D  ->  D] : j = j  comp  j  sqsub  id }`:
     the Scott-continuous
projections of `D` (idempotent self-maps below the identity), as a predicate on `[D  ->  D]`.
    -/
def IsProjection (j : ScottMap D D) : Prop :=
  j.comp j = j  /\  j  <=  ScottMap.idMap

/-- Pointwise characterization of projections: idempotent and deflationary. -/
theorem isProjection_iff {j : ScottMap D D} :
    IsProjection j  <->  (forall  x, (j : D  ->  D) (j x) = j x)  /\  (forall  x, (j : D  ->  D
    ) x  <=  x) := by
  constructor
  * rintro <hidem, hle>
    refine <fun x => ?_, fun x => ?_>
    * have h : ((j.comp j : ScottMap D D) : D  ->  D) x = (j : D  ->  D) x := by rw [hidem]
      rwa [ScottMap.comp_apply] at h
    * have h := (ScottMap.le_def.mp hle) x
      rwa [ScottMap.idMap_apply] at h
  * rintro <hidem, hle>
    exact <ScottMap.ext fun x => by rw [ScottMap.comp_apply]; exact hidem x,
      ScottMap.le_def.mpr fun x => by rw [ScottMap.idMap_apply]; exact hle x>

/-- `False` (the constant `False` map) is a projection -- Scott's ` sqsup {}  in  J_D`. -/
theorem isProjection_bot : IsProjection (False : ScottMap D D) := by
  rw [isProjection_iff]
  refine <fun x => ?_, fun x => ?_>
  * simp only [ScottMap.bot_apply]
  * rw [ScottMap.bot_apply]; exact bot_le

/-- `J_D` is closed under binary joins (Scott 1972, 3.12). The key step: since `j x  sqsup  k x
      sqsub  x`,
monotonicity and idempotency force `j (j x  sqsup  k x) = j x` and `k (j x  sqsup  k x) = k x`.
     -/
theorem isProjection_sup {j k : ScottMap D D} (hj : IsProjection j) (hk : IsProjection k) :
    IsProjection (j  sqsup  k) := by
  rw [isProjection_iff] at hj hk  |-
  obtain <hjidem, hjle> := hj
  obtain <hkidem, hkle> := hk
  refine <fun x => ?_, fun x => ?_>
  * have hjkle : (j : D  ->  D) x  sqsup  k x  <=  x := sup_le (hjle x) (hkle x)
    have hj_eq : (j : D  ->  D) (j x  sqsup  k x) = j x :=
      le_antisymm (j.monotone hjkle) (le_of_eq_of_le (hjidem x).symm (j.monotone le_sup_left))
    have hk_eq : (k : D  ->  D) (j x  sqsup  k x) = k x :=
      le_antisymm (k.monotone hjkle) (le_of_eq_of_le (hkidem x).symm (k.monotone le_sup_right))
    rw [ScottMap.sup_apply, ScottMap.sup_apply, hj_eq, hk_eq]
```

```
  * rw [ScottMap.sup_apply]; exact sup_le (hjle x) (hkle x)

/-- `J_D` is closed under finite joins. -/
theorem isProjection_finsetSup {T : Finset (ScottMap D D)} (hT : forall  j  in  T, IsProjection
     j) :
    IsProjection (T.sup id) :=
  Finset.sup_induction isProjection_bot (fun _ ha _ hb => isProjection_sup ha hb) hT

/-- `J_D` is closed under *directed* joins (Scott 1972, 3.12). Continuity of each member lets
    the
inner `( sqsup S)`-application distribute over the directed family `{ j x : j  in  S }`, and
    directedness
plus idempotency collapse the double family `{ k (j x) }` to `( sqsup S) x`. -/
theorem isProjection_directedSup {S : Set (ScottMap D D)} (hSne : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) (hS : forall  j  in  S, IsProjection j) :
    IsProjection (sSup S) := by
  have hidem : forall  j  in  S, forall  x, (j : D  ->  D) (j x) = j x :=
    fun j hj => (isProjection_iff.mp (hS j hj)).1
  have hle : forall  j  in  S, forall  x, (j : D  ->  D) x  <=  x := fun j hj => (
    isProjection_iff.mp (hS j hj)).2
  rw [isProjection_iff]
  refine <fun x => ?_, fun x => ?_>
  * set A := Set.image (fun j : ScottMap D D => (j : D  ->  D) x) S with hA
    have hAne : A.Nonempty := hSne.image _
    have hAdir : DirectedOn (*  <=  *) A := by
      rintro _ <j, hj, rfl> _ <k, hk, rfl>
      obtain <m, hm, hjm, hkm> := hSdir j hj k hk
      exact <(m : D  ->  D) x, Set.mem_image_of_mem _ hm, hjm x, hkm x>
    have hsSx : ((sSup S : ScottMap D D) : D  ->  D) x = sSup A := ScottMap.sSup_apply S x
    apply le_antisymm
    * rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ <k, hk, rfl>
      show (k : D  ->  D) (sSup A)  <=  sSup A
      rw [k.preservesDirectedSup_coe A hAne hAdir]
      refine sSup_le ?_
      rintro _ <_, <j, hj, rfl>, rfl>
      obtain <m, hm, hjm, hkm> := hSdir j hj k hk
      calc (k : D  ->  D) (j x)  <=  (m : D  ->  D) (j x) := hkm (j x)
        _  <=  (m : D  ->  D) (m x) := m.monotone (hjm x)
        _ = (m : D  ->  D) x := hidem m hm x
        _  <=  sSup A := le_sSup (Set.mem_image_of_mem _ hm)
    * rw [hsSx, ScottMap.sSup_apply]
      refine sSup_le ?_
      rintro _ <j, hj, rfl>
      have hjxA : (j : D  ->  D) x  <=  sSup A := le_sSup (Set.mem_image_of_mem _ hj)
      calc (j : D  ->  D) x = (j : D  ->  D) (j x) := (hidem j hj x).symm
        _  <=  (j : D  ->  D) (sSup A) := j.monotone hjxA
        _  <=  sSup (Set.image (fun f : ScottMap D D => (f : D  ->  D) (sSup A)) S) :=
              le_sSup (Set.mem_image_of_mem _ hj)
  * rw [ScottMap.sSup_apply]
    refine sSup_le ?_
    rintro _ <j, hj, rfl>
    exact hle j hj x

/-- **Scott 1972, Proposition 3.12 (` sqsup `-closure).** `J_D` is closed under arbitrary
    suprema in
`[D  ->  D]`: every supremum is the directed supremum of finite sub-joins (`finsetSupOf`), each
     a
projection by `isProjection_finsetSup`. -/
theorem isProjection_sSup {T : Set (ScottMap D D)} (hT : forall  j  in  T, IsProjection j) :
    IsProjection (sSup T) := by
  rw [IsContinuousLatticeProjection.sSup_finsetSupOf T]
```

```
  refine isProjection_directedSup <False, {}, by simp, by simp>
    (IsContinuousLatticeProjection.directedOn_finsetSupOf T) ?_
  rintro z <F, hF, rfl>
  exact isProjection_finsetSup fun j hj => hT j (hF j hj)

/-- **Scott 1972, Definition 3.11.** The space `J_D` of projections of `D`, as a subtype of
`[D  ->  D]`. -/
abbrev Projections (D : Type*) [CompleteLattice D] : Type _ := {j : ScottMap D D //
    IsProjection j}

namespace Projections

noncomputable instance instSupSet : SupSet (Projections D) :=
  <fun T => <sSup (Set.image Subtype.val T), isProjection_sSup (by rintro j <p, _, rfl>; exact
    p.2)>>

theorem isLUB_sSup (T : Set (Projections D)) : IsLUB T (sSup T) := by
  constructor
  * intro p hp
    apply Subtype.coe_le_coe.mp
    show (p : ScottMap D D)  <=  ((sSup T : Projections D) : ScottMap D D)
    exact le_sSup (Set.mem_image_of_mem _ hp)
  * intro q hq
    apply Subtype.coe_le_coe.mp
    show ((sSup T : Projections D) : ScottMap D D)  <=  (q : ScottMap D D)
    refine sSup_le ?_
    rintro _ <p, hp, rfl>
    exact Subtype.coe_le_coe.mpr (hq hp)

/-- **Scott 1972, Proposition 3.12.** `J_D` is a complete lattice (a ` sqsup `-closed subspace
    of
`[D  ->  D]`). Suprema are inherited from `[D  ->  D]`; infima are derived by `
    completeLatticeOfSup`. -/
noncomputable instance instCompleteLattice : CompleteLattice (Projections D) :=
  completeLatticeOfSup (Projections D) isLUB_sSup

end Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous) lattice `D`,
    the
projections `J_D` form a complete lattice as a ` sqsup `-closed subspace of `[D  ->  D]`: the `
     sqsup `-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.instCompleteLattice`.
    -/
theorem proposition_3_12 :
    (forall  T : Set (ScottMap D D), (forall  j  in  T, IsProjection j)  ->  IsProjection (sSup
     T))
       /\  Nonempty (CompleteLattice (Projections D)) :=
  <fun _ h => isProjection_sSup h, <Projections.instCompleteLattice>>

end Projections

/-! ### Proposition 3.13: `D` is a projection of `[D  ->  D]` -/

namespace Proposition313

/-- **Scott 1972, Proposition 3.13.** `con : D  ->  [D  ->  D]` sends `x` to the constant
    function `x`. -/
noncomputable def con : ScottMap D (ScottMap D D) :=
  <fun x => ScottMap.const x, continuous_of_preservesDirectedSup (by
    intro S _ _
    apply ScottMap.ext
    intro y
```

```
    rw [ScottMap.sSup_apply, Set.image_image]
    simp only [ScottMap.const_apply, Set.image_id'])>

/-- **Scott 1972, Proposition 3.13.** `min : [D  ->  D]  ->  D` sends `f` to its least value `f
    (False)`. -/
noncomputable def min : ScottMap (ScottMap D D) D :=
  <fun f => (f : D  ->  D) False, continuous_of_preservesDirectedSup (by
    intro F _ _
    show ((sSup F : ScottMap D D) : D  ->  D) False
      = sSup (Set.image (fun f : ScottMap D D => (f : D  ->  D) False) F)
    rw [ScottMap.sSup_apply])>

@[simp] theorem con_apply (x y : D) : ((con x : ScottMap D D) : D  ->  D) y = x := rfl

@[simp] theorem min_apply (f : ScottMap D D) : (min f : D) = (f : D  ->  D) False := rfl

/-- **Scott 1972, Proposition 3.13.** `(con, min)` makes `D` a projection of `[D  ->  D]`:
`min  comp  con = id` (a constant's least value is its value) and `con  comp  min  sqsub  id` (
    the constant `f(False)`
is ` <=  f` pointwise, since `f(False)  sqsub  f(y)` by monotonicity). -/
noncomputable def projection : IsContinuousLatticeProjection D (ScottMap D D) where
  incl := con
  retr := min
  retr_incl := fun _ => rfl
  incl_retr_le := fun f => by
    rw [ScottMap.le_def]
    intro y
    show ((ScottMap.const ((f : D  ->  D) False) : ScottMap D D) : D  ->  D) y  <=  (f : D  ->
     D) y
    rw [ScottMap.const_apply]
    exact f.monotone bot_le

end Proposition313

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of its
    function
space `[D  ->  D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : IsContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  <Proposition313.projection>

/-! ### Proposition 3.14: the fixed-point operator `fix : [D  ->  D]  ->  D` -/

namespace Proposition314

/-- The monotone map underlying a Scott map, suitable for `OrderHom.lfp`. -/
def toOrderHom (f : ScottMap D D) : D  -> o D := <(f : D  ->  D), f.monotone>

/-- **Scott 1972, Proposition 3.14.** `fix f` is the least (pre-)fixed point of `f`, supplied
    by
mathlib's `OrderHom.lfp`. -/
noncomputable def fix (f : ScottMap D D) : D := (toOrderHom f).lfp

/-- `fix f` is a fixed point of `f`. -/
theorem fix_eq (f : ScottMap D D) : (f : D  ->  D) (fix f) = fix f :=
  (toOrderHom f).map_lfp

/-- `fix f` is below every pre-fixed point: `f x  sqsub  x ? fix f  sqsub  x`. -/
theorem fix_le {f : ScottMap D D} {x : D} (h : (f : D  ->  D) x  <=  x) : fix f  <=  x :=
  (toOrderHom f).lfp_le h

/-- `fix` is monotone in `f`: if `f  sqsub  f'` then `fix f  sqsub  fix f'`, since `f (fix f')
     sqsub  f' (fix f') =
```

```
fix f'` makes `fix f'` a pre-fixed point of `f`. -/
theorem fix_mono {f f' : ScottMap D D} (hff' : f  <=  f') : fix f  <=  fix f' :=
  fix_le (le_trans (ScottMap.le_def.mp hff' (fix f')) (le_of_eq (fix_eq f')))

/-- **Scott 1972, Proposition 3.14 (continuity).** `fix` preserves directed suprema. Direct
lattice argument (no Kleene iteration): write `g =  sqsup S` and `a =  sqsup {fix f : f  in  S
    }`. The reverse
bound `a  sqsub  fix g` is `fix`-monotonicity. For `fix g  sqsub  a` it suffices (by `fix_le`)
    that `a` is a
pre-fixed point of `g`. Now `g a =  sqsup _{f in S} f a` (pointwise sup), and each `f a =
    sqsup _{f' in S} f (fix f')`
(continuity of `f` on the directed family `{fix f'}`); for any `f, f'  in  S` pick `h  in  S`
    above both,
so `f (fix f')  sqsub  h (fix f')  sqsub  h (fix h) = fix h  sqsub  a`. Hence `g a  sqsub  a`.
    -/
theorem fix_preservesDirectedSup : PreservesDirectedSup (fix : ScottMap D D  ->  D) := by
  intro S hSne hSdir
  set T : Set D := Set.image fix S with hTdef
  have hTne : T.Nonempty := hSne.image fix
  have hTdir : DirectedOn (*  <=  *) T := by
    rintro _ <f, hf, rfl> _ <f', hf', rfl>
    obtain <h, hh, hfh, hf'h> := hSdir f hf f' hf'
    exact <fix h, Set.mem_image_of_mem fix hh, fix_mono hfh, fix_mono hf'h>
  show fix (sSup S) = sSup T
  apply le_antisymm
  * apply fix_le
    show ((sSup S : ScottMap D D) : D  ->  D) (sSup T)  <=  sSup T
    rw [ScottMap.sSup_apply]
    apply sSup_le
    rintro _ <f, hf, rfl>
    show (f : D  ->  D) (sSup T)  <=  sSup T
    rw [f.preservesDirectedSup_coe T hTne hTdir]
    apply sSup_le
    rintro _ <_, <f', hf', rfl>, rfl>
    obtain <h, hh, hfh, hf'h> := hSdir f hf f' hf'
    calc (f : D  ->  D) (fix f')
         <=  (h : D  ->  D) (fix f') := ScottMap.le_def.mp hfh (fix f')
      _  <=  (h : D  ->  D) (fix h) := h.monotone (fix_mono hf'h)
      _ = fix h := fix_eq h
      _  <=  sSup T := le_sSup (Set.mem_image_of_mem fix hh)
  * apply sSup_le
    rintro _ <f, hf, rfl>
    exact fix_mono (le_sSup hf)

/-- **Scott 1972, Proposition 3.14.** The fixed-point operator as a Scott-continuous map. -/
noncomputable def fixMap : ScottMap (ScottMap D D) D :=
  <fix, continuous_of_preservesDirectedSup fix_preservesDirectedSup>

@[simp] theorem fixMap_apply (f : ScottMap D D) : (fixMap f : D) = fix f := rfl
```

Lean 4 source (lines 1601–1626)

```
/-- Uniqueness: any value that is a fixed point of `f` and below every pre-fixed point equals
`fix f` (the least fixed point is unique). -/
theorem fix_unique {f : ScottMap D D} {a : D} (hfix : (f : D  ->  D) a = a)
    (hleast : forall  x, (f : D  ->  D) x  <=  x  ->  a  <=  x) : a = fix f :=
  le_antisymm (hleast (fix f) (le_of_eq (fix_eq f))) (fix_le hfix.le)

end Proposition314

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D  ->  D]  ->  D` such that `f (fix f) = fix f` for all `f`, and `
    fix f  sqsub  x`
```

```
whenever `f x  sqsub  x`. Existence and continuity are `Proposition314.fixMap`; the defining
    equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix`) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : IsContinuousLattice D) :
    exists  Fix : ScottMap (ScottMap D D) D,
      (forall  f : ScottMap D D, (f : D  ->  D) (Fix f) = Fix f)
         /\  (forall  (f : ScottMap D D) (x : D), (f : D  ->  D) x  <=  x  ->  (Fix f : D)  <=
     x)
         /\  (forall  g : ScottMap D D  ->  D,
            (forall  f : ScottMap D D, (f : D  ->  D) (g f) = g f)  ->
            (forall  (f : ScottMap D D) (x : D), (f : D  ->  D) x  <=  x  ->  g f  <=  x)  ->
            forall  f : ScottMap D D, g f = Fix f) :=
  <Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x hx)>

end Scott1972.ContinuousLattice
```

A.9 Scott1972/ContinuousLattice/Theorem212.lean

291 lines.

Lean 4 source

```
import Scott1972.ContinuousLattice.Constructions
import Scott1972.ContinuousLattice.FunctionSpaces

/-!
# Theorem 2.12: injective spaces are exactly the continuous lattices

Scott's Theorem 2.12 states that a `T_0`-space is injective iff it is a continuous lattice
    under
its Scott topology. The forward direction (continuous lattice ? injective) is
`theorem_2_12_forward` (= Proposition 2.11) in `Constructions.lean`.

This file supplies the **backward** direction. The argument (Scott 1972, S2) is:

* an injective space is a retract of a power of the Sierpinski space ` O  = Prop` (Corollary
    1.6);
* a power of ` O ` is a continuous lattice whose Scott topology is its product topology
  (`sierpinskiPower_isContinuousLattice`, `scottTopology_sierpinskiPower`);
* a retract of a continuous lattice is a continuous lattice (Proposition 2.10).

The retraction `r :  O ^I  ->  D` with section `s : D  ->  O ^I` makes `e := s  comp  r` a
    Scott-continuous
idempotent on ` O ^I`, whose fixed-point set is a continuous lattice (Proposition 2.10) and is
homeomorphic to `D`. Hence every injective space is homeomorphic to a continuous lattice under
its Scott topology.

The order-theoretic core is the construction `IdemFix`: the fixed points of a Scott-continuous
idempotent on a complete lattice form a complete lattice (`IdemFix.completeLattice`).
-/

namespace Scott1972.ContinuousLattice

open Topology Set

section FixedPoints

variable {L : Type*} [CompleteLattice L]

/-- The fixed-point set `{x | e x = x}` of an endomap `e`, as a subtype. -/
```

```
abbrev IdemFix (e : L  ->  L) : Type _ := {x : L // e x = x}

namespace IdemFix

variable {e : L  ->  L}

/-- The supremum in the fixed-point set: apply `e` to the ambient supremum (which lands back in
the fixed-point set because `e` is idempotent). -/
@[reducible] noncomputable def supSet (hidem : forall  x, e (e x) = e x) : SupSet (IdemFix e)
    :=
  <fun S => <e (sSup (Subtype.val '' S)), hidem _>>

/-- With the ambient-then-`e` supremum, every set has a least upper bound in `IdemFix e`. This
needs only that `e` is monotone (which follows from Scott continuity). -/
theorem isLUB_sSup (hidem : forall  x, e (e x) = e x) (hmono : Monotone e) :
    letI := supSet hidem
    forall  S : Set (IdemFix e), IsLUB S (sSup S) := by
  letI := supSet hidem
  intro S
  constructor
  * intro k hk
    rw [Subtype.coe_le_coe.symm, Subtype.coe_mk]
    show (k : L)  <=  e (sSup (Subtype.val '' S))
    calc (k : L) = e k := k.2.symm
      _  <=  e (sSup (Subtype.val '' S)) :=
          hmono (le_sSup (Set.mem_image_of_mem _ hk))
  * intro w hw
    rw [Subtype.coe_le_coe.symm, Subtype.coe_mk]
    show e (sSup (Subtype.val '' S))  <=  (w : L)
    calc e (sSup (Subtype.val '' S))  <=  e (w : L) := by
          refine hmono (sSup_le ?_)
          rintro _ <k, hk, rfl>
          exact hw hk
      _ = (w : L) := w.2

/-- **Fixed points of a monotone idempotent form a complete lattice.** The supremum is the
    ambient
supremum corrected by `e`; the rest follows from `completeLatticeOfSup`. -/
@[reducible] noncomputable def completeLattice (hidem : forall  x, e (e x) = e x) (hmono :
    Monotone e) :
    CompleteLattice (IdemFix e) :=
  letI := supSet hidem
  completeLatticeOfSup (IdemFix e) (isLUB_sSup hidem hmono)

/-- The ambient supremum corrected by `e` is the supremum in `IdemFix e`. -/
theorem coe_sSup (hidem : forall  x, e (e x) = e x) (hmono : Monotone e) (S : Set (IdemFix e))
    :
    letI := completeLattice hidem hmono
    ((sSup S : IdemFix e) : L) = e (sSup (Subtype.val '' S)) := rfl

end IdemFix

/-! ### The fixed-point set of a Scott-continuous idempotent is a continuous lattice -/

variable {e : L  ->  L}

theorem idemFix_incl_preservesDirectedSup (hidem : forall  x, e (e x) = e x)
    (hsc : PreservesDirectedSup e) :
    letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
    PreservesDirectedSup (Subtype.val : IdemFix e  ->  L) := by
  letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  intro S hS hSdir
  show e (sSup (Subtype.val '' S)) = sSup (Subtype.val '' S)
```

```
  have hdir' : DirectedOn (*  <=  *) (Subtype.val '' S) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <c.val, Set.mem_image_of_mem _ hc, hac, hbc>
  have hne' : (Subtype.val '' S).Nonempty := hS.image _
  rw [hsc hne' hdir']
  congr 1
  rw [ <-  Set.image_comp]
  exact Set.image_congr (fun s _ => s.2)

theorem idemFix_retr_preservesDirectedSup (hidem : forall  x, e (e x) = e x)
    (hsc : PreservesDirectedSup e) :
    letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
    PreservesDirectedSup (fun x : L => (<e x, hidem x> : IdemFix e)) := by
  letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  have hmono := preservesDirectedSup_monotone hsc
  intro T hT hTdir
  apply Subtype.ext
  rw [IdemFix.coe_sSup hidem hmono]
  show e (sSup T) = e (sSup (Subtype.val '' ((fun x : L => (<e x, hidem x> : IdemFix e)) '' T))
    )
  have himg : Subtype.val '' ((fun x : L => (<e x, hidem x> : IdemFix e)) '' T) = e '' T := by
    rw [ <-  Set.image_comp]; rfl
  rw [himg]
  have hTdir' : DirectedOn (*  <=  *) (e '' T) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hTdir a ha b hb
    exact <e c, Set.mem_image_of_mem _ hc, hmono hac, hmono hbc>
  have hTne' : (e '' T).Nonempty := hT.image _
  rw [hsc hT hTdir, hsc hTne' hTdir']
  congr 1
  rw [ <-  Set.image_comp]
  exact Set.image_congr (fun x _ => (hidem x).symm)

/-- The fixed-point set of a Scott-continuous idempotent `e` on `L`, with the
ambient-supremum-corrected complete-lattice structure, is a *retract of `L`*: the inclusion and
     the
corestriction of `e` are Scott-continuous and compose to the identity. -/
noncomputable def idemFixRetraction (hidem : forall  x, e (e x) = e x) (hsc :
    PreservesDirectedSup e) :
    @IsContinuousLatticeRetraction (IdemFix e) L
      (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)) _ :=
  letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  { incl := <Subtype.val,
      continuous_of_preservesDirectedSup (idemFix_incl_preservesDirectedSup hidem hsc)>
    retr := <fun x => <e x, hidem x>,
      continuous_of_preservesDirectedSup (idemFix_retr_preservesDirectedSup hidem hsc)>
    retr_incl := fun k => Subtype.ext k.2 }

/-- **Fixed points of a Scott-continuous idempotent form a continuous lattice.** Combine the
retraction `idemFixRetraction` with Proposition 2.10(a). -/
theorem idemFix_isContinuousLattice (hidem : forall  x, e (e x) = e x) (hsc :
    PreservesDirectedSup e)
    (hL : IsContinuousLattice L) :
    @IsContinuousLattice (IdemFix e)
      (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)) := by
  letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  exact proposition_2_10_a (idemFixRetraction hidem hsc) hL

/-- **Scott topology of the fixed-point lattice = subspace topology.** Proposition 2.10(b)
    applied
to `idemFixRetraction`: the Scott topology of `IdemFix e` is the topology induced by the
    inclusion
```

```
from the Scott topology of `L`. -/
theorem idemFix_scottTopology (hidem : forall  x, e (e x) = e x) (hsc : PreservesDirectedSup e)
    (hL : IsContinuousLattice L) :
    @scottTopologicalSpace (IdemFix e)
        (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)) =
      TopologicalSpace.induced (Subtype.val : IdemFix e  ->  L) scottTopologicalSpace := by
  letI := IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)
  exact proposition_2_10_b (idemFixRetraction hidem hsc) hL

end FixedPoints

/-! ### Theorem 2.12, backward direction -/

universe u

/-- **Scott 1972, Theorem 2.12 (backward direction).** Every injective `T_0`-space is
    homeomorphic
to a continuous lattice equipped with its Scott topology.

The injective space `D` is a retract of a power ` O ^I = (I  ->  Prop)` of the Sierpinski space
(Corollary 1.6). Writing `s : D  ->  O ^I` for the embedding and `r :  O ^I  ->  D` for the
    retraction,
`e := s  comp  r` is a Scott-continuous idempotent on ` O ^I` (its topology is its Scott
    topology by
`scottTopology_sierpinskiPower`). The fixed-point set `IdemFix e` is therefore a continuous
    lattice
(`idemFix_isContinuousLattice`, via Proposition 2.10), and `d  |->  s d` is a homeomorphism `D
    ??
IdemFix e` with inverse the retraction. -/
theorem theorem_2_12_backward {D : Type u} [TopologicalSpace D] [T0Space D]
    (hD : IsInjectiveSpace.{u, u} D) :
    exists  (E : Type u) (inst : CompleteLattice E),
      @IsContinuousLattice E inst  /\
        Nonempty (@Homeomorph D E _ (@scottTopologicalSpace E inst)) := by
  obtain <?, <R>> := (corollary_1_6 D).1 hD
  -- `e = s  comp  r` on the Sierpinski power `L = ?  ->  Prop`.
  have hidem : forall  x, R.section' (R.retraction (R.section' (R.retraction x)))
      = R.section' (R.retraction x) :=
    fun x => congrArg R.section' (R.retraction_section (R.retraction x))
  have hL : IsContinuousLattice (?  ->  Prop) := sierpinskiPower_isContinuousLattice ?
  -- `e` is Scott-continuous: it is product-continuous (`s  comp  r`), and the Scott topology
    of the
  -- Sierpinski power is its product topology.
  have hcont_e : @Continuous (?  ->  Prop) (?  ->  Prop) scottTopologicalSpace
    scottTopologicalSpace
      (fun x => R.section' (R.retraction x)) := by
    rw [scottTopology_sierpinskiPower ?]
    exact R.section'.continuous.comp R.retraction.continuous
  have hsc : PreservesDirectedSup (fun x => R.section' (R.retraction x)) :=
    continuous_preservesDirectedSup hcont_e
  refine <IdemFix (fun x => R.section' (R.retraction x)),
    IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc),
    idemFix_isContinuousLattice hidem hsc hL, ?_>
  -- The Scott topology of `IdemFix e` equals the subspace topology it inherits as a subtype of
     the
  -- Sierpinski power: both are `induced Subtype.val` of the (Scott = product) topology of ` O
    ^I`.
  have heq : (@scottTopologicalSpace (IdemFix (fun x => R.section' (R.retraction x)))
      (IdemFix.completeLattice hidem (preservesDirectedSup_monotone hsc)))
      = instTopologicalSpaceSubtype := by
    rw [idemFix_scottTopology hidem hsc hL, scottTopology_sierpinskiPower ?]
    rfl
  rw [heq]
```

```
  -- the homeomorphism `D ?? IdemFix e`, with the subspace topology (where `Continuous.comp`
    works)
  have hfix : forall  d, (fun x => R.section' (R.retraction x)) (R.section' d) = R.section' d
    :=
    fun d => congrArg R.section' (R.retraction_section d)
  exact <{
    toFun := fun d => <R.section' d, hfix d>
    invFun := fun k => R.retraction k.1
    left_inv := fun d => R.retraction_section d
    right_inv := fun k => Subtype.ext k.2
    continuous_toFun := by exact R.section'.continuous.subtype_mk hfix
    continuous_invFun := by exact R.retraction.continuous.comp continuous_subtype_val }>

/-- **Scott 1972, Theorem 2.12.** A `T_0`-space is injective iff it is (homeomorphic to) a
continuous lattice under its Scott topology. The forward direction is Proposition 2.11; the
backward direction is `theorem_2_12_backward`. -/
theorem theorem_2_12 {D : Type u} [TopologicalSpace D] [T0Space D] :
    IsInjectiveSpace.{u, u} D  <->
      exists  (E : Type u) (inst : CompleteLattice E),
        @IsContinuousLattice E inst  /\
          Nonempty (@Homeomorph D E _ (@scottTopologicalSpace E inst)) := by
  refine <theorem_2_12_backward, ?_>
  rintro <E, inst, hE, <h>>
  letI tauE : TopologicalSpace E := @scottTopologicalSpace E inst
  exact @IsInjectiveSpace.of_retract E D tauE _ (theorem_2_12_forward hE)
    <?h, h.continuous> <?h.symm, h.symm.continuous> (fun d => h.left_inv d)

/-! ### Lemma 3.9: compatibility of maximal extensions with a projection on the range -/

section Lemma39

variable {X Y D D' : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [CompleteLattice D] [CompleteLattice D']

/-- **Scott 1972, Lemma 3.9.** With `e : X  ->  Y` a subspace embedding and `i, j : D  <=>  D'`
     a
projection on the range, if continuous `f : X  ->  D` and `g : X  ->  D'` satisfy `f = j  comp
     g`, then the
maximal extensions (Proposition 3.8) satisfy `f = j  comp  g`.

The proof is Scott's, recast through the faithful `scottExtend`:
* `j  comp  g  sqsub  f`: `j  comp  g` is a continuous solution of `(j  comp  g)  comp  e = j
    comp  g = f`, so maximality of
  `f` (`scottExtend_maximal`) gives the bound;
* `i  comp  f  sqsub  g`: `(i  comp  f)  comp  e = i  comp  f = i  comp  j  comp  g  sqsub  g`
    (since `i  comp  j  sqsub  id`), so the *sub*-solution
  maximality of `g` (`scottExtend_maximal_le`, the remark after 3.8) gives the bound;
* combining: `f = j  comp  i  comp  f  sqsub  j  comp  g  sqsub  f` (using `j  comp  i = id`
    and monotonicity of `j`). -/
theorem lemma_3_9 (hD : IsContinuousLattice D) (hD' : IsContinuousLattice D')
    (P : IsContinuousLatticeProjection D D') (e : X  ->  Y) (he : IsEmbedding e)
    {f : X  ->  D} {g : X  ->  D'} (hf : @Continuous X D _ scottTopologicalSpace f)
    (hg : @Continuous X D' _ scottTopologicalSpace g)
    (hfg : forall  x, f x = (P.retr : D'  ->  D) (g x)) (y : Y) :
    scottExtend e f y = (P.retr : D'  ->  D) (scottExtend e g y) := by
  have hfbar : @Continuous Y D _ scottTopologicalSpace (scottExtend e f) :=
    scottExtend_continuous hD e f
  have hgbar : @Continuous Y D' _ scottTopologicalSpace (scottExtend e g) :=
    scottExtend_continuous hD' e g
  -- `j  comp  g` and `i  comp  f` are continuous (Scott topology is not an instance; register
    it locally so
  -- `Continuous.comp` can fire, scoped to avoid the lattice ` <= ` clashing with
    specialization order).
```

```
  have hjg : @Continuous Y D _ scottTopologicalSpace (fun z => (P.retr : D'  ->  D) (
    scottExtend e g z)) := by
    letI : TopologicalSpace D := scottTopologicalSpace
    letI : TopologicalSpace D' := scottTopologicalSpace
    exact P.retr.continuous.comp hgbar
  have hif : @Continuous Y D' _ scottTopologicalSpace (fun z => (P.incl : D  ->  D') (
    scottExtend e f z)) := by
    letI : TopologicalSpace D := scottTopologicalSpace
    letI : TopologicalSpace D' := scottTopologicalSpace
    exact P.incl.continuous.comp hfbar
  -- Step A: `j  comp  g  sqsub  f`.
  have hA : forall  z, (P.retr : D'  ->  D) (scottExtend e g z)  <=  scottExtend e f z := by
    intro z
    refine scottExtend_maximal hD e hjg ?_ z
    intro x
    show (P.retr : D'  ->  D) (scottExtend e g (e x)) = f x
    rw [scottExtend_eq_of_continuous hD' e he g hg x,  <-  hfg x]
  -- Step B: `i  comp  f  sqsub  g`.
  have hB : forall  z, (P.incl : D  ->  D') (scottExtend e f z)  <=  scottExtend e g z := by
    intro z
    refine scottExtend_maximal_le hD' e hif ?_ z
    intro x
    show (P.incl : D  ->  D') (scottExtend e f (e x))  <=  g x
    rw [scottExtend_eq_of_continuous hD e he f hf x, hfg x]
    exact P.incl_retr_le (g x)
  refine le_antisymm ?_ (hA y)
  calc scottExtend e f y
      = (P.retr : D'  ->  D) ((P.incl : D  ->  D') (scottExtend e f y)) :=
        (P.retr_incl (scottExtend e f y)).symm
    _  <=  (P.retr : D'  ->  D) (scottExtend e g y) := P.retr.monotone (hB y)

end Lemma39

end Scott1972.ContinuousLattice
```

A.10 Scott1972/ContinuousLattice/InverseLimits.lean

595 lines.

Lean 4 source (lines 1–400)

```
import Scott1972.ContinuousLattice.Constructions
import Scott1972.ContinuousLattice.FunctionSpaces
import Mathlib.Order.GaloisConnection.Basic

/-!
# Inverse limits of continuous lattices (Scott 1972, S4)

We formalize Scott's **Proposition 4.1**: the inverse limit `D_ inf ` of an ?-system of
    continuous
lattices `<D_n, j_n>` with each `j_n : D_{n+1}  ->  D_n` a projection is again a continuous
    lattice.

Scott proves this through injectivity (`D_ inf ` is an injective `T_0`-space, hence -- Theorem
    2.12 -- a
continuous lattice), using the maximal-extension Proposition 3.8 and the compatibility Lemma
    3.9.
The *retraction* of `? D_n` onto `D_ inf ` that Scott's argument constructs (extend the
    identity of
`D_ inf ` along its inclusion) is realized here **order-theoretically and without topology**:

* each projection `j_n = (P n).retr` is the upper adjoint of its embedding `i_n = (P n).incl`
  (`projection_galoisConnection`), hence preserves arbitrary infima;
```

```
* therefore the compatibility predicate is closed under pointwise `sInf`, making `D_ inf ` a
    complete
  lattice (`completeLatticeOfInf`);
* the inclusion `D_ inf  ? ? D_n` preserves infima, so it has a **left adjoint** `r : ? D_n  ->
      D_ inf `
  (`invLimRetr`); a left adjoint preserves all suprema, in particular directed ones, so `r` is
  Scott-continuous, and `r  comp  incl = id`.

Thus `D_ inf ` is a (Scott-continuous) retract of the continuous lattice `? D_n` (Prop 2.9a),
    so by
Prop 2.10a it is a continuous lattice. This is exactly the retraction Scott builds via
    injectivity,
obtained here as the adjoint of the inclusion.
-/

namespace Scott1972.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]
variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `i_n  dashv  j_n`:
    from
`j_n  comp  i_n = id` and `i_n  comp  j_n  sqsub  id` we get `i_n x  sqsub  y  <->  x  sqsub
    j_n y`. In particular `j_n` (the upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : Nat) :
    GaloisConnection ((P n).incl : D n  ->  D (n + 1)) ((P n).retr : D (n + 1)  ->  D n) := by
  intro x y
  constructor
  * intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  * intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : forall  n, D n) : Prop := forall  n, (P n).retr (x (n + 1)) = x n

/-- **Scott 1972, S4.** The inverse limit `D_ inf ` as the subspace of compatible sequences. -/
abbrev InverseLimit : Type u := {x : forall  n, D n // Compatible D P x}

/-- `j_n` preserves arbitrary infima (it is the upper adjoint of `i_n`). -/
theorem retr_sInf (n : Nat) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (sInf (Subtype.val '' S)) := by
  intro n
  rw [sInf_apply_eq_sInf_image, sInf_apply_eq_sInf_image, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro _ <x, _, rfl>; exact x.2 n)

noncomputable instance : InfSet (InverseLimit D P) :=
  <fun S => <sInf (Subtype.val '' S), compatible_sInf D P S>>
```

```
theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : forall  n, D n) = sInf (Subtype.val '' S) := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) : IsGLB S (sInf S) := by
  constructor
  * intro x hx
    refine Subtype.coe_le_coe.mp ?_
    rw [coe_sInf]
    exact sInf_le (Set.mem_image_of_mem _ hx)
  * intro b hb
    refine Subtype.coe_le_coe.mp ?_
    rw [coe_sInf]
    refine le_sInf ?_
    rintro _ <x, hx, rfl>
    exact Subtype.coe_le_coe.mpr (hb hx)

noncomputable instance instCompleteLattice : CompleteLattice (InverseLimit D P) :=
  completeLatticeOfInf (InverseLimit D P) (isGLB_sInf' D P)

/-- For a directed, nonempty family, the inverse-limit supremum is computed pointwise (each `
    j_n`
is Scott-continuous, so the pointwise sup of compatible sequences is compatible and is the
    least
upper bound in `D_ inf `). -/
theorem coe_sSup_of_directed (S : Set (InverseLimit D P)) (hSne : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) :
    ((sSup S : InverseLimit D P) : forall  n, D n) = sSup (Subtype.val '' S) := by
  have hcompat : Compatible D P (sSup (Subtype.val '' S)) := by
    intro n
    rw [sSup_apply_eq_sSup_image, sSup_apply_eq_sSup_image]
    set A : Set (D (n + 1)) := (fun f : forall  m, D m => f (n + 1)) '' (Subtype.val '' S) with
     hA
    have hAne : A.Nonempty := (hSne.image _).image _
    have hAdir : DirectedOn (*  <=  *) A := by
      rintro _ <_, <x, hxS, rfl>, rfl> _ <_, <y, hyS, rfl>, rfl>
      obtain <z, hzS, hxz, hyz> := hSdir x hxS y hyS
      exact <z.1 (n + 1), <z.1, <z, hzS, rfl>, rfl>, hxz (n + 1), hyz (n + 1)>
    rw [(P n).retr.preservesDirectedSup_coe A hAne hAdir]
    congr 1
    rw [hA, Set.image_image]
    exact Set.image_congr (by rintro _ <x, _, rfl>; exact x.2 n)
  set p : InverseLimit D P := <sSup (Subtype.val '' S), hcompat> with hp
  have hlub : IsLUB S p := by
    constructor
    * intro x hx
      refine Subtype.coe_le_coe.mp ?_
      exact le_sSup (Set.mem_image_of_mem _ hx)
    * intro b hb
      refine Subtype.coe_le_coe.mp ?_
      refine sSup_le ?_
      rintro _ <x, hx, rfl>
      exact Subtype.coe_le_coe.mpr (hb hx)
  rw [(isLUB_sSup S).unique hlub]

/-- The inclusion `D_ inf  ? ? D_n` preserves directed suprema. -/
theorem incl_preservesDirectedSup :
    PreservesDirectedSup (Subtype.val : InverseLimit D P  ->  forall  n, D n) := by
  intro S hSne hSdir
  exact coe_sSup_of_directed D P S hSne hSdir

/-- Scott's retraction `r : ? D_n  ->  D_ inf `, realized as the **left adjoint** of the
    inclusion:
```

```
`r y =  sqcap  { x  in  D_ inf  : y  sqsub  x }`, the least compatible sequence above `y`. -/
noncomputable def invLimRetr (y : forall  n, D n) : InverseLimit D P :=
  sInf {x : InverseLimit D P | y  <=  x.1}

theorem le_coe_invLimRetr (y : forall  n, D n) : y  <=  (invLimRetr D P y).1 := by
  rw [invLimRetr, coe_sInf]
  refine le_sInf ?_
  rintro _ <x, hx, rfl>
  exact hx

/-- `r  dashv  incl`: the retraction is left adjoint to the inclusion. -/
theorem invLimRetr_galoisConnection :
    GaloisConnection (invLimRetr D P) (Subtype.val : InverseLimit D P  ->  forall  n, D n) :=
    by
  intro y x
  constructor
  * intro h
    exact le_trans (le_coe_invLimRetr D P y) (Subtype.coe_le_coe.mpr h)
  * intro h
    exact sInf_le (show x  in  {x' : InverseLimit D P | y  <=  x'.1} from h)

/-- The retraction preserves directed suprema (a left adjoint preserves all suprema). -/
theorem invLimRetr_preservesDirectedSup :
    PreservesDirectedSup (invLimRetr D P) := by
  intro S _ _
  rw [(invLimRetr_galoisConnection D P).l_sSup, sSup_image]

/-- `r  comp  incl = id`: the retraction fixes `D_ inf `. -/
theorem invLimRetr_incl (x : InverseLimit D P) : invLimRetr D P x.1 = x := by
  refine le_antisymm ?_ ?_
  * exact sInf_le (show x  in  {x' : InverseLimit D P | (x.1 : forall  n, D n)  <=  x'.1} from
    le_refl x.1)
  * refine le_sInf ?_
    intro x' hx'
    exact Subtype.coe_le_coe.mp hx'

/-- `D_ inf ` is a Scott-continuous retract of the product `? D_n`. -/
noncomputable def inverseLimitRetraction :
    IsContinuousLatticeRetraction (InverseLimit D P) (forall  n, D n) where
  incl := <Subtype.val, continuous_of_preservesDirectedSup (incl_preservesDirectedSup D P)>
  retr := <invLimRetr D P, continuous_of_preservesDirectedSup (invLimRetr_preservesDirectedSup
    D P)>
  retr_incl := invLimRetr_incl D P

/-- **Scott 1972, Proposition 4.1.** The inverse limit `D_ inf ` of an ?-system of continuous
    lattices
with projection bonding maps is itself a continuous lattice. The product `? D_n` is a
    continuous
lattice (Prop 2.9a) and `D_ inf ` is a retract of it (`inverseLimitRetraction`), so Prop 2.10a
    applies. -/
theorem proposition_4_1 (hD : forall  n, IsContinuousLattice (D n)) :
    IsContinuousLattice (InverseLimit D P) :=
  proposition_2_10_a (inverseLimitRetraction D P) (proposition_2_9_a D hD)

/-! ### Proposition 4.2: the maps `j_{ inf n}` are projections

We construct Scott's embeddings `i_{n inf } : D_n  ->  D_ inf ` and show that `<i_{n inf }, j_{
     inf n}>` is a
projection, where `j_{ inf n}(x) = x_n`. The component `i_{n inf }(x)_m` climbs the tower of
    embeddings
`i? = (P k).incl` for `m  >=  n` (`embLE`) and descends the tower of projections `j? = (P k).
    retr`
for `m < n` (`projLE`). -/
```

```
/-- Climb the tower of embeddings: for `n <= m`, `embLE h = i_{m-1} comp ... comp i_n :
    D_n -> D_m`. -/
def embLE {n m : Nat} (h : n <= m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : Nat} (h : n <= n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : Nat} (h1 : n <= m) (h2 : n <= m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : Nat} (h1 : n <= m) (h2 : n + 1 <= m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

theorem embLE_mono {n m : Nat} (h : n <= m) {x y : D n} (hxy : x <= y) :
    embLE D P h x <= embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- Descend the tower of projections: for `m <= n`, `projLE h = j_m comp ... comp j_{n-1}
     : D_n -> D?`. -/
def projLE {m n : Nat} (h : m <= n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k -> D m) h
    (fun {k} (g : D k -> D m) (w : D (k + 1)) => g ((P k).retr w)) id x

theorem projLE_self {m : Nat} (h : m <= m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem projLE_succ {m n : Nat} (h1 : m <= n) (h2 : m <= n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k -> D m) h1]

theorem projLE_mono {m n : Nat} (h : m <= n) {x y : D n} (hxy : x <= y) :
    projLE D P h x <= projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [projLE_self, projLE_self]
      exact hxy
  | succ k hk ih =>
      rw [projLE_succ D P hk (Nat.le_succ_of_le hk), projLE_succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- Peeling the last projection: `(P m).retr comp projLE (m+1 <= n) = projLE (m <= n)`.
    -/
theorem projLE_retr {m : Nat} : forall {n : Nat} (h : m + 1 <= n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [projLE_succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [projLE_self]
```

```
  | succ k hk ih =>
      intro x
      rw [projLE_succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        projLE_succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]

/-- Scott's embedding component `i_{n inf }(x)_m`: climb for `m  >=  n`, descend for `m < n`.
    -/
def iComp (n : Nat) (x : D n) (m : Nat) : D m :=
  if h : n  <=  m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : Nat} (h : n  <=  m) (x : D n) : iComp D P n x m = embLE D P h x :=
  dif_pos h

theorem iComp_of_ge {n m : Nat} (h : not  n  <=  m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dif_neg h

theorem iComp_self (n : Nat) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]

/-- The sequence `i_{n inf }(x)` is compatible, hence a genuine point of `D_ inf `. -/
theorem iComp_compatible (n : Nat) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n  <=  m
  * rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  * by_cases h2 : n  <=  m + 1
    * have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE_succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE_self]
    * rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE_retr D P (le_of_lt (not_le.mp h2)) x]

/-- For a compatible sequence `y`, descending from level `n` to `m  <=  n` recovers `y?`. -/
theorem projLE_compatible {m : Nat} (y : InverseLimit D P) :
    forall  {n : Nat} (h : m  <=  n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [projLE_self]
  | succ k hk ih => rw [projLE_succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- For a compatible sequence `y`, climbing `y_n` up to level `m  >=  n` stays below `y?`. -/
theorem embLE_le {n : Nat} (y : InverseLimit D P) {m : Nat} (h : n  <=  m) :
    embLE D P h (y.1 n)  <=  y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
           <=  (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _  <=  y.1 (k + 1) := (P k).incl_retr_le _

/-- `i_{n inf }(y_n)  sqsub  y` coordinatewise (the heart of `incl_retr_le` for `j_{ inf n}`).
    -/
theorem iComp_incl_le (n : Nat) (y : InverseLimit D P) (m : Nat) :
    iComp D P n (y.1 n) m  <=  y.1 m := by
  by_cases h : n  <=  m
  * rw [iComp_of_le D P h]; exact embLE_le D P y h
  * rw [iComp_of_ge D P h]
    exact le_of_eq (projLE_compatible D P y (le_of_lt (not_le.mp h)))
```

```
/-- The tower of embeddings is Scott-continuous (a composite of the `i?`). -/
theorem embLE_preservesDirectedSup {n m : Nat} (h : n  <=  m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y)  comp  embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

/-- The tower of projections is Scott-continuous (a composite of the `j?`). -/
theorem projLE_preservesDirectedSup {m n : Nat} (h : m  <=  n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => projLE_self D P (le_refl m) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk  comp  (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => projLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

theorem iComp_preservesDirectedSup (n m : Nat) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n  <=  m
  * have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  * have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h)) :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE_preservesDirectedSup D P _

theorem iComp_monotone (n m : Nat) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The embedding `i_{n inf } : D_n  ->  D_ inf ` as a bare function into the inverse limit. -/
def embInfFun (n : Nat) (x : D n) : InverseLimit D P := <iComp D P n x, iComp_compatible D P n
    x>

@[simp] theorem embInfFun_coe (n : Nat) (x : D n) : (embInfFun D P n x).1 = iComp D P n x :=
    rfl

theorem embInf_monotone (n : Nat) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [ <-  Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : Nat) : PreservesDirectedSup (embInfFun D P n) := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (*  <=  *) (embInfFun D P n '' S) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <embInfFun D P n c, Set.mem_image_of_mem _ hc,
```

```
      embInf_monotone D P n hac, embInf_monotone D P n hbc>
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

/-- The projection `j_{ inf n} : D_ inf   ->  D_n` as a bare function. -/
def projInfFun (n : Nat) (y : InverseLimit D P) : D n := y.1 n

theorem eval_preservesDirectedSup (n : Nat) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n) '' S)
    := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

/-- The embedding `i_{n inf } : D_n  ->  D_ inf `, Scott-continuous. -/
noncomputable def embInf (n : Nat) : ScottMap (D n) (InverseLimit D P) :=
  <embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D P n)>

/-- The projection `j_{ inf n} : D_ inf   ->  D_n`, Scott-continuous. -/
noncomputable def projInf (n : Nat) : ScottMap (InverseLimit D P) (D n) :=
  <projInfFun D P n, continuous_of_preservesDirectedSup (eval_preservesDirectedSup D P n)>

/-- **Scott 1972, Proposition 4.2.** Each `j_{ inf n} : D_ inf   ->  D_n` is a projection of
    continuous
lattices, with embedding `i_{n inf } = embInf n`. -/
```

Lean 4 source (lines 401–595)

```
noncomputable def proposition_4_2 (n : Nat) :
    IsContinuousLatticeProjection (D n) (InverseLimit D P) where
  incl := embInf D P n
  retr := projInf D P n
  retr_incl x := iComp_self D P n x
  incl_retr_le y := by
    rw [ <-  Subtype.coe_le_coe]
    intro m
    exact iComp_incl_le D P n y m

/-- **Scott 1972, S4 (recursion equation).** `i_{n inf } = i_{(n+1) inf }  comp  i_n`. -/
theorem embInf_succ (n : Nat) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n  <=  m
  * by_cases h2 : n + 1  <=  m
    * rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    * have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE_succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE_self, (P m).retr_incl]
  * have h2 : not  n + 1  <=  m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE_succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl x),
      (P n).retr_incl]
```

```
/-- The family `n  |->  i_{n inf }(x_n)` is monotone (the lub defining `x` is monotone). -/
theorem embInf_le_succ (x : InverseLimit D P) (n : Nat) :
    embInf D P n (x.1 n)  <=  embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [ <-  embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2 n]
          _  <=  x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (*  <=  *) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

/-- **Scott 1972, S4.** Each `x  in  D_ inf ` is the (monotone) lub of its projections:
`x =  bigcup _n i_{n inf }(x_n)`. -/
theorem inverseLimit_eq_iSup (x : InverseLimit D P) :
    x =  bigcup  n, embInf D P n (x.1 n) := by
  have hdir := embInf_family_directed D P x
  have hne : (Set.range (fun n => embInf D P n (x.1 n))).Nonempty := Set.range_nonempty _
  refine le_antisymm ?_ ?_
  * rw [ <-  Subtype.coe_le_coe]
    have hcoe : (( bigcup  n, embInf D P n (x.1 n) : InverseLimit D P)).1
        = sSup (Subtype.val '' Set.range (fun n => embInf D P n (x.1 n))) := by
      rw [ <-  sSup_range]
      exact coe_sSup_of_directed D P _ hne hdir
    rw [hcoe]
    intro m
    have hmem : (embInf D P m (x.1 m)).1
         in  Subtype.val '' Set.range (fun n => embInf D P n (x.1 n)) :=
      <embInf D P m (x.1 m), <m, rfl>, rfl>
    have hle := (le_sSup hmem) m
    rwa [show (embInf D P m (x.1 m)).1 m = x.1 m from iComp_self D P m (x.1 m)] at hle
  * exact iSup_le fun n => by
      rw [ <-  Subtype.coe_le_coe]; intro m; exact iComp_incl_le D P n x m

/-- **Scott 1972, S4 (functional equation, "the remark following 4.2").** The identity of `D_
    inf ` is
the directed lub of the approximating projections `r_n = i_{n inf }  comp  j_{ inf n}`:
`id =  bigcup _n i_{n inf }  comp  j_{ inf n}`. This is the algebraic identity at the heart of
    Theorem 4.4. -/
theorem idInf_eq_iSup :
    (ScottMap.idMap : ScottMap (InverseLimit D P) (InverseLimit D P))
      =  bigcup  n, (embInf D P n).comp (projInf D P n) := by
  apply ScottMap.ext
  intro x
  rw [show ( bigcup  n, (embInf D P n).comp (projInf D P n))
       = sSup (Set.range fun n => (embInf D P n).comp (projInf D P n)) from sSup_range.symm,
    ScottMap.sSup_apply, ScottMap.idMap_apply,  <-  Set.range_comp, sSup_range]
  exact inverseLimit_eq_iSup D P x

/-- **Scott 1972, Lemma 4.5.** A criterion for recognizing projections out of the limit: if a
sequence `u : forall  n, D_{n+1}` satisfies the (shifted) recursion `j_{n+1}(u_{n+2}) = u_{n
    +1}`, then the
monotone limit `u_ inf  =  bigcup _n i_{(n+1) inf }(u_n)` has `j_{ inf (n+1)}(u_ inf ) = u_n`.

Proof: extend `u` to a *compatible* sequence `w` (`w_0 = j_0(u_0)`, `w_{k+1} = u_k`); then `w`
    is a
point of `D_ inf `, and since the family `k  |->  i_{k inf }(w_k)` is monotone, dropping its
    `0`-th term does
not change the lub (`Monotone.iSup_nat_add`), so `u_ inf  =  bigcup ? i_{k inf }(w_k) = w` by `
    inverseLimit_eq_iSup`.
Hence `j_{ inf (n+1)}(u_ inf ) = w_{n+1} = u_n`. -/
theorem lemma_4_5 (u : forall  n, D (n + 1))
```

```
    (hu : forall  n, (P (n + 1)).retr (u (n + 1)) = u n) (n : Nat) :
    ( bigcup  k, embInf D P (k + 1) (u k) : InverseLimit D P).1 (n + 1) = u n := by
  have hw : Compatible D P (fun k => Nat.casesOn k ((P 0).retr (u 0)) (fun m => u m)) := by
    intro k
    cases k with
    | zero => rfl
    | succ m => exact hu m
  set wlim : InverseLimit D P :=
    <fun k => Nat.casesOn k ((P 0).retr (u 0)) (fun m => u m), hw> with hwlim
  have hGmono : Monotone (fun k => embInf D P k (wlim.1 k)) :=
    monotone_nat_of_le_succ (embInf_le_succ D P wlim)
  have hsup : ( bigcup  k, embInf D P (k + 1) (u k) : InverseLimit D P) = wlim := by
    have h1 : ( bigcup  k, embInf D P (k + 1) (u k) : InverseLimit D P)
        =  bigcup  k, embInf D P (k + 1) (wlim.1 (k + 1)) := rfl
    rw [h1, Monotone.iSup_nat_add hGmono 1,  <-  inverseLimit_eq_iSup D P wlim]
  rw [hsup]

/-! ### Corollary 4.3: `D_ inf ` is also the *direct* limit

Given continuous `f_n : D_n  ->  D'` into any complete lattice with `f_n = f_{n+1}  comp  i_n`,
     the map
`f inf (x) =  bigcup _n f_n(x_n)` is the unique continuous mediating map with `f_n = f inf
    comp  i_{n inf }`. -/

variable {D' : Type*} [CompleteLattice D']

/-- The mediating map of Corollary 4.3: `f inf (x) =  bigcup _n f_n(x_n)`. -/
def coconeInf (f : forall  n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
   bigcup  n, f n (x.1 n)

theorem coconeInf_apply (f : forall  n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x =  bigcup  n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...i_n x) = f_n(x)`. -/
theorem coconeInf_climb (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) {n : Nat} (x : D n) :
    forall  {m : Nat} (h : n  <=  m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk),  <-  hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x)  sqsub  f_n(x)`. -/
theorem coconeInf_descend (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) {m : Nat} :
    forall  {n : Nat} (h : m  <=  n) (x : D n), f m (projLE D P h x)  <=  f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE_self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x))  <=  f k ((P k).retr x) := ih ((P k).retr x)
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _  <=  f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f inf   comp  i_{n inf }`. -/
theorem coconeInf_comp_embInf (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) (n : Nat) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  * show ( bigcup  m, f m ((embInf D P n x).1 m))  <=  f n x
    refine iSup_le fun m => ?_
```

```
    show f m (iComp D P n x m)  <=  f n x
    by_cases h : n  <=  m
    * rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    * rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h)) x
  * refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f inf ` is Scott-continuous: `f inf ( sqsup S) =  sqsup  f inf (S)` for directed `S` (each
     `f_n` is continuous and a
double ` bigcup ` over `Nat  x  S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : forall  n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : forall  n, DirectedOn (*  <=  *) ((fun s : InverseLimit D P => s.1 n) '' S) := by
    intro n
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <c.1 n, <c, hc, rfl>, hac n, hbc n>
  have key : forall  n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s.1 n)) ''
     S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      eval_preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show ( bigcup  n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

/-- **Scott 1972, Corollary 4.3.** Within complete lattices, `D_ inf ` is also the *direct*
    limit of
the `D_n` along the embeddings `i_n`: for every compatible cocone `f_n : D_n  ->  D'` (
    continuous, with
`f_n = f_{n+1}  comp  i_n`) there is a **unique** continuous `f inf  : D_ inf   ->  D'` with `
    f_n = f inf   comp  i_{n inf }`,
namely `f inf (x) =  bigcup _n f_n(x_n)`. -/
theorem corollary_4_3 (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) :
    exists ! g : ScottMap (InverseLimit D P) D', forall  n x, f n x = g (embInf D P n x) := by
  refine <<coconeInf D P f,
    continuous_of_preservesDirectedSup (coconeInf_preservesDirectedSup D P f)>, ?_, ?_>
  * exact fun n x => (coconeInf_comp_embInf D P f hf n x).symm
  * intro g hg
    ext x
    -- `g x = f inf  x`, since `x =  bigcup _n i_{n inf }(x_n)` and `g` is continuous on this
    directed family
    calc g x = g ( bigcup  n, embInf D P n (x.1 n)) := by rw [ <-  inverseLimit_eq_iSup D P x]
      _ = g (sSup (Set.range fun n => embInf D P n (x.1 n))) := by rw [sSup_range]
      _ = sSup (g '' Set.range fun n => embInf D P n (x.1 n)) :=
          g.preservesDirectedSup_coe _ (Set.range_nonempty _) (embInf_family_directed D P x)
      _ =  bigcup  n, g (embInf D P n (x.1 n)) := by rw [ <-  Set.range_comp, sSup_range]; rfl
      _ =  bigcup  n, f n (x.1 n) := by simp_rw [ <-  hg]

end InverseLimit

end Scott1972.ContinuousLattice
```

A.11 Scott1972/ContinuousLattice/FunctionSpaceTower.lean

632 lines.

Lean 4 source (lines 1–400)

```
import Scott1972.ContinuousLattice.InverseLimits
```

```
import Mathlib.Order.Hom.Basic

/-!
# The function-space tower and Scott's `D_ inf  ~=  [D_ inf  ->  D_ inf ]` (Scott 1972, S4,
    Theorem 4.4)

Starting from a continuous lattice `D_0` together with a chosen projection `j_0 : [D_0  ->  D_0
    ]  ->  D_0`
(Proposition 3.13 provides one), we build the recursively-defined ?-system

  `D_{n+1} = [D_n  ->  D_n]`,  `j_{n+1} = [j_n  ->  j_n]`  (the function-space functor,
    Proposition 3.7),

and form its inverse limit `D_ inf `. Theorem 4.4 is that `D_ inf ` is *homeomorphic to its own
     function
space* `[D_ inf  ->  D_ inf ]`.
-/

namespace Scott1972.ContinuousLattice

universe u

/-- A complete lattice bundled with its instance, used to define the function-space tower by
recursion on `Nat` (the type at level `n+1` depends on the lattice structure at level `n`). -/
structure CLat : Type (u + 1) where
  carrier : Type u
  [str : CompleteLattice carrier]

attribute [instance] CLat.str

/-- The tower `D_0, [D_0 -> D_0], [[D_0 -> D_0] -> [D_0 -> D_0]], ...` as bundled complete
    lattices. -/
noncomputable def towerCLat (D_0 : CLat.{u}) : Nat  ->  CLat.{u}
  | 0 => D_0
  | (n + 1) => <ScottMap (towerCLat D_0 n).carrier (towerCLat D_0 n).carrier>

/-- The carrier `D_n` of the function-space tower. -/
def towerType (D_0 : CLat.{u}) (n : Nat) : Type u := (towerCLat D_0 n).carrier

noncomputable instance towerCompleteLattice (D_0 : CLat.{u}) (n : Nat) :
    CompleteLattice (towerType D_0 n) := (towerCLat D_0 n).str

@[simp] theorem towerType_zero (D_0 : CLat.{u}) : towerType D_0 0 = D_0.carrier := rfl

/-- `D_{n+1}` is *definitionally* the function space `[D_n  ->  D_n]`. -/
theorem towerType_succ (D_0 : CLat.{u}) (n : Nat) :
    towerType D_0 (n + 1) = ScottMap (towerType D_0 n) (towerType D_0 n) := rfl

/-- View an element of `D_{n+1}` as the Scott map `[D_n  ->  D_n]` it definitionally is. -/
def towerToMap {D_0 : CLat.{u}} {n : Nat} (f : towerType D_0 (n + 1)) :
    ScottMap (towerType D_0 n) (towerType D_0 n) := f

/-- Apply an element of `D_{n+1}` as a function `D_n  ->  D_n` (definitional via `
    towerType_succ`). -/
instance towerCoeFun {D_0 : CLat.{u}} {n : Nat} :
    CoeFun (towerType D_0 (n + 1)) (fun _ => towerType D_0 n  ->  towerType D_0 n) where
  coe f := towerToMap f

@[simp] theorem towerToMap_coe {D_0 : CLat.{u}} {n : Nat} (f : towerType D_0 (n + 1))
    (x : towerType D_0 n) : towerToMap f x = f x := rfl

/-! ### The function-space functor on projections (Proposition 3.7, continuous form)
```

```
`proposition_3_7_projection` builds the embedding/projection pair on function spaces as *plain
functions*; here we upgrade them to genuine Scott maps, so that `[D_n  ->  D_n]` is a
    continuous-lattice
projection of `[D_{n+1}  ->  D_{n+1}]`. The map `f  |->  post  comp  f  comp  pre` (conjugation
    ) is Scott-continuous
because directed suprema of function spaces are computed pointwise. -/

section Conj

open Set

variable {Y Z W : Type u} [CompleteLattice Y] [CompleteLattice Z] [CompleteLattice W]

/-- Conjugation `f  |->  post  comp  f  comp  pre` as a bare function `[Y  ->  Y]  ->  [W  ->
    Z]`. -/
def conjMapFun (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) : ScottMap W Z :=
  post.comp (f.comp pre)

theorem conjMapFun_apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x : W)
    :
    conjMapFun post pre f x = post (f (pre x)) := rfl

theorem conjMap_preservesDirectedSup (post : ScottMap Y Z) (pre : ScottMap W Y) :
    PreservesDirectedSup (conjMapFun post pre) := by
  intro F hF hFdir
  apply ScottMap.ext
  intro x
  have hdir : DirectedOn (*  <=  *) ((fun f : ScottMap Y Y => f (pre x)) '' F) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hFdir a ha b hb
    exact <c (pre x), <c, hc, rfl>, hac (pre x), hbc (pre x)>
  show post ((sSup F : ScottMap Y Y) (pre x)) = (sSup (conjMapFun post pre '' F) : ScottMap W Z
    ) x
  rw [ScottMap.sSup_apply F (pre x), post.preservesDirectedSup_coe _ (hF.image _) hdir,
    ScottMap.sSup_apply, Set.image_image, Set.image_image]
  rfl

theorem conjMap_preservesDirectedSup_apply (post : ScottMap Y Z) (pre : ScottMap W Y)
    (F : Set (ScottMap Y Y)) (hF : F.Nonempty) (hFdir : DirectedOn (*  <=  *) F) (y : W) :
    post ((sSup F : ScottMap Y Y) (pre y)) = (sSup (conjMapFun post pre '' F) : ScottMap W Z) y
     := by
  have hdir : DirectedOn (*  <=  *) ((fun f : ScottMap Y Y => f (pre y)) '' F) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hFdir a ha b hb
    exact <c (pre y), <c, hc, rfl>, hac (pre y), hbc (pre y)>
  rw [ScottMap.sSup_apply F (pre y), post.preservesDirectedSup_coe _ (hF.image _) hdir,
    ScottMap.sSup_apply, Set.image_image, Set.image_image]
  rfl

/-- Conjugation `f  |->  post  comp  f  comp  pre` as a Scott map `[Y  ->  Y]  ->  [W  ->  Z]`.
     -/
noncomputable def conjMap (post : ScottMap Y Z) (pre : ScottMap W Y) :
    ScottMap (ScottMap Y Y) (ScottMap W Z) :=
  <conjMapFun post pre, continuous_of_preservesDirectedSup (conjMap_preservesDirectedSup post
    pre)>

@[simp] theorem conjMap_apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x
    : W) :
    conjMap post pre f x = post (f (pre x)) := rfl

end Conj

/-- **Scott 1972, Proposition 3.7 (continuous projection on the diagonal).** If `D` is a
```

```
continuous-lattice projection of `D'`, then `[D  ->  D]` is a projection of `[D'  ->  D']` via
`i_{[*]}(f) = i  comp  f  comp  j` and `j_{[*]}(g) = j  comp  g  comp  i`. -/
noncomputable def IsContinuousLatticeProjection.functionSpace
    {A B : Type u} [CompleteLattice A] [CompleteLattice B]
    (P : IsContinuousLatticeProjection A B) :
    IsContinuousLatticeProjection (ScottMap A A) (ScottMap B B) where
  incl := conjMap P.incl P.retr
  retr := conjMap P.retr P.incl
  retr_incl f := by
    apply ScottMap.ext; intro x
    simp only [conjMap_apply, P.retr_incl]
  incl_retr_le g := by
    rw [ScottMap.le_def]; intro x
    simp only [conjMap_apply]
    exact le_trans (P.incl_retr_le _) (g.monotone (P.incl_retr_le x))

/-- The projection tower `j_{n+1} = [j_n  ->  j_n]`, anchored at a chosen base projection
`j_0 : [D_0  ->  D_0]  ->  D_0`. -/
noncomputable def towerProj (D_0 : CLat.{u})
    (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier)) :
    forall  n, IsContinuousLatticeProjection (towerType D_0 n) (towerType D_0 (n + 1))
  | 0 => j_0
  | (n + 1) => (towerProj D_0 j_0 n).functionSpace

theorem towerProj_succ (D_0 : CLat.{u})
    (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier)) (n :
    Nat) :
    towerProj D_0 j_0 (n + 1) = (towerProj D_0 j_0 n).functionSpace := rfl

section Tower

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Scott 1972, S4 (recursion for the embeddings).** `i_{n+1}(x) = i_n  comp  x  comp  j_n`.
    -/
theorem towerProj_succ_incl_apply (n : Nat) (x : towerType D_0 (n + 1)) (y : towerType D_0 (n +
     1)) :
    ((towerProj D_0 j_0 (n + 1)).incl x) y
      = (towerProj D_0 j_0 n).incl (x ((towerProj D_0 j_0 n).retr y)) := rfl

/-- **Scott 1972, S4 (recursion for the projections).** `j_{n+1}(x') = j_n  comp  x'  comp  i_n
    `. -/
theorem towerProj_succ_retr_apply (n : Nat) (x' : towerType D_0 (n + 2)) (y : towerType D_0 n)
    :
    ((towerProj D_0 j_0 (n + 1)).retr x') y
      = (towerProj D_0 j_0 n).retr (x' ((towerProj D_0 j_0 n).incl y)) := rfl

/-- **Scott 1972, S4 (application is preserved).** `i_n(f(x)) = i_{n+1}(f)(i_n(x))` for `f  in
     D_{n+1}`,
`x  in  D_n`. The embeddings turn functional application into application one level up. -/
theorem towerProj_incl_apply (n : Nat) (f : towerType D_0 (n + 1)) (x : towerType D_0 n) :
    (towerProj D_0 j_0 n).incl (f x)
      = ((towerProj D_0 j_0 (n + 1)).incl f) ((towerProj D_0 j_0 n).incl x) := by
  rw [towerProj_succ_incl_apply, (towerProj D_0 j_0 n).retr_incl]

end Tower

/-! ### Theorem 4.4(a): the limit maps `i_ inf ` and `j_ inf `

We now wire the concrete inverse limit `D_ inf ` of the function-space tower and write down
    Scott's pair
```

```
'''
i_ inf (x) =  bigcup _n (i_{n inf }  comp  x_{n+1}  comp  j_{ inf n})      : D_ inf   ->  [D_
    inf   ->  D_ inf ]
j_ inf (f) =  bigcup _n i_{(n+1) inf }(j_{ inf n}  comp  f  comp  i_{n inf })  : [D_ inf   ->
    D_ inf ]  ->  D_ inf
'''

Each summand is itself a Scott map (a composite of `conjMap`, `embInf`, `projInf`), so each of
`i_ inf `, `j_ inf ` is a *supremum of Scott maps* and is therefore Scott-continuous
    automatically: by
Theorem 3.3 the function space `[A  ->  B]` is a complete lattice in which suprema are computed
pointwise. No bespoke continuity argument is needed. -/

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The inverse limit `D_ inf ` of the function-space tower `<D_n, j_n>`. -/
abbrev DInf : Type u := InverseLimit (towerType D_0) (towerProj D_0 j_0)

/-- The function space `[D_ inf   ->  D_ inf ]`. -/
abbrev DInfFn : Type u := ScottMap (DInf D_0 j_0) (DInf D_0 j_0)

/-- The `n`-th summand of `i_ inf `: the Scott map `x  |->  i_{n inf }  comp  x_{n+1}  comp  j_
    { inf n}`, where `x_{n+1}` is
the `(n+1)`-st component of `x  in  D_ inf `. As a map `D_ inf   ->  [D_ inf   ->  D_ inf ]` it
     is the composite of the
component projection `j_{ inf (n+1)}` with conjugation by `(i_{n inf }, j_{ inf n})`. -/
noncomputable def iInfTerm (n : Nat) : ScottMap (DInf D_0 j_0) (DInfFn D_0 j_0) :=
  (conjMap (embInf (towerType D_0) (towerProj D_0 j_0) n)
           (projInf (towerType D_0) (towerProj D_0 j_0) n)).comp
    (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1))

@[simp] theorem iInfTerm_apply (n : Nat) (x z : DInf D_0 j_0) :
    (iInfTerm D_0 j_0 n x) z
      = embInf (towerType D_0) (towerProj D_0 j_0) n
          ((x.1 (n + 1)) ((projInf (towerType D_0) (towerProj D_0 j_0) n) z)) := rfl

/-- **Scott 1972, S4 (Theorem 4.4).** The embedding `i_ inf  : D_ inf   ->  [D_ inf   ->  D_
    inf ]`,
`i_ inf (x) =  bigcup _n i_{n inf }  comp  x_{n+1}  comp  j_{ inf n}`. It is Scott-continuous
    because it is a supremum of the
Scott maps `iInfTerm n` (suprema in `[D_ inf   ->  [D_ inf   ->  D_ inf ]]` are computed
    pointwise). -/
noncomputable def embInfInf : ScottMap (DInf D_0 j_0) (DInfFn D_0 j_0) :=
   bigcup  n, iInfTerm D_0 j_0 n

/-- `i_ inf ` evaluated at `x` is the pointwise supremum of the summands `iInfTerm n x`. -/
theorem embInfInf_apply (x : DInf D_0 j_0) :
    embInfInf D_0 j_0 x =  bigcup  n, iInfTerm D_0 j_0 n x := by
  show (sSup (Set.range (iInfTerm D_0 j_0)) : ScottMap _ _) x = _
  rw [ScottMap.sSup_apply,  <-  Set.range_comp, sSup_range]
  rfl

/-- The `n`-th summand of `j_ inf `: the Scott map `f  |->  i_{(n+1) inf }(j_{ inf n}  comp  f
     comp  i_{n inf })`. As a map
`[D_ inf   ->  D_ inf ]  ->  D_ inf ` it is conjugation by `(j_{ inf n}, i_{n inf })` (landing
    in `D_{n+1}`) followed by the
embedding `i_{(n+1) inf }`. -/
noncomputable def jInfTerm (n : Nat) : ScottMap (DInfFn D_0 j_0) (DInf D_0 j_0) :=
  (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)).comp
    (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
```

```
            (embInf (towerType D_0) (towerProj D_0 j_0) n))

@[simp] theorem jInfTerm_apply (n : Nat) (f : DInfFn D_0 j_0) :
    jInfTerm D_0 j_0 n f
      = embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
          (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                   (embInf (towerType D_0) (towerProj D_0 j_0) n) f) := rfl

/-- **Scott 1972, S4 (Theorem 4.4).** The projection `j_ inf  : [D_ inf   ->  D_ inf ]  ->  D_
    inf `,
`j_ inf (f) =  bigcup _n i_{(n+1) inf }(j_{ inf n}  comp  f  comp  i_{n inf })`. Scott-
    continuous as a supremum of the Scott maps
`jInfTerm n`. -/
noncomputable def projInfInf : ScottMap (DInfFn D_0 j_0) (DInf D_0 j_0) :=
   bigcup  n, jInfTerm D_0 j_0 n

/-- `j_ inf ` evaluated at `f` is the supremum of the summands `jInfTerm n f`. -/
theorem projInfInf_apply (f : DInfFn D_0 j_0) :
    projInfInf D_0 j_0 f =  bigcup  n, jInfTerm D_0 j_0 n f := by
  show (sSup (Set.range (jInfTerm D_0 j_0)) : ScottMap _ _) f = _
  rw [ScottMap.sSup_apply,  <-  Set.range_comp, sSup_range]
  rfl

/-- `i_ inf ` is Scott-continuous (it is a bundled `ScottMap`). -/
theorem embInfInf_preservesDirectedSup :
    PreservesDirectedSup (embInfInf D_0 j_0 : DInf D_0 j_0  ->  DInfFn D_0 j_0) :=
  (proposition_2_5 _).mp (embInfInf D_0 j_0).continuous

/-- `j_ inf ` is Scott-continuous (it is a bundled `ScottMap`). -/
theorem projInfInf_preservesDirectedSup :
    PreservesDirectedSup (projInfInf D_0 j_0 : DInfFn D_0 j_0  ->  DInf D_0 j_0) :=
  (proposition_2_5 _).mp (projInfInf D_0 j_0).continuous

/-! ### Theorem 4.4(b): `j_ inf   comp  i_ inf  = id` on `D_ inf ` 

Scott's calculation (~lines 1290-1294): expand `j_ inf (i_ inf (x))` as a double sup, collapse
    the monotone
double limit to the diagonal using `projInf  comp  embInf = id` on each summand, then recognize
     `x` via
`inverseLimit_eq_iSup` (with a one-step index shift). -/

section Thm44b

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- Conjugation commutes with a supremum indexed by `Nat` (the range is directed when `f` is
    monotone).
Since `conjMap post pre` is itself a Scott map, this is just preservation of directed suprema.
    -/
theorem conjMap_iSup (n : Nat) (post : ScottMap (DInf D_0 j_0) (towerType D_0 n))
    (pre : ScottMap (towerType D_0 n) (DInf D_0 j_0))
    (f : Nat  ->  ScottMap (DInf D_0 j_0) (DInf D_0 j_0)) (hf : Monotone f) :
    conjMap post pre ( bigcup  m, f m) =  bigcup  m, conjMap post pre (f m) := by
  have hdir : DirectedOn (*  <=  *) (Set.range f) :=
    directedOn_range.2 fun a b => <max a b, hf (le_max_left a b), hf (le_max_right a b)>
  have hne := Set.range_nonempty f
  rw [show ( bigcup  m, f m) = sSup (Set.range f) from sSup_range.symm,
    (conjMap post pre).preservesDirectedSup_coe (Set.range f) hne hdir,  <-  Set.range_comp]
  rfl

/-- The inverse-limit embedding commutes with a supremum in `D_{n+1}` (monotone  =>  directed).
     -/
```

```
theorem embInf_succ_iSup (n : Nat) (f : Nat  ->  towerType D_0 (n + 1)) (hf : Monotone f) :
    embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) ( bigcup  m, f m) =
       bigcup  m, embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) (f m) := by
  have hdir : DirectedOn (*  <=  *) (Set.range f) :=
    directedOn_range.2 fun a b => <max a b, hf (le_max_left a b), hf (le_max_right a b)>
  have hne := Set.range_nonempty f
  rw [show ( bigcup  m, f m) = sSup (Set.range f) from sSup_range.symm,
    (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)).preservesDirectedSup_coe
      (Set.range f) hne hdir,  <-  Set.range_comp]
  rfl

/-- **Diagonal simplification.** `conjMap (j_{ inf n}, i_{n inf })` applied to the `n`-th
    summand of `i_ inf `
recovers the component `x_{n+1}`. This is exactly `j_{[*]}  comp  i_{[*]} = id` for the
    function-space
projection built from `proposition_4_2`. -/
theorem conj_iInfTerm_eq (n : Nat) (x : DInf D_0 j_0) :
    conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
            (embInf (towerType D_0) (towerProj D_0 j_0) n)
            (iInfTerm D_0 j_0 n x) = x.1 (n + 1) :=
  (proposition_4_2 (towerType D_0) (towerProj D_0 j_0) n).functionSpace.retr_incl
    (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) x)

/-- `i_{n inf }(y_n)  sqsub  y_{n+1}`: climbing one level and embedding stays below the next
    component. -/
theorem incl_projInf_le_projInf_succ (n : Nat) (w : DInf D_0 j_0) :
    (towerProj D_0 j_0 n).incl (projInf (towerType D_0) (towerProj D_0 j_0) n w)
       <=  projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) w := by
  have h := (towerProj D_0 j_0 n).incl_retr_le
    (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) w)
  rwa [show (towerProj D_0 j_0 n).retr (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) w)
      = projInf (towerType D_0) (towerProj D_0 j_0) n w from w.2 n] at h

/-- `i_ inf ` is monotone in its summand index. -/
theorem iInfTerm_le_succ (x : DInf D_0 j_0) (m : Nat) :
    iInfTerm D_0 j_0 m x  <=  iInfTerm D_0 j_0 (m + 1) x := by
  rw [ScottMap.le_def]
  intro z
  rw [iInfTerm_apply, iInfTerm_apply,
     <-  embInf_succ (towerType D_0) (towerProj D_0 j_0) m
      (x.1 (m + 1) (projInf (towerType D_0) (towerProj D_0 j_0) m z))]
  refine embInf_monotone (towerType D_0) (towerProj D_0 j_0) (m + 1) ?_
  have hab : (towerProj D_0 j_0 m).incl (projInf (towerType D_0) (towerProj D_0 j_0) m z)
       <=  projInf (towerType D_0) (towerProj D_0 j_0) (m + 1) z :=
    incl_projInf_le_projInf_succ D_0 j_0 m z
  calc (towerProj D_0 j_0 m).incl
          (x.1 (m + 1) (projInf (towerType D_0) (towerProj D_0 j_0) m z))
      = (towerProj D_0 j_0 m).incl
          (((towerProj D_0 j_0 (m + 1)).retr (x.1 (m + 2)))
            (projInf (towerType D_0) (towerProj D_0 j_0) m z)) := by
        rw [show ((towerProj D_0 j_0 (m + 1)).retr (x.1 (m + 2))) = x.1 (m + 1) from x.2 (m +
    1)]
    _ = (towerProj D_0 j_0 m).incl
          ((towerProj D_0 j_0 m).retr
            (x.1 (m + 2) ((towerProj D_0 j_0 m).incl
              (projInf (towerType D_0) (towerProj D_0 j_0) m z)))) := by
        rw [towerProj_succ_retr_apply]
    _  <=  x.1 (m + 2) ((towerProj D_0 j_0 m).incl
          (projInf (towerType D_0) (towerProj D_0 j_0) m z)) :=
        (towerProj D_0 j_0 m).incl_retr_le _
    _  <=  x.1 (m + 2) (projInf (towerType D_0) (towerProj D_0 j_0) (m + 1) z) :=
        ScottMap.monotone (x.1 (m + 2)) hab
```

```
theorem iInfTerm_monotone (x : DInf D_0 j_0) : Monotone (fun m => iInfTerm D_0 j_0 m x) :=
  monotone_nat_of_le_succ (iInfTerm_le_succ D_0 j_0 x)

/-- A monotone double `iSup` over `Nat  x  Nat` equals the diagonal `iSup`. -/
theorem iSup_2_monotone_eq_diagonal {a : Type*} [CompleteLattice a] (f : Nat  ->  Nat  ->  a)
    (hfm : forall  n, Monotone (f n)) (hfn : forall  m, Monotone (fun n => f n m)) :
     bigcup  n,  bigcup  m, f n m =  bigcup  n, f n n := by
  apply le_antisymm
  * refine iSup_le fun n => iSup_le fun m => ?_
    have hk : n  <=  n  sqsup  m := le_sup_left
    have hk' : m  <=  n  sqsup  m := le_sup_right
    calc f n m  <=  f (n  sqsup  m) m := hfn m hk
      _  <=  f (n  sqsup  m) (n  sqsup  m) := hfm (n  sqsup  m) hk'
      _  <=   bigcup  n', f n' n' := le_iSup (fun n' => f n' n') (n  sqsup  m)
  * refine iSup_le fun n => le_trans (le_iSup (f n) n) (le_iSup (fun n' =>  bigcup  m, f n' m)
    n)

/-- **Conjugation climbs the tower (one step).** Lifting `conjMap (j_{ inf n}, i_{n inf }) f`
    along `i_{n+1}`
stays below `conjMap (j_{ inf (n+1)}, i_{(n+1) inf }) f`. Both sides live in `D_{n+2}`; the
    inequality is the
algebraic content that makes the double sup defining `j_ inf   comp  i_ inf ` monotone in the
    level index `n`. -/
theorem conjMap_incl_le_conjMap_succ (n : Nat) (f : DInfFn D_0 j_0) :
    (towerProj D_0 j_0 (n + 1)).incl
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                 (embInf (towerType D_0) (towerProj D_0 j_0) n) f)
       <=  conjMap (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1))
                (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)) f := by
  refine ScottMap.le_def.mpr fun y => ?_
  calc (towerProj D_0 j_0 n).incl
         (projInf (towerType D_0) (towerProj D_0 j_0) n
           (f (embInf (towerType D_0) (towerProj D_0 j_0) n ((towerProj D_0 j_0 n).retr y))))
       <=  (towerProj D_0 j_0 n).incl
         (projInf (towerType D_0) (towerProj D_0 j_0) n
           (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) y))) := by
       refine (towerProj D_0 j_0 n).incl.monotone
         ((projInf (towerType D_0) (towerProj D_0 j_0) n).monotone (f.monotone ?_))
       rw [ <-  embInf_succ (towerType D_0) (towerProj D_0 j_0) n ((towerProj D_0 j_0 n).retr
    y)]
       exact embInf_monotone (towerType D_0) (towerProj D_0 j_0) (n + 1)
         ((towerProj D_0 j_0 n).incl_retr_le y)
    _  <=  projInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
         (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) y)) :=
       incl_projInf_le_projInf_succ D_0 j_0 n
         (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) y))

/-- `j_ inf (i_ inf (x)) =  bigcup _n i_{(n+1) inf }(x_{n+1})`. -/
theorem projInfInf_embInfInf_eq (x : DInf D_0 j_0) :
    projInfInf D_0 j_0 (embInfInf D_0 j_0 x) =  bigcup  n, embInf (towerType D_0) (towerProj
    D_0 j_0) (n + 1) (x.1 (n + 1)) := by
  rw [projInfInf_apply]
  set g := fun n m =>
    embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
      (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
               (embInf (towerType D_0) (towerProj D_0 j_0) n)
               (iInfTerm D_0 j_0 m x)) with hg
  have hmono (n : Nat) : Monotone (fun m =>
      conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
              (embInf (towerType D_0) (towerProj D_0 j_0) n)
              (iInfTerm D_0 j_0 m x)) := fun a b hab =>
    (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
             (embInf (towerType D_0) (towerProj D_0 j_0) n)).monotone
```

```
      (ScottMap.le_def.mpr (iInfTerm_monotone D_0 j_0 x hab))
  have hinner (n : Nat) : jInfTerm D_0 j_0 n (embInfInf D_0 j_0 x) =  bigcup  m, g n m :=
    calc jInfTerm D_0 j_0 n (embInfInf D_0 j_0 x)
        = embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
            ( bigcup  m, conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                         (embInf (towerType D_0) (towerProj D_0 j_0) n)
                         (iInfTerm D_0 j_0 m x)) := by
```

Lean 4 source (lines 401–632)

```
          rw [jInfTerm_apply, embInfInf_apply,
            conjMap_iSup D_0 j_0 n (projInf (towerType D_0) (towerProj D_0 j_0) n)
              (embInf (towerType D_0) (towerProj D_0 j_0) n)
              (fun m => iInfTerm D_0 j_0 m x) (iInfTerm_monotone D_0 j_0 x)]
          rfl
      _ =  bigcup  m, embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
             (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                      (embInf (towerType D_0) (towerProj D_0 j_0) n)
                      (iInfTerm D_0 j_0 m x)) :=
          embInf_succ_iSup D_0 j_0 n _ (hmono n)
      _ =  bigcup  m, g n m := by simp only [hg]
  have g_mono_m (n : Nat) : Monotone (g n) := by
    intro a b hab
    rw [hg]
    exact (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)).monotone
      ((conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                (embInf (towerType D_0) (towerProj D_0 j_0) n)).monotone
        (ScottMap.le_def.mpr (iInfTerm_monotone D_0 j_0 x hab)))
  have g_mono_n_succ (m n : Nat) : g n m  <=  g (n + 1) m := by
    rw [hg]
    dsimp only
    rw [ <-  embInf_succ (towerType D_0) (towerProj D_0 j_0) (n + 1)
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                 (embInf (towerType D_0) (towerProj D_0 j_0) n)
                 (iInfTerm D_0 j_0 m x))]
    exact embInf_monotone (towerType D_0) (towerProj D_0 j_0) (n + 2)
      (conjMap_incl_le_conjMap_succ D_0 j_0 n (iInfTerm D_0 j_0 m x))
  have g_mono_n (m : Nat) : Monotone (fun n => g n m) :=
    monotone_nat_of_le_succ (g_mono_n_succ m)
  have hin : ( bigcup  n, jInfTerm D_0 j_0 n (embInfInf D_0 j_0 x)) =  bigcup  n,  bigcup  m, g
     n m := by
    congr 1
    funext n
    exact hinner n
  rw [hin, iSup_2_monotone_eq_diagonal g g_mono_m g_mono_n]
  congr 1
  funext n
  rw [hg]
  dsimp only
  rw [conj_iInfTerm_eq D_0 j_0 n x]

/-- **Scott 1972, S4 (Theorem 4.4, first half).** `j_ inf   comp  i_ inf  = id` on `D_ inf `.
    -/
theorem projInfInf_comp_embInfInf :
    (projInfInf D_0 j_0).comp (embInfInf D_0 j_0) = ScottMap.idMap := by
  apply ScottMap.ext
  intro x
  have hmono : Monotone (fun k => embInf (towerType D_0) (towerProj D_0 j_0) k (x.1 k)) :=
    monotone_nat_of_le_succ (embInf_le_succ (towerType D_0) (towerProj D_0 j_0) x)
  rw [ScottMap.comp_apply, ScottMap.idMap_apply, projInfInf_embInfInf_eq D_0 j_0 x,
    Monotone.iSup_nat_add hmono 1]
  exact (inverseLimit_eq_iSup (towerType D_0) (towerProj D_0 j_0) x).symm

/-! ### Theorem 4.4(c): `i_ inf   comp  j_ inf  = id` on `[D_ inf   ->  D_ inf ]`
```

```
This is the converse half (Scott ~lines 1322-1335). The restrictions
`u_n = j_{ inf n}  comp  f  comp  i_{n inf }  in  D_{n+1}` satisfy the Lemma-4.5 recursion `j_{
    n+1}(u_{n+2}) = u_{n+1}`
(`towerProj_retr_conjMap_succ`), so Lemma 4.5 identifies the components of `j_ inf (f)`.
    Expanding
`i_ inf (j_ inf (f))` then yields the approximation ` bigcup _n r_n  comp  f  comp  r_n` with `
    r_n = i_{n inf }  comp  j_{ inf n}`, and the
functional equation `id =  bigcup _n r_n` (via `inverseLimit_eq_iSup`) plus continuity of `f`
    collapse this
to `f`. -/

/-- **Step 1 (Lemma-4.5 recursion).** `j_{n+1}(j_{ inf (n+1)}  comp  f  comp  i_{(n+1) inf }) =
     j_{ inf n}  comp  f  comp  i_{n inf }`.
This is the equality counterpart of `conjMap_incl_le_conjMap_succ`; it is the hypothesis Lemma
    4.5
needs to recognize `j_ inf (f)` from its restrictions. -/
theorem towerProj_retr_conjMap_succ (n : Nat) (f : DInfFn D_0 j_0) :
    (towerProj D_0 j_0 (n + 1)).retr
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1))
                 (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)) f)
      = conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                (embInf (towerType D_0) (towerProj D_0 j_0) n) f := by
  apply ScottMap.ext
  intro y
  show (towerProj D_0 j_0 n).retr
      (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
        (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
          ((towerProj D_0 j_0 n).incl y))))
    = projInf (towerType D_0) (towerProj D_0 j_0) n
        (f (embInf (towerType D_0) (towerProj D_0 j_0) n y))
  rw [embInf_succ (towerType D_0) (towerProj D_0 j_0) n y]
  exact (f (embInf (towerType D_0) (towerProj D_0 j_0) n y)).2 n

/-- **Scott 1972, S4 (Theorem 4.4, second half).** `i_ inf   comp  j_ inf  = id` on `[D_ inf
    ->  D_ inf ]`. -/
theorem embInfInf_comp_projInfInf :
    (embInfInf D_0 j_0).comp (projInfInf D_0 j_0) = ScottMap.idMap := by
  apply ScottMap.ext
  intro f
  rw [ScottMap.comp_apply, ScottMap.idMap_apply]
  apply ScottMap.ext
  intro z
  -- `r_n = i_{n inf }  comp  j_{ inf n}`, the Proposition-4.2 approximation of the identity on
     `D_ inf `.
  set r : Nat  ->  ScottMap (DInf D_0 j_0) (DInf D_0 j_0) :=
    fun n => (embInf (towerType D_0) (towerProj D_0 j_0) n).comp
               (projInf (towerType D_0) (towerProj D_0 j_0) n) with hr
  have hrw : forall  n (w : DInf D_0 j_0), r n w
      = embInf (towerType D_0) (towerProj D_0 j_0) n
          (projInf (towerType D_0) (towerProj D_0 j_0) n w) := by
    intro n w
    simp only [hr, ScottMap.comp_apply]
  have hr_mono : forall  (w : DInf D_0 j_0), Monotone (fun m => r m w) := by
    intro w
    refine monotone_nat_of_le_succ (fun m => ?_)
    show r m w  <=  r (m + 1) w
    rw [hrw, hrw]
    exact embInf_le_succ (towerType D_0) (towerProj D_0 j_0) w m
  -- The functional equation `id =  bigcup _n r_n` (remark following Proposition 4.2),
    pointwise.
  have hA : forall  (w : DInf D_0 j_0), w =  bigcup  m, r m w := by
    intro w
```

```
  have h1 : ( bigcup  m, r m w)
      =  bigcup  m, embInf (towerType D_0) (towerProj D_0 j_0) m
              (projInf (towerType D_0) (towerProj D_0 j_0) m w) :=
    iSup_congr (fun m => hrw m w)
  rw [h1]
  exact inverseLimit_eq_iSup (towerType D_0) (towerProj D_0 j_0) w
-- Evaluating a supremum of Scott maps `D_ inf   ->  D_ inf ` at a point is pointwise.
have hsup_apply : forall  (g : Nat  ->  ScottMap (DInf D_0 j_0) (DInf D_0 j_0)) (w : DInf D_0
   j_0),
    ( bigcup  n, g n) w =  bigcup  n, g n w := by
  intro g w
  rw [show ( bigcup  n, g n) = sSup (Set.range g) from sSup_range.symm,
    ScottMap.sSup_apply,  <-  Set.range_comp, sSup_range]
  rfl
-- A Scott map commutes with a monotone `Nat`-indexed supremum.
have hcont : forall  (g : ScottMap (DInf D_0 j_0) (DInf D_0 j_0)) (a : Nat  ->  DInf D_0 j_0)
  ,
    Monotone a  ->  g ( bigcup  m, a m) =  bigcup  m, g (a m) := by
  intro g a ha
  have hdir : DirectedOn (*  <=  *) (Set.range a) :=
    directedOn_range.2 fun i j => <max i j, ha (le_max_left i j), ha (le_max_right i j)>
  rw [show ( bigcup  m, a m) = sSup (Set.range a) from sSup_range.symm,
    g.preservesDirectedSup_coe (Set.range a) (Set.range_nonempty a) hdir,
     <-  Set.range_comp, sSup_range]
  rfl
-- `j_ inf (f) =  bigcup ? i_{(k+1) inf }(j_{ inf k}  comp  f  comp  i_{k inf })`.
have hpi : projInfInf D_0 j_0 f
    =  bigcup  k, embInf (towerType D_0) (towerProj D_0 j_0) (k + 1)
       (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) k)
                (embInf (towerType D_0) (towerProj D_0 j_0) k) f) := by
  rw [projInfInf_apply]
  exact iSup_congr (fun n => jInfTerm_apply D_0 j_0 n f)
-- Lemma 4.5: the `(n+1)`-st component of `j_ inf (f)` is the restriction `j_{ inf n}  comp
  f  comp  i_{n inf }`.
have hcoord : forall  n, (projInfInf D_0 j_0 f).1 (n + 1)
    = conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
            (embInf (towerType D_0) (towerProj D_0 j_0) n) f := by
  intro n
  rw [hpi]
  exact lemma_4_5 (towerType D_0) (towerProj D_0 j_0)
    (fun k => conjMap (projInf (towerType D_0) (towerProj D_0 j_0) k)
                      (embInf (towerType D_0) (towerProj D_0 j_0) k) f)
    (fun m => towerProj_retr_conjMap_succ D_0 j_0 m f) n
-- Evaluate `i_ inf (j_ inf (f))` pointwise as a sup of summands.
have hev : embInfInf D_0 j_0 (projInfInf D_0 j_0 f) z
    =  bigcup  n, (iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z := by
  rw [embInfInf_apply]
  exact hsup_apply (fun n => iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z
-- Each summand is `r_n (f (r_n z))` (using `hcoord` and `conjMap`).
have hterm : forall  n, (iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z = r n (f (r n z)) := by
  intro n
  rw [hrw, hrw, iInfTerm_apply, hcoord n]
  rfl
have hmono_frz : Monotone (fun m => f (r m z)) :=
  fun a b hab => f.monotone (hr_mono z hab)
have hfm : forall  n, Monotone (fun m => r n (f (r m z))) :=
  fun n _ _ hab => (r n).monotone (f.monotone (hr_mono z hab))
have hfn : forall  m, Monotone (fun n => r n (f (r m z))) :=
  fun m => hr_mono (f (r m z))
-- The analytic step (Scott ~1326-1334): confine the lub via continuity of `f`, then collapse
   the
-- monotone double sup to its diagonal.
have hfz : f z =  bigcup  n, r n (f (r n z)) :=
```

```
    calc f z =  bigcup  k, r k (f z) := hA (f z)
      _ =  bigcup  k, r k (f ( bigcup  m, r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [ <-  hA z]
      _ =  bigcup  k, r k ( bigcup  m, f (r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [hcont f (fun m => r m z) (hr_mono z)]
      _ =  bigcup  k,  bigcup  m, r k (f (r m z)) :=
            iSup_congr (fun k => hcont (r k) (fun m => f (r m z)) hmono_frz)
      _ =  bigcup  n, r n (f (r n z)) :=
            iSup_2_monotone_eq_diagonal (fun n m => r n (f (r m z))) hfm hfn
  calc embInfInf D_0 j_0 (projInfInf D_0 j_0 f) z
      =  bigcup  n, (iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z := hev
    _ =  bigcup  n, r n (f (r n z)) := iSup_congr hterm
    _ = f z := hfz.symm

end Thm44b

/-! ### Theorem 4.4(d): capstone `D_ inf   ~=  [D_ inf   ->  D_ inf ]`

Package the mutually-inverse Scott maps from (b) and (c). Scott's homeomorphism follows because
`i_ inf ` and `j_ inf ` are Scott-continuous (`embInfInf` / `projInfInf` are bundled `ScottMap`
    s). -/

section Thm44d

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

theorem projInfInf_embInfInf (x : DInf D_0 j_0) :
    projInfInf D_0 j_0 (embInfInf D_0 j_0 x) = x := by
  have h := congrArg (fun g => g x) (projInfInf_comp_embInfInf D_0 j_0)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

theorem embInfInf_projInfInf (f : DInfFn D_0 j_0) :
    embInfInf D_0 j_0 (projInfInf D_0 j_0 f) = f := by
  have h := congrArg (fun g => g f) (embInfInf_comp_projInfInf D_0 j_0)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

theorem embInfInf_le_iff (x y : DInf D_0 j_0) :
    embInfInf D_0 j_0 x  <=  embInfInf D_0 j_0 y  <->  x  <=  y := by
  constructor
  * intro h
    have := (projInfInf D_0 j_0).monotone h
    rwa [projInfInf_embInfInf, projInfInf_embInfInf] at this
  * intro h; exact (embInfInf D_0 j_0).monotone h

/-- **Scott 1972, S4 (Theorem 4.4).** The inverse limit `D_ inf ` of the function-space tower
    is
order-isomorphic to its own function space `[D_ inf   ->  D_ inf ]` via the mutually-inverse
    Scott maps
`i_ inf  = embInfInf` and `j_ inf  = projInfInf`. -/
theorem theorem_4_4 :
    (projInfInf D_0 j_0).comp (embInfInf D_0 j_0) = ScottMap.idMap  /\
    (embInfInf D_0 j_0).comp (projInfInf D_0 j_0) = ScottMap.idMap :=
  <projInfInf_comp_embInfInf D_0 j_0, embInfInf_comp_projInfInf D_0 j_0>

/-- The order isomorphism `D_ inf  ?o [D_ inf   ->  D_ inf ]` witnessing Theorem 4.4. Both
    directions are
Scott-continuous (they are bundled `ScottMap`s), so this is the order-theoretic half of Scott's
homeomorphism. -/
noncomputable def theorem_4_4_orderIso : OrderIso (DInf D_0 j_0) (DInfFn D_0 j_0) :=
  (Equiv.mk (embInfInf D_0 j_0) (projInfInf D_0 j_0)
```

```
      (projInfInf_embInfInf D_0 j_0) (embInfInf_projInfInf D_0 j_0)).toOrderIso
    (embInfInf D_0 j_0).monotone (projInfInf D_0 j_0).monotone

end Thm44d

end LimitMaps

end Scott1972.ContinuousLattice
```